

Entregable C2-EP2

Nombre del Proyecto

Componente 2 Iteración 2 Especificación del proyecto

Nombre del equipo o consultoría

Escuela Superior de Cómputo, IPN

Borrador del 21 de febrero del 2017
(para revisión)

1. Planteamiento del problema	3
1.1. Antecedentes	3
1.2. Planteamiento del problema	4
1.3. Propuesta de solución	4
2. Definición del proyecto	6
2.1. Objetivo general	6
2.1.1. Objetivos específicos	7
2.2. Justificación	7
2.3. Alcance	8
2.4. Requerimientos	8
2.5. Modelo de información	8
2.6. Metodología de desarrollo	9
3. Estado del arte	10
3.1. Introducción	10
3.2. Estado del arte en técnicas de reconocimiento facial	11
3.2.1. Métodos tradicionales en reconocimiento facial	11
3.2.2. Redes Neuronales Convolucionales	12
3.3. Reconocimiento facial en el contexto escolar	13
3.3.1. Sistemas de gestión de asistencia	14
3.3.2. Sistemas de supervisión en exámenes	15
3.3.3. Control de acceso a universidades	16
3.4. Desafíos y limitaciones en el reconocimiento facial	18
3.4.1. Impacto de las variaciones de iluminación y pose	18
3.4.2. Efectos de oclusión	18
3.4.3. Suplantación de la identidad por medio de imágenes y vídeos	19

4. Marco Teórico	20
4.1. Evaluaciones a Título de Suficiencia (ETS)	20
4.1.1. Definición y contexto	20
4.1.2. Procedimiento para realizar un ETS	21
4.1.3. Problemáticas identificadas	21
4.2. Tecnologías de Identificación y Control de Acceso	22
4.2.1. Códigos QR	22
4.3. Desarrollo de la Aplicación Móvil	25
4.3.1. Tipos de aplicaciones móviles	25
4.3.2. Aplicaciones nativas	26
4.3.3. Aplicaciones Web	27
4.3.4. Aplicaciones Híbridas	27
4.3.5. Aplicaciones Progresivas Web Apps (PWA)	28
4.4. Elección de tecnología	29
4.4.1. Kotlin	29
4.4.2. Android	30
4.5. Herramientas de Desarrollo	30
4.5.1. Android Studio	30
4.5.2. Jetpack Compose	30
4.5.3. Gradle	32
4.5.4. Git y GitHub	32
4.6. Bases de datos (BD)	32
4.6.1. Bases de datos relacionales:	33
4.6.2. Bases de datos no relacionales:	34
4.6.3. Bases de datos vectoriales	34
4.7. Spring Boot	35
4.8. Java Persistence API (JPA)	36
4.9. Patrones de arquitectura de software	38
4.9.1. Arquitectura de capas	39
4.9.2. Arquitectura orientada a servicios	39
4.9.3. Arquitectura de micro servicios	39
4.9.4. El patrón modelo vista controlador	39
4.10. Redes neuronales artificiales	40
4.10.1. Estructura de una red neuronal artificial	40
4.11. Proceso de aprendizaje	41
4.11.1. Forward propagation	41
4.11.2. Backpropagation	41
4.11.3. Funciones de activación	41
4.12. Tipos de redes neuronales	42
4.12.1. Perceptrón	42
4.12.2. Perceptrón multicapa	42
4.12.3. Redes neuronales convolucionales	42
4.12.4. Redes neuronales recurrentes	42

4.13. Limitaciones	42
4.14. Reconocimiento facial	43
4.14.1. Pipeline de un sistema moderno de reconocimiento facial	43
4.14.2. Identificación de rostros	45
4.14.3. Verificación de rostros	45
4.14.4. Reconocimiento facial mediante verificación	45
4.15. Algoritmos para la detección de rostros y extracción de características	46
4.15.1. MTCNN (Multitask Cascaded Convolutional Networks)	46
4.15.2. FaceNet	48
5. Trabajo realizado	53
5.1. Aplicación Móvil	53
5.1.1. Tecnologías y herramientas utilizadas	53
5.1.2. Arquitectura MVVM (Model-View-ViewModel)	53
5.1.3. Componentes de las pantallas (Jetpack Compose y LaunchEffect)	54
5.2. Lógica de los ViewModels	56
5.2.1. Tecnologías y Componentes Clave en ViewModels	56
5.3. Lógica de las Instancias de Retrofit	58
5.3.1. Configuración de Instancias de Retrofit	58
5.4. Lógica de las Data Classes	60
5.4.1. Propósito y Características de las Data Classes	60
5.4.2. Ejemplos de Data Classes Implementadas	60
5.5. Sistema Web Django	61
5.6. Servidor Java Spring Boot	61
6. Pruebas	63
6.1. Pruebas de humo y detección temprana de errores	63
6.2. Pruebas de Integración	64
6.3. Pruebas de Subsistema	64
6.4. Pruebas de Aceptación: Encuestas de Usuario	65
6.5. Pruebas funcionales	66
6.5.1. Caso de prueba CP01 - Inicio de sesión de la app móvil	66
6.5.2. Caso de prueba CP02 - Consultar calendario escolar	67
6.5.3. Caso de prueba CP05 - Consultar ETS asignados	68
6.5.4. Caso de prueba CP06 - Mostrar información de los ETS asignados	69
6.5.5. Caso de prueba CP07 - Solicitar reemplazo de ETS	70
6.5.6. Caso de prueba CP08 - Consultar lista de alumnos inscritos a un ETS	72
6.5.7. Caso de prueba CP09 - Tomar asistencias a los ETS	73
6.5.8. Caso de prueba CP11 - Mostrar la foto e información del alumno	74
6.5.9. Caso de prueba CP12 - Consultar alumno mediante código QR de la credencial	75
6.5.10. Caso de prueba CP13 - Consultar alumnos mediante búsqueda por boleta	77
6.5.11. Caso de prueba CP14 - Consultar alumnos mediante búsqueda por nombre	78
6.5.12. Caso de prueba CP15 - Registrar ingreso a la instalación	80
6.5.13. Caso de prueba CP042 - Asignar docente de reemplazo	81

6.6. Pruebas al modelo de reconocimiento facial	83
6.6.1. Configuración experimental	83
6.6.2. Preparación del Conjunto de Datos LFW	84
6.6.3. Metodología de Evaluación	84
6.6.4. Curvas ROC y Análisis Visual	85
6.7. Pruebas al modelo de reconocimiento facial sobre conjuntos de datos personales	85
6.7.1. Directorio de Imágenes	86
6.7.2. Modelo de Reconocimiento Facial	86
6.7.3. Backend del Detector Facial	86
6.7.4. Métrica de Distancia y Umbral	86
6.8. Propósito del Experimento	87
7. Resultados	89
7.1. Alumnos	89
7.1.1. Satisfacción del usuario con la aplicación	89
7.1.2. Puntos de mejora	94
7.2. Docentes	95
7.2.1. Resultados de la encuesta: reconocimiento facial en la sección del docente	95
7.2.2. Comparación de Credenciales mediante Código QR	98
7.2.3. Efectividad contra suplantación de identidad	100
7.3. Personal de seguridad	102
7.3.1. Facilidad de uso del escaneo del código QR	102
7.3.2. Visualización de la información	103
7.3.3. Experiencia de usuario	103
7.3.4. Efectividad contra suplantación de identidad	104
7.4. Modelos de reconocimiento facial	106
8. Trabajo futuro	113

4.1. Patrones de detección de posición.	22
4.2. Patrones de alineación.	23
4.3. Patrones de temporización.	23
4.4. Información sobre la versión.	23
4.5. Información del formato.	24
4.6. Código de corrección de datos y errores.	24
4.7. Márgenes o zonas quietas.	24
4.8. Tipo de aplicaciones móviles.	26
4.9. Pipeline moderno de un sistema de reconocimiento facial.	43
4.10. Diferencia entre verificación e identificación de rostros.	46
4.11. Arquitectura de MTCNN.	47
4.12. Arquitectura general de FaceNet.	48
4.13. Aprendizaje basado en tripletas.	49
4.14. Conjunto de datos CASIA WebFace.	51
4.15. Algunos pares en el conjunto LFW.	52
5.1. Arquitectura MVVM.	54
6.1. Conjunto de imágenes mostrando un ejemplo de imagen de registro y su correspondiente imagen de prueba.	88
7.1. Tiempo del proceso del reconocimiento facial.	90
7.2. Frecuencia de resultados correctos del reconocimiento facial.	90
7.3. Seguridad del alumno con la precisión del reconocimiento facial.	91
7.4. Intentos necesarios para que el reconocimiento facial lo identificara correctamente.	91
7.5. Rapidez y fluidez del proceso del reconocimiento facial.	92
7.6. Utilidad de la información mostrada en los detalles del ETS.	92
7.7. Utilidad de las instrucciones mostradas para el ingreso de los ETS.	93

7.8. Comodidad del usuario al navegar por la app móvil.	93
7.9. Correcta paleta de colores.	94
7.10. Tiempo aproximado del proceso de reconocimiento facial para identificar a un alumno.	95
7.11. Frecuencia con la que el reconocimiento facial identificó correctamente al alumno, según la percepción de los docentes.	96
7.12. Nivel de seguridad de los docentes respecto a la precisión del reconocimiento facial.	96
7.13. Número promedio de intentos requeridos para la identificación correcta por reconocimiento facial, según los docentes.	97
7.14. Percepción de los docentes sobre la fluidez y rapidez del proceso general de verificación de identidad por reconocimiento facial.	97
7.15. Percepción de los docentes sobre la facilidad para escanear el código QR de la credencial con la app móvil.	98
7.16. Tiempo percibido por los docentes para la lectura del código QR y visualización de la información de la credencial.	98
7.17. Frecuencia con la que la app móvil leyó correctamente el código QR de la credencial, según la percepción de los docentes.	99
7.18. Percepción de los docentes sobre la claridad y legibilidad de la información mostrada tras escanear el código QR.	99
7.19. Número promedio de intentos para la lectura correcta del código QR por la aplicación, según los docentes.	100
7.20. Evaluación de docente sobre la efectividad del sistema de inteligencia artificial contra suplantación de identidad.	100
7.21. Distribución de respuestas sobre la facilidad de escaneo del código QR.	102
7.22. Distribución de respuestas sobre el tiempo de escaneo del código QR.	103
7.23. Evaluación de la experiencia de navegación en la aplicación.	104
7.24. Evaluación del esquema de colores por los usuarios.	104
7.25. Evaluación de usuarios sobre la efectividad del sistema QR contra suplantación de identidad.	105
7.26. Curvas ROC para las distintas configuraciones experimentales en el dataset LFW (subconjunto 1000 muestras).	106
7.27. Captura de pantalla de la ejecución de la matriz de confusión.	107
7.28. Captura de pantalla de la ejecución de la matriz de confusión.	108
7.29. Captura de pantalla de la ejecución de la matriz de confusión.	109
7.30. Captura de pantalla de la ejecución de la matriz de confusión.	110
7.31. Reporte final para la matriz de confusión.	111
7.32. Matriz de confusión final.	112

Índice de cuadros

4.1. Comparación tipos de aplicaciones móviles. Elaboración propia	29
4.2. Comparación entre bases de datos vectoriales y tradicionales	35

El presente documento es el reporte técnico final del Trabajo Terminal con número de registro 2025-A138, titulado:

"Prototipo de sistema de identificación de estudiantes basado en reconocimiento facial dentro de la Escuela Superior de Cómputo para evitar la suplantación de identidad durante la aplicación de exámenes a título de suficiencia mediante el uso de credenciales", elaborado por Alejandra De la Cruz De la Cruz, Luis Antonio Flores Esquivel, José Alfredo Jiménez Rodríguez y Daniel Martín Huertas Ramírez para obtener el grado de Ingeniero en Inteligencia Artificial.

Este trabajo fue dirigido por los profesores José Félix Serrano Talamantes y Ulises Vélez Saldaña, quienes fungieron como directores del proyecto.

El presente documento va dirigido a los sinodales, Andrés García Floriano, Chadwick Carreto Arellano y Saúl de la O Torres, con la finalidad de exponer de manera detallada los objetivos, el desarrollo, los resultados y conclusiones del trabajo realizado, así como sustentar su validez técnica y académica para la evaluación correspondiente.

En este documento se describe el diseño, implementación, pruebas y resultados de un sistema de identificación mediante reconocimiento facial, enfocado en asegurar la identidad de los estudiantes que presentan Exámenes a Título de Suficiencia en la Escuela Superior de Cómputo perteneciente al Instituto Politécnico Nacional.

Organización del documento

El presente documento se encuentra estructurado por los siguientes capítulos y anexos:

En el capítulo 1, se encuentran los antecedentes que permitieron identificar la problemática que rige este proyecto, lo que a su vez nos guió al análisis de esta misma para sustentar la propuesta de solución planteada a lo largo de este documento

En el capítulo 2, se presenta la definición del proyecto, donde se explica la justificación desde un contexto social, personal y académico del proyecto y además, se da una explicación breve y concisa sobre el alcance, los requerimientos, la arquitectura, las propiedades de software, el modelo de información y la metodología

de desarrollo.

En el capítulo 3, se encuentra el estado del arte, en donde se revisaron y analizaron trabajos anteriores con similitudes al proyecto a realizar, esto con el fin de realizar una comparación de las principales características e identificar posibles áreas de mejora.

En el capítulo 4, se presenta el marco teórico donde se explican los términos, descripciones y demás elementos importantes que se relacionan en menor o mayor medida con este trabajo a fin de tener un panorama general sobre la terminología utilizada.

En el capítulo 5, se presenta los detalles del trabajo realizado, es decir se explica cómo y con que tecnologías y métodos se implementó cada una de las secciones del sistema.

En el capítulo 6, se exponen las pruebas realizadas en la aplicación móvil a los casos de uso más importantes, para buscar errores para su posterior corrección y/o determinar si la funcionalidad y/o proceso se está realizando de manera satisfactoria o cumple con lo esperado, además, se muestran las pruebas hechas al sistema de reconocimiento facial.

En el capítulo 7, se presentan los resultados obtenidos de las encuestas hechas a los alumnos, docentes y personal de seguridad de la escuela superior de cómputo (ESCOM), sobre el funcionamiento de la aplicación, las opiniones sobre esta y si les parece útil o si creen que podría ayudar reducir el problema de la suplantación de identidad durante los ETS. También, se presentan los resultados de las pruebas de la red neuronal y sus matrices de confusión.

Finalmente, en el capítulo 8, se presentan las direcciones futuras y consideraciones identificadas para este proyecto, una vez culminada la fase de Trabajo Terminal actual. Si bien se ha logrado la implementación de la aplicación móvil y el sistema de gestión asociado, incluyendo el reconocimiento facial en tiempo real, este capítulo delinea las potenciales expansiones y mejoras que podrían llevarse a cabo en una eventual continuación del proyecto, más allá de los objetivos académicos de este Trabajo Terminal.

En el anexo A, se sustenta la metodología de trabajo empleada, se presentan los resultados del estudio de factibilidad realizado y el análisis del sistema que compone la propuesta de solución a fin de delimitar el alcance de este y los elementos a desarrollar, asimismo, se presenta el análisis de riesgos correspondiente.

En el Anexo B, se muestran las decisiones de diseño tomadas para este proyecto, esto se encuentra representado a través de diagramas para ofrecer una visualización y explicación del funcionamiento del sistema y las interacciones que existen entre los componentes que lo conforman. Estos incluyen el diagrama de estructura general del sistema, la estructura interna de los módulos, diagramas de clases, diagramas de componentes y diagramas de secuencia.

Planteamiento del problema

En este capítulo se describen los antecedentes y se delimita la problemática principal que guía el proyecto a realizar, así como los objetivos propuestos y la justificación de su desarrollo.

1.1. Antecedentes

En la actualidad, la Escuela Superior de Cómputo (ESCOM) cuenta con diversos mecanismos que permiten evaluar los conocimientos del alumnado, siguiendo un esquema de educación tradicional en el que los alumnos tienen que obtener un mínimo de calificación para acreditar una materia y, donde además, es necesario acreditar todas las materias que componen el plan de estudios de la carrera para poder egresar de esta [1].

Estos mecanismos de evaluación difieren unos de otros dependiendo del momento en el que se aplican, del valor sobre la calificación final del alumno y de la situación académica de este. Algunos ejemplos incluyen las evaluaciones ordinarias parciales, la entrega de proyectos, evaluaciones extraordinarias, etc.

Sin embargo, la *evaluación a título de suficiencia* también conocida como ETS, es un tipo de evaluación especial que se aplica en los planteles del Instituto Politécnico Nacional (IPN) y que permite a los alumnos acreditar materias que no hayan podido acreditar durante los periodos ordinarios y extraordinarios [2].

A pesar de ser una evaluación que se realiza en todos los planteles del IPN, no existe medio alguno que permita obtener información clara y precisa sobre esta, ya que la poca información que se puede encontrar en la red se limita a instructivos de su inscripción de distintos planteles.

Es debido a esto, y a su importancia en el proceso académico de cada una de las instituciones que conforman al IPN que se propuso la realización de este proyecto.

1.2. Planteamiento del problema

Actualmente, la inseguridad es una de las principales problemáticas de nuestro país, lo que obliga a las instituciones educativas a destinar recursos significativos para reforzar los protocolos de seguridad, tanto dentro como fuera de sus instalaciones. Estos esfuerzos buscan garantizar la integridad física y emocional de la comunidad educativa y propiciar un entorno seguro donde los estudiantes puedan desarrollar plenamente sus capacidades creativas e intelectuales [3].

Una de las principales preocupaciones de las instituciones educativas es el acceso no autorizado a sus instalaciones, ya que las brechas en los protocolos de control de acceso pueden derivar en situaciones que comprometen la seguridad de la comunidad escolar [4]. Según un estudio realizado en los planteles de Iztapalapa y Xochimilco de la Universidad Autónoma Metropolitana, los alumnos han sido víctimas de robos, tanto con violencia como sin ella, frecuentemente atribuidos al ingreso de personas ajenas a la institución. Por ejemplo, en el plantel de Iztapalapa, el 79.3 por ciento del alumnado considera que la inseguridad está relacionada con el acceso de personas externas [5].

En el caso de la Escuela Superior de Cómputo (ESCOM), la creciente matrícula estudiantil ha presentado desafíos adicionales en materia de seguridad. Entre las problemáticas más frecuentes se encuentra la suplantación de identidad durante los Exámenes a Título de Suficiencia (ETS) y los Exámenes a Título de Suficiencia Especiales. Este problema ocurre cuando personas externas o incluso estudiantes de la institución presentan exámenes en lugar de los alumnos registrados, ya sea mediante credenciales falsificadas o acuerdos entre estudiantes. Estas prácticas no solo afectan la transparencia del proceso educativo, sino que también ponen en riesgo la seguridad de la comunidad escolar al permitir el ingreso de personas cuya intención puede ser desconocida.

La falta de un control de acceso eficiente contribuye a este problema, ya que protocolos débiles permiten que las personas ingresen sin una verificación adecuada de su identidad. Actualmente, el ingreso a las instalaciones y a los exámenes en la ESCOM depende de métodos tradicionales que pueden ser fácilmente manipulados.

1.3. Propuesta de solución

Tomando como punto de partida las tecnologías que se describen en capítulo 3 la propuesta de solución a la problemática de la suplantación de la identidad durante las evaluaciones ETS en la ESCOM consta de dos partes.

En primer lugar se desarrollara una aplicación móvil con soporte para dispositivos Android que servirá como una herramienta tanto para el personal de seguridad como para los docentes para reforzar los protocolos de control de acceso a las instalaciones y verificación de la identidad de los alumnos respectivamente.

Al funcionar como una herramienta se pretende brindar la funcionalidad necesaria para identificar situaciones en las que los alumnos quieran acceder a las instalaciones aún cuando estos no tienen inscrito algún

ETS, en las fechas de aplicación de este. Por otro lado, también se busca que los docentes puedan tener los elementos necesarios para verificar la identidad de los estudiantes. Estos dos enfoques de la aplicación móvil serán abordados mediante la implementación de un sistema de consulta de alumnos que se conectara a la base de datos escolar de la institución para verificar los accesos y asistencia a los ETS según sea el caso, en este se mostrara la información necesaria para que tanto el personal de seguridad como los docentes puedan tomar la decisión de permitir el acceso o presentar el examen a los alumnos, esto último también permitirá registrar la asistencia de los alumnos a las evaluaciones para su posterior consulta.

Además de esto, el módulo de reconocimiento facial se encuentra pensando para servir como una capa de seguridad adicional que permita a los docentes verificar la identidad de los estudiantes cuando la información que estos obtienen de las consultas al sistema no son suficientes para tomar una decisión.

De igual forma la propuesta de solución contará con un módulo de notificaciones e informativa relacionada a los ETS inscritos, asignados y horarios de aplicación.

Finalmente, para que la aplicación pueda simular un escenario real se pretende realizar un sistema web para la gestión de los usuarios que componen al sistema es decir, alumnos, docentes, personal de seguridad y de exámenes ETS.

Definición del proyecto

Retomando lo dicho en la propuesta de solución, se llevó a cabo una aplicación móvil que sirve como herramienta para el personal de seguridad y el personal académico en temporadas de ETS para disminuir la suplantación de identidad dentro de la ESCOM.

La aplicación móvil cuenta con varios apartados dependiendo del usuario que la esté usando, ya sea un alumno, un personal académico o un personal de seguridad, sin embargo, la parte fundamental de esta aplicación es la del docente y la del alumno en la que pueden realizar el reconocimiento facial. El alumno puede hacer el reconocimiento facial a su persona para rectificar que no haya ningún problema con su pase de asistencia al momento de llegar a realizar su ETS. El personal académico cuenta con la parte de reconocimiento facial como una forma de pasarle la asistencia al ETS al alumno.

Es importante mencionar que el pase de asistencia no se realiza únicamente con el reconocimiento facial, se puede realizar mediante otras formas, como con un escaneo a la credencial del alumno o más fácil, si el personal académico ya conoce al alumno de antemano, lo puede especificar así al momento de registrar la asistencia al alumno.

Por parte del personal de seguridad, puede registrar el ingreso a la unidad académica (ESCOM), por medio del escaneo de la credencial del alumno.

La aplicación móvil cuenta con más secciones, tales como la visualización de los datos de los ETS, el calendario escolar del IPN, una sección de mensajes para que se los alumnos se puedan comunicar con el personal académico y para que el personal académico se pueda comunicar tanto con los alumnos como con otro personal académico.

2.1. Objetivo general

Desarrollar un sistema de autenticación y control de acceso de alumnos con el motivo de evitar posibles casos de suplantación de identidad durante la aplicación de Exámenes a Título de Suficiencia (ETS) en la Escuela Superior de Cómputo (ESCOM), mediante el uso de tecnologías de reconocimiento facial, dispositivos móviles y credenciales escolares.

2.1.1. Objetivos específicos

- Explorar el estado del arte en cuanto a técnicas para el reconocimiento facial y recuperación de datos (rostros).
- Evaluar distintos algoritmos y técnicas de reconocimiento facial para determinar aquellas que se ajusten mejor a las necesidades y recursos del proyecto.
- Desarrollar una aplicación móvil para estudiantes y docentes que integre la tecnología de reconocimiento facial y proporcione un acceso seguro a los servicios relacionados con los exámenes (consulta de horarios), incluida la verificación de la identidad.
- Diseñar e implementar un módulo de reconocimiento facial capaz de verificar la identidad de los estudiantes durante las fechas de aplicación de los ETS, garantizando que solo los estudiantes inscritos puedan presentar el examen.
- Implementar una sección dentro de la aplicación móvil que permita reforzar el control de acceso a las instalaciones durante los días de aplicación de ETS con la ayuda de códigos QR.
- Desarrollar un sistema web básico para la gestión de ETS y que permita la integración de una API para simular escenarios reales con el fin de demostrar y evaluar la funcionalidad del sistema.
- Evaluar el desempeño del prototipo de sistema de control de acceso en cuanto a precisión, confiabilidad y seguridad.

2.2. Justificación

La implementación de un sistema de control de acceso basado en técnicas biométricas, como el reconocimiento facial, en conjunto con el uso de métodos de autenticación convencionales y ya establecidos como lo es el uso de las credenciales escolares, representa una innovación significativa en el ámbito educativo, particularmente en el contexto de los Exámenes a Título de Suficiencia (ETS). Este proyecto se justifica no solo por su capacidad de abordar retos específicos, sino también por la oportunidad que ofrece para explorar y aplicar tecnologías emergentes en el campo de la Inteligencia Artificial (IA) y la Visión por Computadora.

El uso de reconocimiento facial como método biométrico es especialmente relevante en el contexto educativo porque aporta precisión y fiabilidad en la verificación de identidad. Además, es un método de autenticación no invasivo y difícil de vulnerar en tiempo real ante la presencia del personal humano. Según estudios recientes, las tecnologías de autenticación biométrica presentan tasas de error mucho menores comparadas con métodos tradicionales, como contraseñas o credenciales físicas, las cuales pueden ser manipuladas fácilmente [6].

Desde una perspectiva educativa y tecnológica, este proyecto también tiene un valor en la formación de profesionales en el área de la inteligencia artificial. Implementar un sistema basado en Visión por Computadora y aprendizaje profundo permite a los desarrolladores ampliar sus conocimientos en tecnologías avanzadas. Estas habilidades son esenciales en un entorno global donde la IA está transformando rápidamente múltiples

sectores, incluyendo el educativo. Incluso en algunos países como Estados Unidos y Australia este tipo de tecnologías ya han sido implementadas para cuestiones relacionadas con la seguridad dentro de los planteles [4].

En términos de beneficios para la comunidad que integra a la ESCOM, este proyecto no solo refuerza la seguridad y transparencia de los procesos evaluativos, sino que también genera confianza en la comunidad educativa. Profesores, estudiantes y personal administrativo podrán realizar sus actividades en un entorno más seguro, minimizando riesgos asociados con el acceso no autorizado [5].

Finalmente, la integración de estas tecnologías en dispositivos tan accesibles como los son los teléfonos celulares permite su escalabilidad a otros procesos académicos que requieran de la validación de la identidad de los alumnos o de llevar un control sobre las personas que entran y salen de las instalaciones, lo que a su vez puede influir en la adopción de este tipo de tecnologías en otros sectores públicos.

2.3. Alcance

Este proyecto tiene como objetivo realizarse dentro de la Ciudad de México, específicamente dentro de la ESCOM del Instituto Politécnico Nacional.

Específicamente, está pensado para que tanto el personal académico (específicamente para los docentes), el personal de seguridad y los alumnos, puedan usar la aplicación móvil planteada dentro de los periodos de ETS, tanto ETS ordinario como especial.

2.4. Requerimientos

El análisis de requerimientos arrojó un total de 26 requisitos de usuario (RU), distribuidos entre ambas plataformas: 17 para la aplicación móvil y 9 para el sistema web. Estos requerimientos fundamentales sirvieron como base para el diseño del sistema, generando 17 casos de uso para la plataforma móvil y 23 para la versión web.

Para consultar la documentación completa relacionada con este proceso, se recomienda revisar los siguientes apartados del Apéndice A:

- **A.1 Modelo del Alcance:** Contiene el detalle completo de los requerimientos de usuario identificados.
- **A.4 Modelo Dinámico:** Incluye la especificación completa de casos de uso para ambas plataformas.

2.5. Modelo de información

Para realizar la parte del modelo de información se realizó un análisis que iba desde cubrir los datos de los alumnos, del personal académico y del personal de seguridad, así como las unidades académicas en las que estos trabajaban. Dentro de las unidades académicas también se vio que tuvieran programas académicos y dentro de estos, unidades de aprendizaje.

Todo el análisis que se llevó a cabo se pensó para cubrir todo el proceso realizado desde que se inscribe un alumno a alguna unidad académica dentro del Instituto Politécnico Nacional, hasta que un alumno reprueba alguna unidad de aprendizaje.

En el anexo ?? se detallan todas las entidades que se abstraieron del análisis de todo este proceso, a su vez, se detalla cada una de las variables que estas entidades contienen.

2.6. Metodología de desarrollo

Se adoptó el modelo en espiral como metodología principal, dado su enfoque iterativo y adaptable que permite incorporar mejoras progresivas, gestionar riesgos y ajustar los requerimientos durante cada fase. Este modelo combina la estructuración del enfoque en cascada con la flexibilidad para evolucionar conforme a las necesidades del proyecto.

El proceso se organizó en seis iteraciones clave, desde la investigación inicial hasta la implementación de prototipos funcionales. En cada ciclo se definieron objetivos específicos, se evaluaron riesgos y se generaron productos intermedios para garantizar el avance controlado del sistema. Para una descripción detallada de la metodología aplicada, consúltase el **Apéndice A.3 Metodología**.

3.1. Introducción

Antes de ahondar en la propuesta de solución a la problemática que guía este proyecto, resulta importante tener en consideración la forma en la que esta ha sido atacada previamente y cómo el reciente desarrollo tecnológico ha permitido que tareas como el reconocimiento facial puedan implementarse en distintos ámbitos de nuestra vida cotidiana.

En años recientes, los avances tecnológicos han provocado el surgimiento de nuevas áreas de estudio enfocadas en la gestión de la información. La Inteligencia Artificial (IA) es un área que representa un campo interesante y en constante crecimiento derivado del gran volumen de datos generados diariamente y a su facilidad para proponer soluciones a problemáticas que hasta hace unos años parecían difíciles o imposibles de resolver.

Los esfuerzos por parte de los gobiernos y autoridades de cada uno de los grupos y comunidades que conforman a la sociedad para garantizar la seguridad de sus individuos ha dado lugar a la adopción de nuevas tecnologías con la finalidad de contar con protocolos de seguridad más robustos. Es por esta razón que cada vez es más común encontrar la incorporación de tecnologías de última generación en sistemas para manejar el control de acceso a las instalaciones, verificar la identidad de los integrantes de su comunidad o simplemente garantizar un monitoreo continuo que permita la detección oportuna de anomalías y prevención de delitos.

La visión por computadora es un subcampo de la inteligencia artificial que ha tomado relevancia en los últimos años gracias a sus casos de uso que han permitido automatizar en gran medida la forma en la que se proponen y desarrollan soluciones en materia de seguridad y vigilancia. De aquí se desprende el reconocimiento facial, una rama fundamental de la visión por computadora que permite identificar o verificar la identidad de un individuo a partir de sus características faciales en imágenes o videos. Este campo ha visto

cambios significativos impulsados en gran medida por el desarrollo del aprendizaje profundo, especialmente las redes neuronales convolucionales CNN.

Teniendo todo esto en mente es que se presenta el siguiente estado del arte, el cual proporciona una revisión breve sobre algunas técnicas usadas para llevar a cabo el reconocimiento facial y la detección de rostros, un paso importante dentro del flujo de trabajo de cualquier sistema de reconocimiento facial. Además, se prestara especial atención a los proyectos que integran estas tecnologías en un contexto estudiantil/académico, así como los desafíos y limitaciones de este campo de estudio.

3.2. Estado del arte en técnicas de reconocimiento facial

Como ya se comentó, el reconocimiento facial es una rama de la visión por computadora que a su vez representa un campo de estudio dentro de la Inteligencia Artificial y, a pesar de que recientemente este último concepto ha ganado cierta popularidad la realidad es que no es algo completamente nuevo, por lo que muchas de las técnicas que hoy en día se utilizan difieren mucho de los métodos tradicionales que permitieron en un inicio dar inicio a la investigación en materia de identificación y verificación de la identidad de las personas a través de sus características faciales.

3.2.1. Métodos tradicionales en reconocimiento facial

Antes del auge del aprendizaje profundo y las redes neuronales se utilizaban diversas técnicas para llevar a cabo el reconocimiento facial. Algunas de estas técnicas incluyen:

- **Análisis de componentes principales (PCA) / Eigenfaces**

Es una técnica estadística de reducción de dimensionalidad que transforma un conjunto de variables posiblemente correlacionadas en un conjunto de variables linealmente no correlacionadas, llamadas componentes principales. En el contexto del reconocimiento facial, estos componentes principales se denominan "eigenfaces".

A grandes rasgos, consiste en aplicar PCA a un conjunto de imágenes de rostros de entrenamiento para encontrar los vectores propios que capturen la mayor varianza en la distribución de los datos.

Cuando se busca que un nuevo rostro sea identificado, este se proyecta en este subespacio definido por las eigenfaces para obtener un conjunto de pesos que luego se comparan con los pesos de los rostros conocidos.

- **Análisis Discriminante Lineal (LDA) / FisherFaces**

Es una técnica clásica.^{en} el reconocimiento de patrones que busca una combinación lineal de características que mejor caracterice o separe dos o más clases de objetos. En el reconocimiento facial, se utiliza para crear "fisherfaces".

A diferencia de PCA, que busca un espacio lineal ortogonal para codificar la información, LDA busca un espacio lineal separable que maximice el poder discriminatorio. Su objetivo es maximizar la relación entre la dispersión entre clases (separando a diferentes individuos) y la dispersión dentro de la clase (agrupando imágenes del mismo individuo).

- **Patrones binarios locales (LBP)**

Esta técnica consiste en un descriptor de textura que se encarga de etiquetar los píxeles de una imagen al umbralizar el vecindario de 3x3 de cada pixel con el valor del pixel central, generando un número binario LBP.

3.2.2. Redes Neuronales Convolucionales

Durante la última década el desarrollo del reconocimiento facial se ha visto impulsado en gran medida por el las CNNs, un tipo de arquitectura de aprendizaje profundo que buscan aprender características jerárquicas directamente de los valores de píxeles brutos de una imagen.

Aunque los elementos que comprenden este tipo de arquitectura se explicaran con mayor detalle en el capítulo para este punto es importante tener en cuenta que estas redes se encuentran formadas por un conjunto de capas convolucionales cuyo propósito es extraer las características de las imágenes que reciben como entrada, capas de pooling que permiten reducir las dimensionares de los mapas de características que se van obteniendo en cada convolución y capas completamente conectadas las cuales reciben como entrada las características extraídas para llevar a cabo tareas de clasificación.

Un punto importante de las redes neuronales convolucionales es su naturaleza experimental ya que dependiendo de la arquitectura a implementar se pueden obtener resultados completamente distintos, además, existen algunas variaciones que permiten que estas redes se ajusten para resolver tareas en específico, como lo es el caso del reconocimiento facial.

A continuación se presentan algunas arquitecturas importantes en esta tarea:

- **FaceNet**

Desarrollado por Google, FaceNet aprende un mapeo de imágenes faciales a un espacio euclidiano compacto de 128 dimensiones, donde las distancias corresponden directamente a una medida de similitud facial. Utiliza una CNN profunda entrenada con una función de pérdida de tripletas para asegurar que las imágenes de la misma persona estén cerca entre sí, mientras que las imágenes de personas

diferentes estén separadas por un margen específico.

- **VGG-Face**

Es un modelo de red neuronal convolucional (CNN) desarrollado por la Universidad de Oxford para reconocimiento facial. Basado en la arquitectura VGG, utiliza 16 o 19 capas profundas para extraer características faciales de imágenes. Entrenado en un gran conjunto de datos de rostros, es efectivo para tareas de verificación e identificación facial, destacando por su capacidad para generar representaciones robustas de rostros.

- **ArcFace**

Es un método avanzado de reconocimiento facial que utiliza una pérdida de margen angular (Additive Angular Margin Loss) para mejorar la discriminación entre clases. Desarrollado por InsightFace, ArcFace optimiza la separación de características faciales en un espacio de alta dimensión, logrando mayor precisión en tareas de identificación y verificación facial.

Otro punto a destacar de este modelo de reconocimiento facial es su capacidad para limpiar grandes conjuntos de datos, al implementar una estrategia basada en sub-centros.

3.3. Reconocimiento facial en el contexto escolar

La escuela es un espacio en el que diariamente interactúan un gran número de personas, desde estudiantes hasta profesores y personal académico y externo. Es por esta razón que el garantizar la seguridad de sus individuos se convierte en una prioridad de cara a fomentar su correcto desarrollo.

En este contexto escolar, existen dos conceptos relacionados con seguridad, por un lado tenemos a la seguridad perimetral y la seguridad interna. La seguridad perimetral hace referencia al conjunto de mecanismos y sistemas relativos al control de acceso físico de las personas a las instalaciones así como a la detección y prevención de intrusiones, por otra parte, la seguridad interna hace referencia a protocolos de seguridad establecidos por cada institución en materia de prevención y gestión de situaciones de riesgo que podrían llegar a presentarse dentro de los planteles. Una falla en la seguridad perimetral puede suponer una brecha en la seguridad interna.

Una de las brechas de seguridad que guían este trabajo consiste en la suplantación de la identidad, un delito que se da cuando alguien se hace pasar por otra para obtener algún tipo de beneficio, puede darse en distintos contextos y, en el caso de una escuela puede dar lugar a fraude durante evaluaciones o el acceso no autorizado a ciertos lugares dentro de las instituciones poniendo en riesgo la seguridad de sus individuos.

Con esta información en mente es que se exploró la forma en la que el reconocimiento facial se ha implementado en el contexto escolar, resaltando sus implementaciones en materia de seguridad y en otros procesos como el control de asistencia.

3.3.1. Sistemas de gestión de asistencia

Este tipo de sistemas buscan automatizar y mejorar la eficiencia de los procesos existentes para el registro de la asistencia de los alumnos.

Algunas de las problemáticas identificadas en los trabajos revisados mencionan que el proceso tradicional para la toma de asistencia de los alumnos consume mucho tiempo, es propenso a errores y, en algunos casos, es vulnerable a casos de suplantación de la identidad.

La forma en la que se aplica el reconocimiento facial en esta tarea incluye la automatización de la asistencia de los alumnos usando dicha tecnología. Estos sistemas de reconocimiento facial toman una foto de un estudiante y, de ser identificado, se registra su asistencia automáticamente.

Algunos de los trabajos que siguen este enfoque incluyen:

- **ATTENDANCE MANAGEMENT SYSTEM USING FACE RECOGNITION [7]**

Un sistema para automatizar el registro de asistencia de los alumnos usando técnicas de reconocimiento facial.

La principal diferencia de este proyecto con respecto a los demás recae en que no hace uso de técnicas de aprendizaje profundo como las bien conocidas arquitecturas de redes neuronales que en la actualidad lideran el campo; por lo tanto, usa técnicas más simples como el algoritmo Haar Cascade para la detección de rostros y Local Binary Pattern Histogram para llevar a cabo el reconocimiento facial.

A pesar de no implementar tecnologías más robustas, los resultados presentados indican que el sistema se desempeña bien ante cambios en la iluminación y en la pose de los alumnos.

Finalmente, la principal desventaja de esta propuesta es que únicamente cuenta con el desarrollo de una interfaz gráfica en tkinter, lo que puede dificultar su futura implementación en un contexto académico.

- **Sistema de monitoreo y control de asistencia de estudiantes usando Faster R-CNN [8]**

El sistema propuesto en este proyecto utiliza arquitecturas basadas en redes neuronales convolucionales CNN para la detección de rostros (Faster R-CNN) y FaceNet para la parte del reconocimiento facial.

Algunos detalles de su implementación son la integración de análisis de videos en tiempo real para el registro de asistencia, detección de anomalías así como la integración de notificaciones y datos centralizados para la creación de reportes.

- **SISTEMA WEB CON RECONOCIMIENTO FACIAL PARA LA ESCUELA “UNIDAD EDUCATIVA ULPIANO NAVARRO [9]**

La implementación de este sistema en un contexto escolar permitió registrar los accesos y salidas de los estudiantes a la institución y en consecuencia, eliminar los pases de asistencia en las aulas.

Nuevamente, vemos la aplicación de las redes neuronales convolucionales usando la biblioteca de face-api y para la parte de la detección de rostros se usó OpenCv cuyos algoritmos son más ligeros en comparación de otras propuestas como Faster R-CNN o MTCNN.

Finalmente, la principal desventaja del sistema es obvia, al ser un sistema web requiere de una conexión continua a internet, además, el uso de una computadora lo que puede aumentar los tiempos destinados para la toma de asistencia de alumnos en condiciones donde los equipos de cómputo no cuenten con los requisitos necesarios.

3.3.2. Sistemas de supervisión en exámenes

Derivado de la pandemia del covid-19 y de la necesidad de continuar con la educación, muchas de las escuelas retomaron sus actividades mediante clases en línea. Sin embargo, un punto importante en el proceso educativo como lo son las evaluaciones tuvieron que adaptarse para funcionar bajo este nuevo esquema a distancia [10].

Es por esta razón que algunos de los trabajos que se presentan a continuación fueron propuestos para contar con una herramienta que permitiera autenticar de forma continua a los estudiantes durante las evaluaciones en línea previniendo así, los casos de fraude.

Además, se encontraron trabajos que por el contrario, buscan prevenir el fraude durante evaluaciones presenciales, un enfoque que aunque comparte la implementación de reconocimiento facial, difiere en la práctica de los sistemas empleados para la autenticación en línea.

- **A Novel Deep Learning-based Online Proctoring System using Face Recognition, Eye Blinking, and Object Detection Techniques [11]**

Como el nombre del proyecto sugiere, este sistema hace uso de técnicas de reconocimiento facial para supervisar los exámenes en línea usando el detector de rostros HOG (Histogram of Oriented Gradients) y una implementación del algoritmo dlib para la parte del reconocimiento facial.

Para evitar que el sistema sea vulnerable al uso de imágenes estáticas, se incorporó la detección de parpadeo ocular y la detección de objetos usando el algoritmo YOLOv3 permitiendo identificar celulares, tabletas e incluso libros.

- **Face Recognition Classification in Proctored Exam [12]**

Este sistema sigue un enfoque similar al proyecto anterior al combinar técnicas de reconocimiento facial junto con detección de objetos para identificar el uso de dispositivos móviles y, en este caso el conteo de personas.

Dentro de los algoritmos utilizados se destaca YOLOv3 para la detección de objetos, la detección de puntos faciales clave mediante Dlib junto con OpenCV para realizar un seguimiento ocular que permitiera obtener una estimación de la mirada de los alumnos y la detección del movimiento de la boca.

Dado que esta propuesta está pensada para funcionar durante evaluaciones en línea, se integra un sistema de alertas que notifica a los evaluadores de posibles casos que requieran de su atención.

- **Reconocimiento facial para la identificación de los alumnos en exámenes finales en la modalidad presencial de la Universidad Continental [13]**

Finalmente, este proyecto trabaja bajo otro contexto para integrar las ventajas del reconocimiento facial durante evaluaciones presenciales para evitar posibles casos de suplantación de la identidad y en consecuencia fraude.

Los algoritmos utilizados incluyen Haar Cascade para la detección de rostros y la biblioteca de face-recognition que contiene implementaciones de arquitecturas basadas en CNN para el reconocimiento de rostros, aunque en el trabajo no se especifica la arquitectura final empleada.

A diferencia de otros trabajos revisados, esta es una implementación más completa que combina precisamente el desarrollo del sistema de reconocimiento facial con un sistema web integral que permite el registro, visualización y administración de estudiantes, lo que puede facilitar su uso en un entorno real.

3.3.3. Control de acceso a universidades

Llevar un control de quien accede a las universidades permite reforzar los protocolos de seguridad perimetral y evitar posibles accesos no autorizados que pueden desembocar en problemas más grandes, además, en algunos casos, la implementación de estos sistemas puede reducir los tiempos asociados a la gestión de acceso.

La principal problemática identificada por los trabajos revisados resalta las limitaciones de los sistemas de control de acceso convencionales basados en el uso de contraseñas e identificaciones escolares las cuales son fácilmente vulnerables.

La integración del reconocimiento facial en los sistemas de control de acceso permiten identificar a los individuos analizando sus características biométricas faciales, proporcionando un método de entrada sin con-

tacto y más robusto.

Algunos trabajos que han explorado la implementación de esta tecnología para el desarrollo de sistemas de control de acceso en universidades son::

- **Facial Recognition-Based Entry System for Student Residence Halls: Enhancing Security and Accessibility [14]**

Un sistema de control de acceso implementado a través de webcams y con una posterior integración en sistemas de circuito cerrado (CCTV) para identificar estudiantes haciendo uso de sus características faciales en tiempo real.

Una vez que se identifico al estudiante, se guarda en un archivo .csv junto con la fecha y hora de registro, en caso de que el estudiante no se encontrará dado de alta en el sistema se identifica como desconocido

.

Dentro de los algoritmos empleados para la detección y generación de embeddings se emplearon los algoritmos MTCNN y FaceNet-128.

Algunas limitantes de este proyecto involucra la necesidad de contar con cámaras de alta calidad y la dependencia de un equipo de cómputo que pueda soportar un uso intensivo del CPU.

- **Facial Recognition System for Access Control through the Application of Convolutional Neural Networks [15]**

Aunque el título del proyecto hace referencia a un sistema de control de acceso, no se especifica como es que fue implementado, sino que se presentan los resultados obtenidos tras aplicar una estrategia de Transfer Learning tomando como base el modelo pre-entrenado VGG16.

La aportación más importante de este proyecto recae en los resultados obtenidos de la experimentación sobre la arquitectura base del modelo VGG16 lo que plantea la idea de que cada institución puede tener su propio sistema de control de acceso con reconocimiento facial.

- **Sistema de control de acceso biométrico mediante reconocimiento facial con técnicas de vivacidad [16]**

El proyecto es un sistema de control de acceso biométrico basado en reconocimiento facial, diseñado para autenticar usuarios en tiempo real utilizando técnicas de detección de vivacidad, evitando fraudes como el uso de fotografías o videos.

Este sistema, implementado en entornos como instituciones educativas y empresas, se enfoca en garantizar un acceso seguro y eficiente. Sus principales características incluyen una arquitectura modular,

integración sencilla y énfasis en la privacidad de los datos, haciéndolo ideal para escenarios que requieren altos estándares de seguridad.

Debido a su enfoque novedoso que incluye identificar que el individuo tenga la mirada fija en la cámara junto con la detección y conteo de parpadeo, su principal deficiencia es la necesidad de obtener imágenes de alta calidad lo que requiere el uso de hardware especializado, dificultando su implementación en entornos poco controlados.

3.4. Desafíos y limitaciones en el reconocimiento facial

A pesar de los avances recientes y de los resultados obtenidos por los sistemas de reconocimiento facial, aun presenta desafíos que en ocasiones pueden limitar su implementación en escenarios del mundo real y por ende, su adopción en espacios como las instituciones educativas.

3.4.1. Impacto de las variaciones de iluminación y pose

Desde los inicios de las técnicas de reconocimiento facial, las variaciones de iluminación y poses ha representado un desafío significativo que impacta en la precisión de estos sistemas. [17]

En primer lugar, los cambios de iluminación pueden alterar la apariencia facial de manera drástica especialmente en aquellos modelos que trabajan sobre imágenes a color.

Por otra parte, las variaciones en las poses modifican las características faciales visibles, por lo que si el sistema no aprendió a generalizar las características faciales de una persona desde distintos ángulos, entonces será difícil lograr una correcta identificación.

3.4.2. Efectos de oclusión

La oclusión ocurre cuando elementos como máscaras, gafas o bufandas obstruyen el rostro de las personas. Recordando que, en su gran mayoría, los sistemas de reconocimiento facial tienden a extraer o transformar las características más relevantes del rostro de una persona, por lo que, en la mayoría de los casos, la calidad de las imágenes es un punto importante a considerar. [18]

Cuando un rostro se encuentra obstruido, el rendimiento de los sistemas baja, lo que lo hace un desafío relevante en escenarios del mundo real donde muchas personas usan este tipo de accesorios diariamente.

Algunos de los esfuerzos para reducir el impacto de estos efectos en los sistemas de reconocimiento facial destacan la recuperación basada en la oclusión en donde los sistemas son entrenados con datos que presenta oclusión, mejorando su desempeño ante estas situaciones.

3.4.3. Suplantación de la identidad por medio de imágenes y vídeos

Finalmente, los sistemas de reconocimiento facial son susceptibles a ataques de suplantación por medio de imágenes y vídeos, el gran problema aquí es que este tipo de sistemas recibe como entrada una imagen de un rostro, sin embargo, después de esto no cuentan con una validación que permita identificar si la imagen proviene de una fotografía a otra fotografía, un vídeo o si realmente fue tomada a una persona. [11]

Para contrarrestar este problema, algunos sistemas integran técnicas para la detección de vida y el análisis de movimiento para garantizar que las imágenes que reciben como entrada los modelos de reconocimiento facial han sido evaluadas ante posibles casos de suplantación de identidad.

Actualmente, las instituciones educativas enfrentan desafíos significativos en cuanto a la protección de la identidad estudiantil y la seguridad académica, especialmente durante evaluaciones de alta importancia académica. En el Instituto Politécnico Nacional (IPN), particularmente en la Escuela Superior de Cómputo (ESCOM), los Exámenes a Título de Suficiencia (ETS) constituyen una oportunidad para que los estudiantes acrediten asignaturas pendientes. No obstante, la ausencia de mecanismos de autenticación robustos ha facilitado casos de suplantación de identidad, comprometiendo así la integridad académica y la seguridad dentro del plantel.

Este proyecto propone el desarrollo de un sistema de identificación y control de acceso basado en reconocimiento facial y códigos QR, integrado en una aplicación móvil para dispositivos Android. La solución busca no solo prevenir la suplantación de identidad, sino también facilitar y agilizar los procesos de verificación por parte de docentes y personal de seguridad.

Para sustentar técnicamente esta propuesta, el marco teórico explora conceptos clave como los ETS, tecnologías biométricas, desarrollo de software móvil y arquitecturas de sistemas, proporcionando así una base sólida para el diseño e implementación del prototipo.

4.1. Evaluaciones a Título de Suficiencia (ETS)

4.1.1. Definición y contexto

En el Instituto Politécnico Nacional (IPN), incluyendo unidades académicas como la Escuela Superior de Cómputo (ESCOM), la acreditación de cada unidad de aprendizaje se realiza semestralmente a través de 3 evaluaciones ordinarias. Si un alumno no acredita alguna unidad de aprendizaje, tendrá la oportunidad de presentar una evaluación extraordinaria. Estos procedimientos están detallados en el programa de estudios y se encuentran especificados en el calendario académico.

El alumno que no logre acreditar una o más de las unidades de aprendizaje en la que se haya inscrito podrá optar por acreditarlas mediante Evaluación a Título de Suficiencia, de acuerdo con lo establecido en el

Artículo 39 del Reglamento Interno del IPN, que señala:

“La evaluación del aprendizaje se llevará a cabo a través de exámenes ordinarios, extraordinarios y a título de suficiencia, cuyos requisitos y procedimientos de elaboración, presentación y exención, así como de otros mecanismos de evaluación continua, se realizarán en los términos que fijen los planes y programas de estudio, el presente Reglamento y los reglamentos respectivos.”

Existen dos rondas de ETS:

- ETS Ordinario: Esta es la primera oportunidad que tiene el alumno para acreditar la materia en la que no obtuvo una calificación aprobatoria. Los ETS ordinarios generalmente se aplican al finalizar el semestre, permitiendo al estudiante demostrar sus conocimientos sin necesidad de repetir el curso completo.
- ETS Especiales: Si el alumno no acredita la materia en el ETS ordinario, puede optar por presentar un ETS Especial. Esta es la segunda oportunidad que tiene el alumno para pasar la materia. Esta evaluación adicional suele programarse el primer viernes del nuevo semestre, brindando al estudiante una opción rápida para regularizar sus situación académica y continuar avanzando en su plan de estudios.

4.1.2. Procedimiento para realizar un ETS

- Pagar en caja, verificar que estén correctos los siguientes datos: Nombre, Boleta, Carrera y Número de unidades de aprendizaje.
- Acudir a ventanilla de gestión escolar para generar créditos en el “SAES”.
- Una vez generados los créditos, inscribe las unidades de aprendizaje en la página del “SAES”.
- Entregar en ventanilla de gestión escolar, el comprobante de inscripción de ES generador por SAES, y el recibo de pago para dar fin a la inscripción al ETS.
- Acudir el día y la hora establecida en el calendario.

4.1.3. Problemáticas identificadas

- Suplantación de identidad: La verificación manual de credenciales es vulnerable a fraudes, ya que no existe un sistema automatizado que valide la autenticidad del estudiante.
- El personal de seguridad no cuenta con herramientas para verificar si un alumno está inscrito en un ETS el día de su aplicación.

Estas limitaciones reflejan la importancia de implementar un sistema integral que incorpore tecnologías como el reconocimiento facial y los códigos QR, con el propósito de asegurar procesos más seguros, eficientes y transparentes durante la aplicación de los ETS.

4.2. Tecnologías de Identificación y Control de Acceso

4.2.1. Códigos QR

Actualmente, las credenciales del Instituto Politécnico Nacional (IPN) incluyen un código QR que, al ser escaneado, proporciona acceso a la información del estudiante. Este código QR contiene datos esenciales como el nombre completo, número de boleta, carrera en la que está inscrito y su fotografía. Además, en algunas unidades académicas del IPN, los códigos QR se utilizan como medio de verificación de identidad para el ingreso a las instalaciones mediante torniquetes electrónicos.

En la elaboración de nuestro trabajo terminal, el código QR desempeña un papel fundamental en la corroboración de la identidad de los estudiantes. Dado que las credenciales escolares contienen un código QR, se puede aprovechar su capacidad para almacenar y transmitir información de manera segura y rápida. Este enfoque permite verificar los datos del estudiante mediante un escaneo simple, reduciendo el tiempo necesario para validar su identidad. Al escanear el código QR de la credencial, el sistema accede a los datos del alumno, lo cual facilita confirmar si coinciden con la persona que se presenta al Examen a Título de Suficiencia (ETS).

Un código QR es un tipo de código de barras bidimensional que puede ser leído por teléfonos inteligentes o dispositivos especializados en su lectura. Al escanear un código QR, los dispositivos pueden acceder directamente a mensajes de texto, correos electrónicos, sitios web, números de teléfono, entre otros [19].

Anatomía y funcionamiento

A continuación, se describen los principales componentes estructurales de un código QR:

Patrones de detección de posición Estos patrones se encuentran en tres de las esquinas del código QR. Gracias a ellos, el escáner puede reconocer y leer el código rápidamente, ya que indican la orientación del mismo y facilitan su detección.



Figura 4.1: Patrones de detección de posición.

Patrones de alineación Estos patrones permiten corregir distorsiones cuando el código QR se imprime sobre superficies curvas. Su tamaño y cantidad varían dependiendo de la cantidad de información almacenada.



Figura 4.2: Patrones de alineación.

Patrones de temporización La alternancia de módulos negros y blancos determina el sistema de información, también conocido como cuadrícula de datos. Estas líneas permiten al escáner identificar la estructura del código.



Figura 4.3: Patrones de temporización.

Información sobre la versión Estos marcadores indican cuál de las 40 versiones del código QR se está utilizando. Generalmente, las versiones utilizadas van de la 1 a la 7.



Figura 4.4: Información sobre la versión.

Información del formato Este componente contiene datos sobre la tolerancia a errores y el patrón de enmascaramiento. Su función es facilitar el proceso de escaneo.



Figura 4.5: Información del formato.

Código de corrección de datos y errores El sistema de corrección de errores comparte espacio con los datos almacenados. Esto permite recuperar información incluso si una parte del código está dañada o ilegible [19].



Figura 4.6: Código de corrección de datos y errores.

Márgenes Los márgenes o zonas quietas rodean el código QR y permiten que el lector distinga claramente sus límites. Son fundamentales para que el escáner identifique correctamente el código.



Figura 4.7: Márgenes o zonas quietas.

Fiabilidad y aplicaciones en seguridad

Los códigos QR están diseñados para mantener su legibilidad incluso si están parcialmente dañados u oscurecidos. Esto es posible gracias a su capacidad de corrección de errores, que inserta la información varias veces en el patrón del código. Dependiendo del nivel de seguridad utilizado, un código QR puede seguir siendo legible incluso si se ha perdido hasta un tercio de su información original.

Esta característica los convierte en una herramienta muy confiable para almacenar datos, especialmente en aplicaciones donde la seguridad y la verificación rápida de identidad son críticas. Por ello, su uso en credenciales escolares representa una solución eficaz para garantizar el acceso controlado a instalaciones o servicios institucionales.

4.3. Desarrollo de la Aplicación Móvil

Una aplicación móvil es un tipo de software diseñado para ejecutarse en dispositivos móviles, como teléfonos inteligentes o tabletas. A diferencia de las aplicaciones tradicionales para computadoras de escritorio, las aplicaciones móviles están optimizadas para operar con los recursos limitados de hardware de los dispositivos móviles, brindando funcionalidades específicas de manera eficiente. Aunque los dispositivos actuales son mucho más sofisticados que en sus primeras generaciones, las aplicaciones móviles siguen enfocándose en funciones concretas, permitiendo a los usuarios seleccionar solo las herramientas que necesitan en sus dispositivos [20].

Esta sección está enfocado en detallar el tipo de aplicación desarrollada para el proyecto, justificando las decisiones tecnológicas tomadas en cuanto a lenguajes, sistemas operativos y herramientas de desarrollo. Se explican los distintos tipos de aplicaciones móviles existentes, las razones detrás del uso de Kotlin y Android, así como las herramientas empleadas para su implementación.

4.3.1. Tipos de aplicaciones móviles

Existen diferentes tipos de aplicaciones móviles que responden a las necesidades y preferencias de los usuarios, así como a las capacidades técnicas de los dispositivos. A continuación, se detallan cada uno:

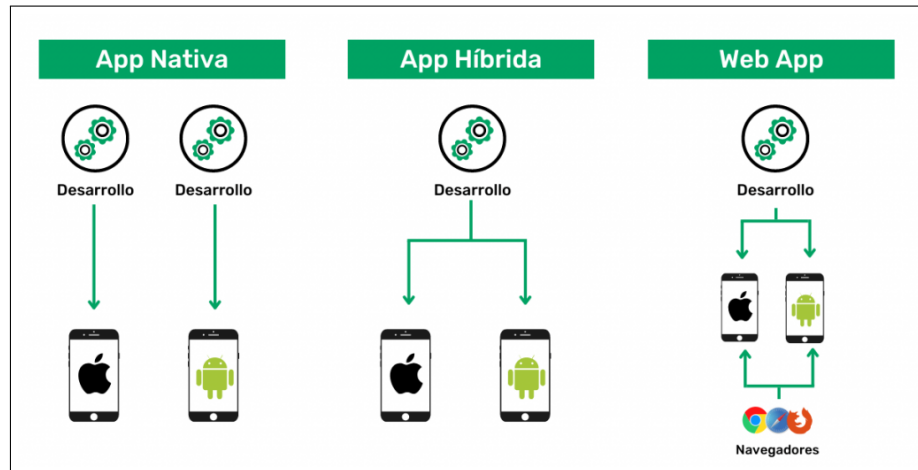


Figura 4.8: Tipo de aplicaciones móviles.

4.3.2. Aplicaciones nativas

Las aplicaciones nativas son apps desarrolladas para un sistema operativo móvil concreto (iOS o Android normalmente), en el lenguaje de programación específico de cada plataforma. Esto quiere decir que una app nativa creada para Android no puede ser utilizada en un dispositivo iOS y viceversa.

Es el tipo de aplicación móvil más conocida. Para que funcione, debemos descargarla desde los markets de apps, como App Store o Google Play e instalarla en nuestro teléfono [21].

Ventajas

- **Tienen el mejor rendimiento.** Las aplicaciones nativas son las más rápidas y tienen un rendimiento superior a otros tipos de apps, ya que han sido optimizadas específicamente para el hardware y el sistema operativo del dispositivo.
- **Acceso completo e integración con las funciones hardware del dispositivo.** Las apps nativas permiten aprovechar al máximo las funcionalidades móviles: cámara, micrófono, lector biométrico de huella, sensores y redes inalámbricas.
- **Pueden funcionar sin acceso a internet (funcionamiento offline)** si han sido diseñadas para ello.

Desventajas

- **Costes de desarrollo altos.** Si queremos tener nuestra app disponible para los dos sistemas, necesitaremos dos líneas de desarrollo diferentes, ya que el código utilizado para un sistema no es reutilizable para otro.
- **Complejidad de desarrollo.** Necesitamos equipos expertos en el lenguaje específico de cada sistema. Por ejemplo, en Kotlin para Android y en Swift para iOS.

- **Tiempo de desarrollo superior.** El desarrollo puede tomar entre 4 a 6 meses.

4.3.3. Aplicaciones Web

Las aplicaciones web realmente son webs especiales diseñadas para navegadores móviles. A diferencia de las apps nativas o híbridas, no necesitan ser descargadas, ya que se accede a ellas desde un navegador web.

Emplean las mismas tecnologías de desarrollo que una web, como HTML, CSS o JavaScript. Así, estaríamos hablando de una web con apariencia de app, por lo que presentaría sus mismas limitaciones. Sin embargo, con la llegada del HTML5, se han conseguido salvar algunas limitaciones, como el acceso a algunas funciones del móvil (geolocalización, cámaras) [20].

Ventajas

- **Carácter multiplataforma.** Con una sola línea de desarrollo.
- **Fácil desarrollo.** Se emplean tecnologías ampliamente conocidas.
- **Tiempo y coste de desarrollo bajo.**

Desventajas

- **Acceso limitado a las funciones del dispositivo.**
- **No se pueden subir a las tiendas de aplicaciones.**
- **Diferentes experiencias de usuario.** Estas dependen del navegador utilizado.
- **Necesidad de conexión a Internet.** Incluso si se cuenta con un modo pensado para ello, es necesario para acceder a las posibles actualizaciones o para entrar por primera vez.

4.3.4. Aplicaciones Híbridas

Las aplicaciones híbridas o multiplataforma combinan elementos de las aplicaciones nativas y las aplicaciones web. Estas aplicaciones se desarrollan utilizando tecnologías web como HTML, CSS y JavaScript, pero se empaquetan en un formato que puede ser instalado en un dispositivo móvil como cualquier otra aplicación nativa. Por tanto, podemos obtener una aplicación para varias plataformas con un único desarrollo [20].

Ventajas

- **Menor coste.** Gracias al uso de lenguajes de programación más conocidos, con una mayor disponibilidad de profesionales en el mercado.
- **Carácter multiplataforma.** Con una sola línea de desarrollo.

- **Acceso a algunas funcionalidades del móvil.**
- **Reducción de los tiempos de desarrollo.** Generalmente, el tiempo de desarrollo se reduce a 3 meses.
- **Disponibilidad en markets.** Se pueden subir a los markets de aplicaciones, como App Store y Google Play.

Desventajas

- **Rendimiento inferior.** Su rendimiento es inferior al de una app nativa, suelen tener un tamaño considerable y, además, ser más lentas.
- **Acceso limitado a las funciones del dispositivo.**

4.3.5. Aplicaciones Progresivas Web Apps (PWA)

Las aplicaciones progresivas son un reciente avance de las Web Apps. Al igual que las Web Apps, son webs diseñadas para móviles, pero esta vez, sí pueden ser descargadas en el móvil como una aplicación más, aunque no es necesario para que ofrezcan un comportamiento similar al de una app nativa a través del navegador.

Las PWA adoptan un comportamiento más propio de aplicaciones nativas que de web, como el funcionamiento sin Internet, un mayor rendimiento o su funcionamiento en segundo plano. Sin embargo, como desventaja, seguimos contando con la imposibilidad de subirlas a los markets de aplicaciones [20].

Para comprender mejor las diferencias entre los tipos de aplicaciones móviles, a continuación se presenta una tabla comparativa que destaca sus características clave:

Tipos de app	Nativa	Híbrida	Web
Interfaz	Basada en web	Específica de la plataforma (iOS, Android)	Basada en web
Tiempo de desarrollo	Alto	Medio	Bajo
Coste de desarrollo	Alto	Medio	Bajo
Multiplataforma	No	Sí	Sí
Rendimiento	Alto	Medio	Bajo
Acceso a los sensores del dispositivo	Completo	Alto o Completo	Limitado
Tiendas de aplicaciones	Sí	Sí	No

Cuadro 4.1: Comparación tipos de aplicaciones móviles. Elaboración propia

Para el desarrollo de nuestro trabajo terminal, hemos decidido optar por una aplicación híbrida con el uso de Kotlin. Esta decisión se basa en varios factores relacionados con los recursos disponibles, las características de nuestro público objetivo y los plazos establecidos.

La aplicación móvil está dirigida para estudiantes, alumnos y personal de seguridad de la Escuela Superior de Cómputo, donde la mayoría utiliza dispositivos con sistema operativo Android. La elección de una aplicación híbrida nos permite optimizar la experiencia en Android, que es la plataforma que predomina entre nuestros usuarios.

Aunque las aplicaciones híbridas suelen desarrollarse con tecnologías web (como React Native o Flutter), para nuestro proyecto hemos decidido incorporar Kotlin para desarrollar modelos donde se requiera un rendimiento nativo o un acceso más profundo a las funciones del sistema operativo Android. Además, las aplicaciones híbridas permiten un desarrollo más rápido en comparación con las aplicaciones completamente nativas, ya que gran parte del código puede compartirse entre plataformas, así mismo, es una buena opción económica que se adapta a nuestro presupuesto [20].

4.4. Elección de tecnología

La elección de Kotlin como lenguaje principal y Android como plataforma se basa en criterios técnicos, prácticos y económicos que se alinean directamente con los requisitos del proyecto.

4.4.1. Kotlin

Kotlin es un lenguaje de alto nivel debido a su abstracción respecto al hardware y la plataforma subyacente. Esto significa que permite a los desarrolladores escribir código sin preocuparse por la gestión de memoria o las interacciones directas con el sistema operativo o el hardware, lo que mejora la productividad y la legibilidad del código, además, este es estáticamente tipado, lo que implica que los tipos de las variables se definen en tiempo de compilación, lo que permite detectar errores antes de ejecutar el programa. Este se

usa principalmente en el desarrollo de aplicaciones Android, donde es el lenguaje recomendado por Google [22].

Además, Kotlin permite el uso de Jetpack Compose, un framework declarativo que moderniza la creación de interfaces de usuario en Android. También es compatible con bibliotecas clave para este proyecto como OpenCV, empleada en el módulo de reconocimiento facial.

4.4.2. Android

La decisión de desarrollar exclusivamente para Android se basa en:

- La mayoría de los usuarios objetivo (alumnos, docentes y personal de seguridad) utilizan dispositivos Android.
- Android permite mayor flexibilidad y personalización, crucial para funciones como escaneo de códigos QR, acceso a la cámara, integración con Python y OpenCV.
- El entorno Android es ideal para pruebas, ya que se puede emular fácilmente en computadoras con Windows.

La compatibilidad de Android con Python facilita la integración del frontend (Android) con el backend, donde Python se encarga del procesamiento de imágenes y entrenamiento de modelos de reconocimiento facial, mientras que Spring Boot gestiona la lógica del sistema. Esta arquitectura garantiza un funcionamiento sincronizado, eficiente y seguro.

4.5. Herramientas de Desarrollo

El desarrollo de la aplicación se realizó con herramientas modernas que forman parte del ecosistema oficial de Android, permitiendo una implementación robusta y escalable.

4.5.1. Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) oficial para el desarrollo de aplicaciones del mismo OS. Se basa en IntelliJ IDEA, un entorno de desarrollo integrado de Java para software, e incorpora sus herramientas de desarrollo y edición de código [23].

Para respaldar el desarrollo de aplicaciones dentro del sistema operativo, Android Studio utiliza un sistema de compilación basado en Gradle, un emulador de Android, plantillas de código e integración de GitHub. Cada proyecto en Android Studio tiene una o más modalidades con código fuente y archivos de recursos [23].

4.5.2. Jetpack Compose

Para la implementación de la aplicación móvil que forma parte de nuestro sistema de identificación y control de acceso, hemos decidido utilizar Jetpack Compose. La elección de la tecnología adecuada es importante para ofrecer una mejor experiencia de usuario y cumplir nuestros objetivos de diseño y funcional.

¿Por que Jetpack compose?

Jetpack compose es un framework (estructura o marco de trabajo que, bajo parámetros estandarizados, ejecutan tareas específicas en el desarrollo de un software) con la particularidad de ejecutar prácticas modernas en los desarrolladores de software a partir de la reutilización de componentes, así como también contando con la oportunidad de crear animaciones y temas oscuros. En este sentido, Jetpack Compose es el conjunto de herramientas ofrecidas por Android para el desarrollo de aplicaciones con un objetivo específico: simplificar y optimizar los códigos en la IU nativas [19].

Ventajas

- **Menos código:** Simplifica el proceso de desarrollo haciendo menos código, todo se basa en funciones de modo que el código será simple y fácil de mantener.
- **Intuitiva:** Tan solo describe tu IU con un enfoque declarativo haciendo “qué hay que hacer” en vez de “cómo se debe hacer”.
- **Potente:** Tiene integrado Material Design con el cual puede crear apps atractivas al usuario con animaciones y mucho más.
- **Acelera el desarrollo:** Es compatible con proyectos existentes, puedes empezar a integrarlo por partes cuando quieras y donde quieras.
- **Kotlin:** Está escrito 100% en Kotlin, lo cual nos permitirá usar sus herramientas potentes y API's intuitivas.

Arquitectura Jetpack

La arquitectura o estructura sobre la que se basa Jetpack Compose es una estructura Jetpack que se encarga principalmente de seguir ejecutando y beneficiándose de aquellos componentes de Android según la funcionalidad disponible. Por lo tanto, Jetpack Compose desarrolla herramientas denominadas «composables» a partir de elementos como botones o listados de objetos. A partir de fuentes de datos, pueden ser reutilizadas y ejecutadas en distintas fuentes sin la necesidad de programar varios códigos repetitivos.

Desde un aspecto comparativo, la creación de UI con Compose se realiza de forma similar a los de React Native. Es decir, mediante componentes reutilizables evitan maximizar la cantidad de códigos necesarios y repetitivos, tal como ocurre con HTML en la forma en la que se conjugan uno con otro. Sin embargo, es importante destacar que dicha compilación de componentes que logran la disminución en cantidad de códigos se realiza gracias a los plugins de compiladores de Kotlin, ejecutando así los componentes mediante la estructura de archivos de Kotlin [?].

Estructura de Jetpack Compose

Para entender de forma más sencilla la estructura de esta herramienta, conviene tener en cuenta componentes como:

- **Compiler:** Se encarga mediante una estructura de plugins en Gradle, donde logra la interpretación y simplificación de códigos.
- **El entorno de ejecución:** Es el ámbito “depurador”, si se puede establecer de este modo, en el que se establecen y diferencian los componentes que necesitan actualización y cuáles no, así como también se crean listados de mantenimiento y composición.
- **UI:** Es el componente que interpreta el lenguaje establecido en el entorno de ejecución y lo refleja en pantalla.

4.5.3. Gradle

Gradle es el sistema de automatización de compilación utilizado por Android Studio, y fue esencial para el desarrollo del presente proyecto, permitiendo una integración fluida entre los módulos móviles y el backend. Esta herramienta de código abierto destaca por su rendimiento y flexibilidad, utilizando Kotlin DSL (Domain Specific Language) para definir scripts de compilación [24].

Gradle facilita la gestión eficiente de dependencias externas como Jetpack Compose, Retrofit y OpenCV, utilizadas en el desarrollo de la aplicación. Además, su capacidad para realizar compilaciones optimizadas en paralelo y reutilizar salidas anteriores permite reducir considerablemente los tiempos de construcción. También permite configurar distintos entornos de ejecución (debug y release), lo que fue útil para probar características específicas antes de su despliegue oficial.

En este proyecto, Gradle también permitió la integración con servicios backend mediante llamadas a APIs REST, conectando la aplicación con el servidor desarrollado en Spring Boot, fortaleciendo así la arquitectura cliente-servidor de la solución.

4.5.4. Git y GitHub

Durante el desarrollo del proyecto, se utilizó Git como sistema de control de versiones y GitHub como plataforma de colaboración y almacenamiento en la nube. Git, diseñado por Linus Torvalds, permite llevar un control detallado del historial de cambios en el código, lo cual es esencial en proyectos donde trabajan varios desarrolladores de forma simultánea [24].

Cada miembro del equipo pudo trabajar en ramas independientes, realizar modificaciones, proponer mejoras y fusionar los cambios mediante revisiones de código, garantizando la integridad del repositorio principal. Esta metodología permitió detectar errores, revertir versiones cuando fue necesario, y mantener una trazabilidad clara del desarrollo de cada módulo.

GitHub, además de servir como repositorio central, funcionó como herramienta de comunicación y documentación del avance del proyecto. Las funcionalidades de issues, pull requests y proyectos ayudaron a distribuir tareas, dar seguimiento a problemas y mantener una organización clara durante todo el ciclo de desarrollo [25].

4.6. Bases de datos (BD)

Para que todo sistema pueda funcionar, debe de tener una forma de almacenar toda su información, ya sea información de los usuarios o para almacenar imágenes, cualquier tipo de información que se requiera

almacenar, es necesario usar una base de datos. Pero en sí, ¿qué es una base de datos?. Una base de datos, según Microsoft [26], es una herramienta para recopilar y organizar información. Estas pueden almacenar información sobre personas, productos, pedidos u otras cosas. Las bases de datos pueden empezar desde un simple documento de textos o archivos Excel, pero conforme más va creciendo esta base de datos empiezan a aparecer redundancias, por lo que es mejor optar por usar una base de datos creada por un sistema gestor de bases de datos. Un sistema gestor de bases de datos es un software constituido por una serie de programas dirigidos a crear, gestionar y administrar la información que se encuentra en una base de datos. El principal objetivo de estos sistemas es servir de interfaz entre los usuarios y las aplicaciones para facilitar la organización de los datos [27]. Por otro lado, dentro del mundo de las bases de datos, existen diferentes tipos de bases de datos, y cada tipo de base de datos tiene una organización diferente y propósito diferente. A continuación, les hablaremos de ciertos tipos de bases de datos:

4.6.1. Bases de datos relacionales:

Este tipo de bases de datos es el más usado en sistemas que necesitan almacenar una gran cantidad de información que está relacionada entre sí. Ahora bien, una base de datos relacional, según Google [28], es una forma de estructurar información en tablas, filas y columnas. La ventaja de este tipo de bases de datos es que tiene la capacidad de establecer relaciones entre la información mediante tablas, esto nos ayuda a visualizar mejor la información sobre la relación entre los datos. Las bases de datos relacionales se componen de una serie de conceptos clave que necesitamos desarrollar para mejorar la comprensibilidad acerca de este tipo de bases de datos:

- **Tablas:** Las tablas, en una base de datos relacional, son objetos que contienen todos los datos de dicho objeto. Como se mencionó anteriormente, estas tablas se organizan en filas y columnas. Aquí, cada fila representa un registro único y cada columna un campo dentro del registro. La manera en que este tipo de tablas guardan datos únicos es por medio de una clave principal, la cuál se va a explicar a continuación. [29]
- **Clave principal:** Como ya se mencionó, las tablas tienen una clave principal, la cuál suele ser una columna o un conjunto de columnas cuyos valores identifican la forma única de cada fila de la tabla. Estas columnas se denominan claves principales de la tabla y aunque la clave principal sea una combinación de columnas, esta combinación es única dentro de la tabla. [30]
- **Clave externa:** Esta es una columna o combinación de columnas que se usa para establecer un vínculo entre los datos de dos tablas. Cuando una tabla tiene dentro de sus columnas la clave principal de otra tabla, se dice que esa primera tabla está referenciando a la otra por medio de una clave externa. [31]
- **Sistema de gestión de bases de datos (SGBD):** Son sistemas que ayudan a controlar las bases de datos. Estos sistemas actúan como interfaz entre los usuarios y las bases de datos, y se encargan justamente de gestionar los datos y las bases de datos como tal. En otras palabras, un sistema de gestión de bases de datos es un software utilizado para gestionar, almacenar y recuperar bases de datos, a su vez, proporciona una interfaz que permite a los usuarios leer, crear, borrar y actualizar datos. [27]

PostgreSQL:

PostgreSQL es una SGBD relacional, el cual, a diferencia de otros este soporta tipos de datos relacionales y no relacionales. Este fue creado con el propósito de soportar cargas de trabajo desde pequeñas aplicaciones hasta sistemas complejos de procesamiento de datos a gran escala. Para este proyecto, se hará uso de este SGBD, ya que tiene compatibilidad con los diferentes lenguajes que vamos a usar, como lo es Java, y además, es libre de restricciones de licencia. [32]

4.6.2. Bases de datos no relacionales:

Este tipo de bases de datos no siguen el esquema de filas y columnas como las bases de datos relacionales, en su lugar, este tipo de bases de datos usan un modelo de almacenamiento que está optimizado para los requisitos del tipo de dato que van a guardar. Dentro de este tipo de bases de datos existe una gran variedad, hay bases de datos que almacenan su información como pares clave/valor simple, como formatos JSON, o como un grafo que consta de bordes y vértices. Lo que caracteriza a este tipo de bases de datos es que no usan el modelo relacional. [31]

4.6.3. Bases de datos vectoriales

Dado la reciente popularidad de los sistemas potenciados por inteligencia artificial cuyas aplicaciones van desde el procesamiento inteligente de los textos, la detección de anomalías o, para fines de este trabajo, la identificación y verificación de las características faciales de las personas es que ha surgido la necesidad de contar con soluciones para almacenar y procesar de forma rápida grandes volúmenes de información y, aunque existen herramientas especializadas para estas tareas es importante saber distinguir la solución que mejor se adapte a las necesidades del proyecto.

Por lo tanto, algunos proyectos trabajan con un tipo de dato conocido como *embedding* que no es más que una representación numérica (vector) de objetos del mundo real como texto, audio o imágenes. Se encuentran optimizados para funcionar como entradas a modelos de aprendizaje automático, aprendizaje profundo y, especialmente, para llevar a cabo búsquedas semánticas.

Los embeddings son representados en un espacio vectorial multidimensional en donde los objetos con características similares se encuentran más cerca entre sí. Además, otra característica distintiva de este tipo de representación es que, a diferencia de otras técnicas de aprendizaje automático, se aprende directamente de los datos mediante el uso de algoritmos de aprendizaje profundo teniendo la ventaja de prescindir de una definición explícita por parte de expertos humanos.

Con esto en mente, una base de datos vectorial es un tipo de base de datos especializada y optimizada para almacenar, indexar y recuperar datos vectoriales de alta dimensionalidad (*embeddings*). A diferencia de las bases de datos relacionales tradicionales, que se basan en tablas estructuradas y coincidencias exactas, las bases de datos vectoriales se centran en las búsquedas de similitud. Esto les permite recuperar puntos de datos que están semánticamente o contextualmente relacionados, en lugar de solo coincidencias exactas [33].

Debido a que este tipo de bases de datos se encuentran diseñadas para trabajar con conjuntos de datos complejos y multidimensionales, el uso de bases de datos tradicionales como las relacionales para la integración de sistemas de inteligencia artificial es difícil, por no decir imposible.

A modo de resumen, se presenta la siguiente tabla comparativa para identificar las principales diferencias entre este mecanismo de almacenamiento con respecto a las bases de datos tradicionales.

Cuadro 4.2: Comparación entre bases de datos vectoriales y tradicionales

Característica	Bases de datos vectoriales	Bases de datos tradicionales (Ej. Relacionales)
Estructura de datos	Vectores de alta dimensionalidad (embeddings)	Tablas estructuradas con esquemas predefinidos
Método de consulta	Búsquedas de similitud (distancia euclidiana, coseno)	SQL (operaciones relacionales, filtros)
Caso de uso principal	Búsqueda semántica, sistemas de recomendación, RAG, reconocimiento facial	Gestión de transacciones, informes estructurados
Modelo de escalabilidad	Horizontal (añadir nodos para capacidad y rendimiento)	Vertical (mejorar hardware) y Horizontal (sharding)
Enfoque de rendimiento	Búsquedas de vecinos más cercanos, indexación ANN	Consultas SQL rápidas, integridad transaccional

4.7. Spring Boot

Spring Boot es una herramienta que sirve para desarrollar tanto aplicaciones web, como microservicios o aplicaciones web en Java. Este se basa en Spring Framework, el cual es un framework para el desarrollo de aplicaciones y contenedores de inversión de control de código abierto, igualmente para Java. Spring Boot permite el desarrollo de aplicaciones web y microservicios de manera rápida y fácil gracias a 3 características que tiene, las cuales son:

- **Configuración automática:** Con esto se refiere a que Spring Boot lo que hace es analizar las dependencias incluidas en un proyecto, como lo son bases de datos, servidores, entre otras cosas y decide qué configuraciones aplicar. Esto se basa en anotaciones que usualmente empiezan con un "@", gracias a esto, podemos desarrollar aplicaciones de manera más rápida y eficiente, pues nos ahorra todo el trabajo de realizar método por método. [34]
- **Enfoque obstinado de la configuración:** Con base en las mejores prácticas de la programación, Spring Boot toma ciertas decisiones de configuración que funcionan bien para la mayoría de casos, esto se refiere a configuraciones de puertos, rutas, entre otras cosas. Además, en caso de que nosotros queramos agregar configuraciones adicionales, o bien, cambiar la configuración de algo de nuestro proyecto, nos va a crear un archivo de configuración en donde podremos especificarle de qué manera lo queremos. [35]

- **Capacidad de crear aplicaciones independientes:** Nos permite crear aplicaciones que se ejecutan de forma independiente, es decir, sin necesidad de un servidor web externo. Esto lo logra gracias a que en el proceso de inicialización, se integra un servidor web en la aplicación, o que le permite funcionar de manera autónoma. [34]

En este caso en particular, se utilizó de Java Spring Boot para hacer un servidor que funcionara como una API REST para que los diferentes sistemas involucrados se pudieran comunicar de manera más sencilla.

4.8. Java Persistence API (JPA)

Antes de irnos a la definición de JPA, hay que entender qué es la persistencia de los datos. La persistencia de los datos es un medio mediante el cual una aplicación puede recuperar información desde un sistema de almacenamiento y hacer que esta persista. Ahora bien, JPA lo que hace es proporcionarnos varias funciones para poder gestionar la persistencia y la correlación de objetos, es decir, lo que hace es proveernos con una serie de interfaces que podemos utilizar para implementar la capa de persistencia de nuestra aplicación. [36]

Algunas de las ventajas que nos presenta JPA es que nos permite hacer el mapeo de entidades, es decir, el definir cómo se relacionan las clases de nuestra aplicación con los elementos de nuestra base de [36]. Esto abarca temas como las relaciones entre clases y tablas en nuestra abse de datos, propiedades de las clases y los campos de las tablas e incluso la relación entre diferentes clases y las claves exrternas de nuestras tablas.

Además, JPA ofrece la capacidad de interactuar con diferentes sistemas de gestión de bases de datos, ya que no usa sentencias SQL o de algún tipo de base de datos en específico, por lo que mejora la portabilidad y escalabilidad de las aplicaciones. [37]

Spring Boot cuenta con una serie de anotaciones que ayudan a simplificar el desarrollo de las aplicaciones. Estas anotaciones son mecanismos que agregan metadatos, los cuales son interpretados por Spring Boot para así configurar y gestionar automáticamente los componentes de una aplicación [?].

Las anotaciones usadas en este proyecto fueron las siguientes:

- **@Configuration:** Esta anotación se usa en clases que definen beans. Esta anotación es un análogo para un archivo de configuración XML, solo que la configuración se hace mediante clases Java.
- **@Value:** Indica una expresión de valor predeterminado para un campo o parámetro al inicializar una propiedad.
- **@Bean:** Se usa a la par de @Configuration para crear beans Spring, es decir, para crear objetos gestionados por Spring.
- **@Override:** Se usa para un sobrescribir un método. Usar @Override garantiza en tiempo de compilación que estás sobrescribiendo correctamente un método.
- **@Service:** Marca una clase Java que realizará algún servicio, como ejecutar la lógica de negocio, realizar cálculos y llamar a API's externas.
- **@PostConstruct:** Define un método que debe ejecutarse después de que el bean haya sido inicializado y todas sus dependencias hayan sido inyectadas.

- **@Embeddable:** Esta anotación quiere decir que las propiedades de una clase pueden ser incluidas en otra como un valor
- **@Column:** Define variables de una clase como columnas en una base de datos.
- **@ManyToOne:** Se encarga de generar una relación de muchos a uno.
- **@JoinColumn:** Sirve para hacer referencia a la columna que es llave foránea en una tabla, y la cual se encarga de definir la relación.
- **@Entity:** Registra una clase como una entidad.
- **@Table:** Se encarga de mapear una entidad contra una tabla que tenga un nombre distinto.
- **@Id:** Indica que una variable será registrada como una llave primaria de una entidad.
- **@JsonProperty:** Sirve para indicar el nombre de una variable en un JSON.
- **@OneToOne:** Se encarga de generar una relación uno a uno en una base de datos.
- **@OneToMany:** Se encarga de generar una relación uno a muchos en una base de datos.
- **@EmbeddedId:** Se utiliza para definir una clave primaria compuesta embebida en una entidad.
- **@MapsId:** Indica que un atributo de una entidad comparte su identificador con otra entidad, esto llega a ser muy útil en relaciones donde se comparten claves primarias.
- **@GeneratedValue:** Esta anotación se usa muy seguido, sobre todo porque hay veces en las que las claves primarias que tenemos deben de ser valores autoincrementables o definidos. Esta anotación es justo lo que hace, definir un valor a una llave primaria.
- **@UniqueConstraint:** Define restricciones de unicidad en columnas de una tabla, para asegurar que los valores que se inserten sean únicos.
- **@Temporal:** Se usa para definir tipos de variables Date, Time.
- **@JoinColumns:** Define múltiples columnas de unión en relaciones compuestas entre entidades.
- **@PrePersist:** Sirve para ejecutar alguna acción antes de que se guarde una nueva entidad.
- **@PreUpdate:** Indica que un método debe ejecutarse antes de que una entidad sea actualizada.
- **@RestController:** Con esta anotación se construyen servicios REST de manera más fácil y efectiva. Indica que una clase manejará las solicitudes HTTP y que devolverá datos JSON.
- **@RequestMapping:** Se usa para mapear solicitudes HTTP a métodos Request.
- **@Autowired:** Inyecta dependencias de una clase automáticamente.
- **@GetMapping:** Se usa para mapear solicitudes HTTP a métodos GET.

- **@PostMapping:** Se usa para mapear solicitudes HTTP a métodos POST.
- **@RequestPart:** Se usa para vincular una parte de una solicitud a un parámetro de un método.
- **@RequestParam:** Indica que un parámetro de un método se debe vincular a un parámetro de una solicitud. Un ejemplo de esto es para extraer datos de una URL.
- **@DeleteMapping:** Se usa para mapear solicitudes HTTP a métodos DELETE.
- **@CrossOrigin:** Permite solicitudes desde diferentes dominios.
- **@ExceptionHandler:** Define un método para manejar excepciones específicas lanzadas por el controlador.
- **@Repository:** Registra una clase como un bean, en este caso, define clases que pueden acceder a la base de datos.
- **@Query:** Define consultas personalizadas usando JPQL o SQL nativo.
- **@Modifying:** Se usa junto con @Query, y es para indicar que la consulta realiza operaciones de modificación, como insert, update o delete.
- **@Transactional:** Permite a una clase ejecutarse dentro de una transacción de una base de datos, es decir que si alguna parte de un método llega a fallar, todas las operaciones realizadas en dicho método se cancelan.
- **@Param:** Se usa para nombrar parámetros en consultas definidas con @Query.
- **@Service:** Define a una clase como un componente de servicio. Estas clases son las que tienen toda la lógica de negocio.
- **@Async:** Permite ejecutar métodos de manera asíncrona.
- **@Scheduled:** Programa la ejecución de un método en intervalos específicos.
- **@SpringBootApplication:** Marca la clase principal de una aplicación Spring Boot.

4.9. Patrones de arquitectura de software

Al momento de desarrollar software, es común toparnos con problemáticas que requieren de la toma de decisiones especialmente cuando hablamos sobre cuestiones relacionadas con el diseño de un sistema de software. Un patrón de arquitectura de software es un conjunto de decisiones tomadas para atacar problemáticas relacionadas con el diseño de un software. Estos incluyen reglas y principios para organizar las interacciones entre subsistemas predefinidos y los roles que estos desempeñan [38].

A menudo pueden ser descritos como los "*Planos*" de un sistema, sin embargo, esto no quiere decir que sea la arquitectura final, sino que funcionan como una guía que describe los elementos necesarios para diseñar la arquitectura de la solución a desarrollar, la selección de una arquitectura sobre otra dependerá

completamente de los objetivos a alcanzar, los recursos disponibles y la experiencia del equipo de desarrollo [39].

Es necesario aclarar que, los patrones de arquitectura no deben confundirse con los patrones de diseño, ya que ambos responden a problemas diferentes durante el desarrollo de un sistema de software, de forma muy breve un patrón de arquitectura describe como crear la lógica de negocio, acceso a los datos, etc. Mientras que los patrones de diseño se usan al implementar estos elementos [38].

A continuación, se describen algunos patrones de arquitectura que pueden ser de utilidad al implementar la propuesta de solución para este trabajo terminal.

4.9.1. Arquitectura de capas

También conocida como *arquitectura de N-capas*, estructura una aplicación en múltiples capas distintas, donde cada una esta encargada de ciertas tareas en específico, lo que permite dividir un sistema en componentes aislados, lo que facilita el desarrollo rápido de aplicaciones ya que los cambios realizados en una capa no deberían afectar la lógica de las demás [40].

Las arquitecturas basadas en este patrón suelen implementar cuatro capas distintas, la capa de presentación, la capa de negocio, de persistencia y de base de datos, sin embargo, y como es de esperarse, la arquitectura de un sistema basado en este patrón puede tener algunas diferencias en el número y tipo de capas que se implementan, por ejemplo, algunas pueden implementar capas de aplicación, servicio o acceso a datos [38].

4.9.2. Arquitectura orientada a servicios

Las aplicaciones diseñadas siguiendo este patrón de arquitectura implementan una colección de servicios poco acoplados que se comunican entre sí a través de una red. Cada uno de los servicios que conforman al sistema se encarga de llevar acabo una función del negocio en específico que después pueden ser requeridos por otro servicio o cliente [40].

4.9.3. Arquitectura de micro servicios

Es una arquitectura que combina patrones de diseño para crear múltiples servicios que trabajan de forma independiente y que en conjunto forman la lógica de una aplicación. Es una alternativa a las aplicaciones monolíticas y de la arquitectura basada en servicios, en donde cada servicio esta orientado a implementar partes muy puntuales de la lógica de negocio [39].

Su principal ventaja radica en que debido a que sus componentes se encuentran poco acoplados pueden ser desarrollados, desplegados y probados de forma independiente [40].

La principal desventaja de este tipo de arquitectura es que puede llegar a ser compleja de implementar ya que requiere de definir la granularidad correcta de los servicios y establecer una comunicación efectiva entre estos [38].

4.9.4. El patrón modelo vista controlador

Este patrón divide una aplicación en tres componentes interconectados. El modelo, la vista y el controlador. Esta separación permite organizar el código al desacoplar la lógica del negocio, interfaz de usua-

rio y el manejo de las entradas de los usuarios, lo que a su vez promueve la modularidad, mantenibilidad y escalabilidad [40].

El modelo contiene los datos y lógica de negocio de la aplicación. Se encarga de regresar, almacenar y procesar la información.

La vista, también conocida como interfaz de usuario (UI) despliega la información al usuario y responde a las interacciones del usuario.

El controlador funciona como un intermediario entre el modelo y la vista. Se encarga de gestionar las entradas del usuario, actualizar el modelo y la vista para reflejar los cambios realizados en el modelo [39].

4.10. Redes neuronales artificiales

Una red neuronal artificial es una técnica para la creación de programas de computación que son capaces de aprender de los datos. Se basa en el conocimiento actual sobre como funciona el cerebro humano. Consisten en un conjunto de nodos o neuronas interconectados entre sí que procesan y aprenden de los datos con los que cuentan, lo que permite llevar a cabo tareas como el reconocimiento de patrones y la toma de decisiones en el aprendizaje de máquina [41].

4.10.1. Estructura de una red neuronal artificial

Toda red neuronal esta formada por capas de nodos, o neuronas artificiales, una capa de entrada, una o más capas ocultas y una capa de salida. Cada nodo se conecta a los demás y tiene un peso y un umbral determinado. A continuación se describe el papel que cada una de estas capas en la arquitectura de una red neuronal común [42].

Capa de entrada

La capa de entrada es responsable de recibir los datos iniciales en forma de vectores. Cada neurona en esta capa corresponde a una característica del conjunto de datos [43].

Capas ocultas

Las capas ocultas son el núcleo de las RNA y donde ocurre el procesamiento de los datos. Estas capas aplican transformaciones no lineales mediante funciones de activación, como *ReLU*, *sigmoide*, o *tangente hiperbólica*. El número y tamaño de las capas ocultas determinan la capacidad de la red para modelar patrones complejos [43].

Capa de salida

La capa de salida genera los resultados finales del modelo. Para tareas de clasificación, por ejemplo, esta capa utiliza funciones como *softmax* para proporcionar probabilidades asociadas a cada clase [43].

4.11. Proceso de aprendizaje

Como ya se mencionó con anterioridad, una red neuronal es un sistema complejo que busca imitar la forma en la que el cerebro humano aprende [?]. Este proceso de aprendizaje se realiza en dos fases conocidas como *backpropagation* y *forward propagation* [43]:

4.11.1. Forward propagation

- **Capa de entrada:** La capa de entrada contiene nodos que representan cada característica del conjunto de datos inicial. Estos nodos reciben y procesan los datos de entrada.
- **Pesos y conexiones:** Los pesos asignados a cada conexión entre neuronas determinan la influencia de una neurona sobre otra. Durante el entrenamiento, estos valores se actualizan constantemente para optimizar el rendimiento de la red.
- **Capas ocultas:** Las neuronas en las capas ocultas combinan las entradas recibidas al multiplicarlas por sus respectivos pesos y sumarlas. Posteriormente, aplican una función de activación que introduce no linealidad, lo que permite identificar relaciones complejas en los datos.
- **Capa de salida:** Este proceso se repite hasta llegar a la capa de salida, donde se genera el resultado final de la red neuronal.

4.11.2. Backpropagation

- **Cálculo del error:** La salida generada por la red se compara con los valores esperados utilizando una función de pérdida. Para problemas de regresión, se emplea comúnmente el *Error Cuadrático Medio* (*Mean Squared Error, MSE*), que calcula la diferencia entre las predicciones y los valores reales:
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
- **Optimización por descenso de gradiente:** Para reducir el error, la red emplea el algoritmo de descenso de gradiente. Este ajusta los pesos calculando la derivada de la función de pérdida con respecto a cada peso, guiando los ajustes necesarios para minimizar el error.
- **Ajuste de pesos:** Este proceso de actualización de pesos se realiza en sentido inverso a través de toda la red, lo que permite optimizar las conexiones entre neuronas.
- **Proceso iterativo de entrenamiento:** Durante el entrenamiento, los pasos de propagación hacia adelante, cálculo del error y retropropagación se repiten varias veces con diferentes muestras de datos, permitiendo que la red refine sus parámetros y aprenda patrones específicos.

4.11.3. Funciones de activación

Las funciones de activación son fundamentales para introducir no linealidad en el modelo. Ejemplos comunes incluyen la función *ReLU* (Rectified Linear Unit) y la sigmoide. Estas funciones determinan si una neurona se activa, lo que depende del valor ponderado de sus entradas.

4.12. Tipos de redes neuronales

Las redes neuronales pueden clasificarse en distintos tipos, cada uno diseñado para cumplir propósitos específicos. Aunque no se trata de una lista exhaustiva, a continuación se describen algunas de las variantes más comunes y sus casos de uso principales [42]:

4.12.1. Perceptrón

El perceptrón es la red neuronal más antigua, desarrollada por Frank Rosenblatt en 1958. Es el precursor de las redes neuronales modernas.

4.12.2. Perceptrón multicapa

También conocidas como *feedforward neural networks*, estas redes están formadas por una capa de entrada, una o más capas ocultas y una capa de salida. Aunque frecuentemente se les llama MLPs, técnicamente están compuestas por neuronas sigmoideas y no por perceptrones simples, ya que los problemas del mundo real suelen ser no lineales. Estas redes se entrenan alimentándolas con datos y son la base de aplicaciones como visión por computadora, procesamiento de lenguaje natural y otros modelos avanzados.

4.12.3. Redes neuronales convolucionales

Las redes neuronales convolucionales (*Convolutional Neural Networks*, CNNs) son una variante de las redes de avance directo que se utilizan principalmente en tareas como el reconocimiento de imágenes, el análisis de patrones y la visión por computadora. Estas redes aplican principios del álgebra lineal, en particular la multiplicación de matrices, para identificar patrones dentro de las imágenes.

4.12.4. Redes neuronales recurrentes

Las redes neuronales recurrentes (*Recurrent Neural Networks*, RNNs) se caracterizan por tener lazos de retroalimentación en su estructura. Estas redes son especialmente útiles para trabajar con datos temporales, donde las predicciones dependen de la información previa. Por ejemplo, se aplican en la predicción de mercados financieros y en la proyección de ventas.

4.13. Limitaciones

Las RNA tienen la capacidad de capturar patrones complejos en grandes volúmenes de datos. Sin embargo, también presentan desafíos como [44]:

- **Necesidad de datos:** Requieren grandes cantidades de datos etiquetados para entrenarse adecuadamente.
- **Costo computacional:** Su entrenamiento puede ser intensivo en términos de tiempo y recursos computacionales.

- **Interpretabilidad:** Las predicciones de las RNA suelen ser difíciles de interpretar, lo que plantea retos en aplicaciones críticas.

4.14. Reconocimiento facial

El reconocimiento facial es una técnica que permite a las computadoras predecir la identidad de una persona desde una imagen [45].

Tiene múltiples aplicaciones en la industria como el desarrollo de sistemas de asistencia, en la salud, sistemas de seguridad, etc [46].

4.14.1. Pipeline de un sistema moderno de reconocimiento facial

A menudo, los términos verificación de rostros e identificación de rostros se usan de forma indistinta, sin embargo, aunque ambos comparten el mismo dominio del problema, lo abordan de forma distinta [47].

Para entender mejor la diferencia entre ambas tareas y su papel en el desarrollo de un sistema de reconocimiento facial, explicaremos como los sistemas modernos de reconocimiento facial operan a través de una secuencia de etapas bien definidas, donde cada una es indispensable para obtener resultados precisos y fiables [46].

En cualquier pipeline, la interdependencia entre sus etapas es crucial, ya que la calidad y el éxito de cada fase dependen directamente del desempeño de la anterior. Un ejemplo claro es la etapa de detección de rostros: los modelos de extracción de características esperan recibir una imagen de un rostro correctamente identificada. Si se emplean técnicas de detección poco robustas, el módulo siguiente puede fallar en la extracción de características, lo que resulta en una menor precisión en los resultados finales [48].

Por lo tanto, a continuación se presentan las etapas fundamentales de un sistema de reconocimiento facial actual:

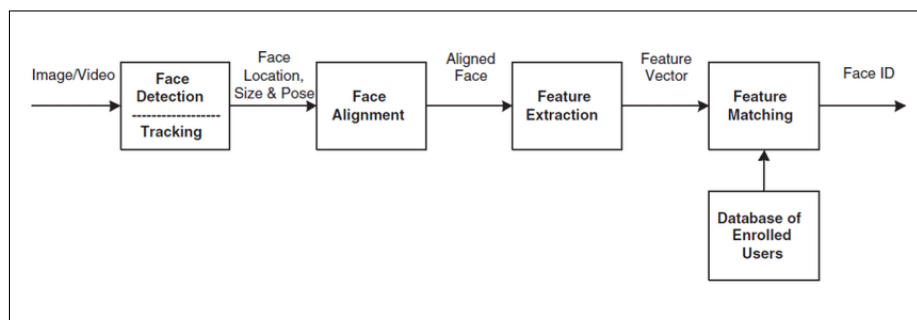


Figura 4.9: Pipeline moderno de un sistema de reconocimiento facial.

Detección de rostros

Esta es la etapa inicial, donde el sistema identifica la presencia de rostros en una imagen o video y determina su ubicación, generalmente mediante el cálculo de cuadros delimitadores. Se emplean modelos especializados que no solo identifican los cuadros delimitadores, sino que también detectan puntos clave faciales (como ojos, nariz y boca), los cuales son esenciales para la alineación posterior [47].

Algunos algoritmos y técnicas empleadas en este punto incluyen métodos tradicionales como el algoritmo Viola Jones o implementaciones más recientes basadas en aprendizaje profundo como MTCNN o RetinaFace [45].

Como se mencionó, este paso es la base del sistema de reconocimiento facial, por lo que su correcta implementación puede repercutir drásticamente en los resultados obtenidos.

Alineación de rostros

Una vez detectados, los rostros se estandarizan en orientación y tamaño utilizando los puntos clave extraídos. Este proceso implica rotar, escalar y recortar los rostros para asegurar que características como los ojos y la boca estén posicionadas de manera consistente [48].

El propósito de la alineación es reducir las variaciones causadas por la pose o la inclinación, haciendo que las entradas sean más uniformes para las etapas posteriores y asegurando que el modelo de embedding se enfoque en las características relevantes, lo que mejora la calidad y fiabilidad de los embeddings [47].

Se ha demostrado que, la alineación de rostros mejora la precisión obtenida por este tipo de sistemas [48].

Extracción de características o representación en forma de embeddings

Una vez que las imágenes de los rostros se encuentran correctamente procesadas, es decir, se identificaron y recortaron los rostros, seguidos de una alineación, se procede a usar algún modelo para la generación de embeddings como *FaceNet* o *ArcFace* los cuales generan vectores de alta dimensionalidad [48].

El propósito de esta etapa es transformar los datos complejos de una imagen en un formato numérico, compacto y sobre todo, comparable para lograr una eficiente comparación en el siguiente punto [47].

Identificación o verificación de rostro

Los embeddings faciales obtenidos se comparan con una base de datos utilizando medidas de similitud como la distancia coseno o euclidiana. Este paso permite determinar si dos rostros corresponden a la misma persona (verificación) o identificar a una persona entre múltiples registros (identificación) [45].

4.14.2. Identificación de rostros

Como ya se mencionó, la identificación de rostros y la verificación de rostros son dos tareas distintas involucradas al momento de implementar un sistema de reconocimiento facial [45].

La identificación facial se refiere a la tarea de identificar la identidad de una persona dada una imagen. La imagen se ingresa por un extractor de características para obtener una representación f . Luego, esta representación, entra a una red de clasificación para asignar la identidad asociada a la imagen de entrada [46].

La identificación de rostros es una tarea de clasificación, por lo que los sistemas de reconocimiento facial basados en esta tarea son entrenados usando la función *softmax* y la pérdida de entropía cruzada [4].

4.14.3. Verificación de rostros

Contrario a la identificación de rostros, la verificación de rostros busca verificar que dos imágenes pertenecen a la misma entidad [45]. Por lo tanto, en un sistema de reconocimiento facial basado en la verificación de rostros recibe como entrada una imagen x y el identificador del rostro a ser identificado y la salida es una decisión binaria sobre si la imagen x pertenece a la persona con el identificador dado [46].

Este enfoque requiere de un proceso de extracción de características de los rostros de las personas que queremos que el sistema verifique y, por lo tanto, almacenar dichas representaciones en una base de datos [46].

4.14.4. Reconocimiento facial mediante verificación

En los sistemas tradicionales de reconocimiento facial, se entrena un clasificador con K clases para asignar a cada rostro de entrada una identidad específica de entre K personas en la base de datos. Sin embargo, este enfoque presenta limitaciones de escalabilidad, ya que al agregar una nueva persona, es necesario reentrenar completamente el sistema con $K + 1$ clases. Esto ocurre porque el clasificador inicial tiene K neuronas, pero para incluir una nueva identidad, se requiere un clasificador con $K + 1$ neuronas [46].

Para abordar este problema, un enfoque más eficiente y escalable es utilizar la comparación basada en similitud, propia de los algoritmos de verificación. En este caso, dado un rostro de entrada x , se ejecuta el algoritmo de verificación K veces, una para cada rostro almacenado en la base de datos. La identidad del rostro de entrada se asigna a la correspondiente al ID para el cual el algoritmo de verificación indica una coincidencia [46].

Por lo tanto, la ventaja de este tipo de enfoque híbrido radica en el hecho de que es posible agregar nuevas entidades al sistema sin la necesidad de re-entrenar los modelos de deep learning utilizados para la generación de los vectores de características de los rostros, más aún si se combina con algún tipo de mecanismo de almacenamiento de estos como las bases de datos vectoriales para acelerar el proceso de inferencia [49].

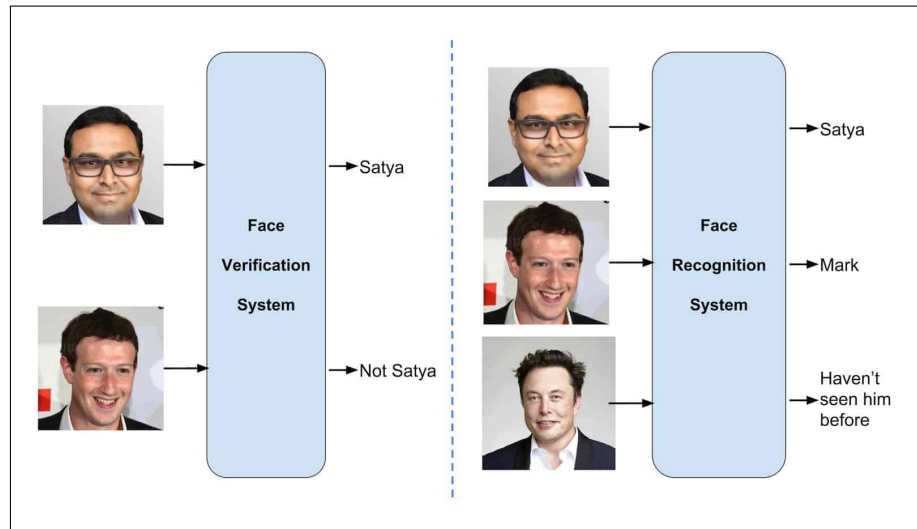


Figura 4.10: Diferencia entre verificación e identificación de rostros.

4.15. Algoritmos para la detección de rostros y extracción de características

Dentro del pipeline de reconocimiento facial, la detección y la extracción de embeddings son etapas críticas que se benefician de algoritmos especializados.

En este trabajo se prestara especial interés en el algoritmo MTCNN para la detección de rostros y FaceNet para la obtención de embeddings.

4.15.1. MTCNN (Multitask Cascaded Convolutional Networks)

Antecedentes

Cuando MTCNN (Multitask Cascaded Convolutional Networks) apareció en escena en 2016, con una presentación más formal en la conferencia ICCV de 2017, rápidamente se convirtió en un referente para la detección facial. ¿Por qué? Porque logra algo que, en ese momento, era un gran avance: combinar la detección de rostros y la alineación de puntos clave faciales en un solo proceso eficiente. En lugar de depender de modelos separados para cada tarea, MTCNN las integra en un pipeline único, lo que reduce significativamente el costo computacional [50].

Arquitectura en Cascada: PNet, RNet, ONet

MTCNN emplea una estructura en cascada compuesta por tres redes neuronales convolucionales (CNNs) que colaboran para refinar progresivamente las propuestas de rostros y detectar los puntos clave [50].

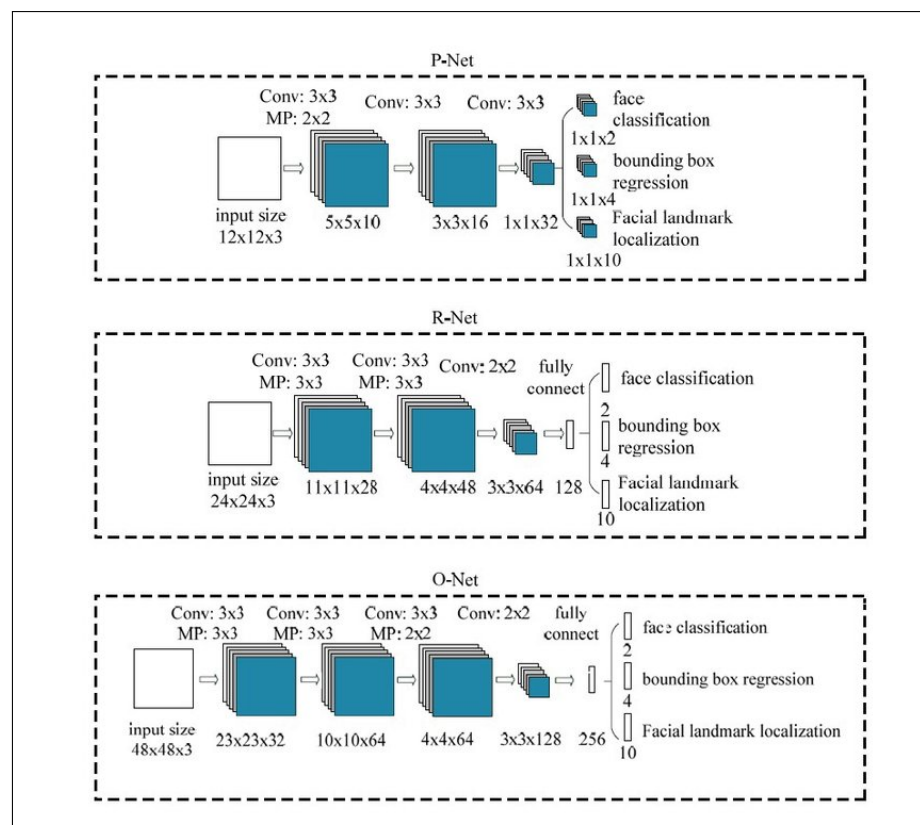


Figura 4.11: Arquitectura de MTCNN.

Dada esta naturaleza en forma de cascada es que se da lugar a un refinamiento progresivo, iniciando con una detección general y avanzando hacia una alineación más precisa.

A continuación, se presenta una breve descripción sobre las redes neuronales que conforman a MTCNN [50]:

1. **PNet (Proposal Network)**: Esta es la red inicial que escanea la imagen en diferentes escalas para generar regiones candidatas de rostros, representadas por cuadros delimitadores. Su función es identificar posibles ubicaciones de rostros de manera eficiente.
2. **RNet (Refinement Network)**: RNet toma las regiones candidatas de rostros generadas por PNet, las refina filtrando los falsos positivos y ajusta aún más los cuadros delimitadores. Esta etapa mejora la precisión de las detecciones iniciales.
3. **ONet (Output Network)**: Como etapa final, ONet refina aún más los cuadros delimitadores y detecta cinco puntos clave faciales: los ojos, la nariz y las comisuras de la boca. Esta red proporciona la salida final de la detección y alineación.

4.15.2. FaceNet

Visión general

FaceNet es un sistema de reconocimiento facial desarrollado por investigadores de Google, presentado por primera vez en la Conferencia IEEE sobre Visión por Computadora y Reconocimiento de Patrones de 2015. Su concepto central es aprender un mapeo, o embedding, directamente desde un conjunto de imágenes faciales a un espacio euclidiano de 128 dimensiones [51].

Una vez que se encuentran los embeddings en este espacio vectorial, su similitud se determina calculando la distancia euclidiana entre los embeddings que se estén comparando.

Además de proporcionar un marco de trabajo para la generación de embeddings de calidad, FaceNet también puede ser usado para crear clusters de personas, lo que lo convierte en un modelo sencillo de entender y robusto hasta la fecha [51].

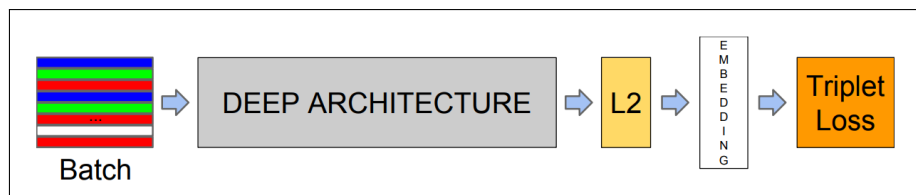


Figura 4.12: Arquitectura general de FaceNet.

Función de pérdida por tripletas (Triplet Loss)

La principal aportación de este modelo con respecto a otros es la implementación de un proceso de aprendizaje basado en Tripletas. Una tripleta hace referencia a un conjunto de tres imágenes, una imagen base, una positiva y una negativa en donde la imagen base es una imagen del rostro de una persona cualquiera, la imagen positiva es una imagen distinta de la misma persona base y, la imagen negativa, es una imagen de una persona distinta a la persona base [51].

Esta función está diseñada para asegurar que los embeddings de la misma persona estén muy cerca entre sí en el espacio vectorial, mientras que los embeddings de personas diferentes estén lo suficientemente lejos [51].

Esta optimización directa del espacio de embedding para el cálculo de similitud es lo que permitió la alta precisión de FaceNet y su influencia en modelos posteriores basados en embeddings [47].

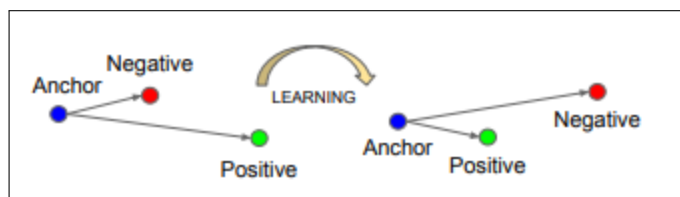


Figura 4.13: Aprendizaje basado en tripletas.

CASIA-WebFace

El conjunto de datos CASIA WebFace fue introducido en el trabajo "Learning Face Representation from Scratch" surgió como una necesidad de contar con conjuntos de datos a gran escala para el entrenamiento de modelos de aprendizaje profundo ya que, este tipo de sistemas requieren de una gran cantidad de datos etiquetados [52].

La principal ventaja de contar con conjuntos de datos grandes como este es la oportunidad que brindan para que nuevas arquitecturas aprendan a identificar y representar los rostros directamente desde cero, eliminando la dependencia de características manuales o los sesgos que pueden acarrear los modelos pre-entrenados [52].

Algunas de las características importantes de este conjunto de datos son [52].:

- **Tamaño:**

Es un conjunto de datos de gran tamaño en comparación con otros como LFW ya que su principal propósito es ser utilizado para el entrenamiento de modelos robustos. Por lo tanto, contiene aproxima-

damente 500 mil imágenes de rostros de un total de 10,575 personas extraídas directamente de la web.

- **Adquisición de imágenes**

Como su propio nombre lo sugiere, las imágenes que conforman a este conjunto de datos fueron obtenidas a través de la web, por lo que presentan una gran variabilidad en las condiciones en las que fueron tomadas, por ejemplo:

- **Iluminación:** Diferentes condiciones de luz (día, noche, sombras, reflejos).
- **Pose:** Varias poses de la cabeza (frontal, perfil, semi-perfil, inclinaciones).
- **Expresión** Diversas expresiones faciales (neutra, feliz, triste, enojada, etc.).
- **Oclusión** Posibles oclusiones parciales del rostro (gafas, cabello, objetos).
- **Fondo Fondos no controlados y variables, lo que añade complejidad.**
- **Calidad de imagen** Las imágenes varían en resolución, nitidez y compresión.
- **Uso:** Dado el tamaño de este conjunto de datos, es posible usarlo para una variedad de tareas dentro del flujo de trabajo de un sistema de reconocimiento facial, por ejemplo, puede usarse para entrenar modelos basados en arquitecturas de redes neuronales profundas y, por otro lado, es posible usar una parte de este para evaluar el rendimiento de modelos que hayan sido entrenados con conjuntos de datos diferentes.

Labeled Face in the Wild

Este conjunto de datos fue creado y mantenido por investigadores de la Universidad de Massachusetts, Amherst y se introdujo en el año 2007. Actualmente, su importancia radica en su papel como punto de referencia para evaluar el rendimiento de algoritmos de reconocimiento facial en condiciones no controladas o como sus propios autores mencionan, en la naturaleza [53].

Sin embargo, a pesar de estar intrínsecamente relacionado con el reconocimiento facial, este conjunto de datos suele usarse especialmente al trabajar el problema de la verificación facial que como se explico anteriormente, consiste en decidir si dos imágenes pertenecen a la misma persona [53].

Dentro de los puntos que diferencian a este conjunto de datos con respecto a otros se destacan los siguientes [53]:



Figura 4.14: Conjunto de datos CASIA WebFace.

- **Tamaño** Contiene aproximadamente 13,233 imágenes de rostros de 5,749 personas únicas. A diferencia de CASIA WebFace, LFW es significativamente más pequeño en términos de número de imágenes y, sobre todo, en el número de identidades.
- **Adquisición de imágenes** El origen de las imágenes que conforman al conjunto de datos es la web, por lo que trae consigo una gran variabilidad y desafíos para el reconocimiento facial como variación intra-clase alta, casos de variación de pose, iluminación y oclusión y la presencia de fondos complejos.
- **Estructura para evaluación** LFW es conocido por su protocolo de evaluación estándar para la verificación facial. Este protocolo define 6,000 pares de imágenes, de los cuales 3000 pares son de la misma persona, 3000 pares son de personas distintas.
- **Uso** Se utiliza principalmente como un marco de referencia para la evaluación de tareas de verificación facial y, comparación de algoritmos de reconocimiento facial.

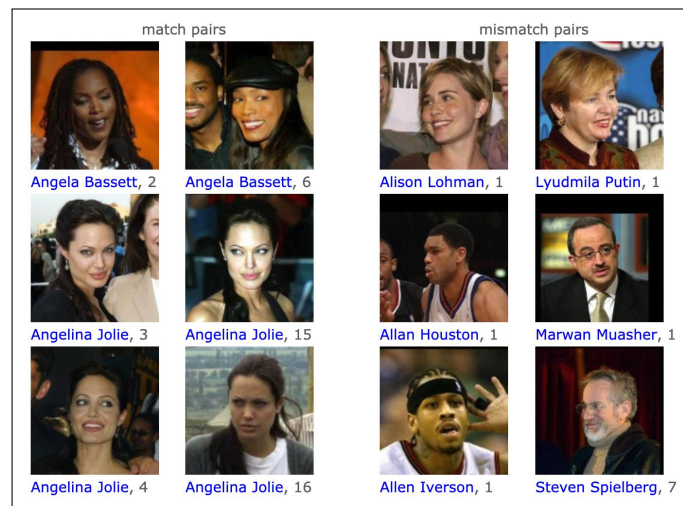


Figura 4.15: Algunos pares en el conjunto LFW.

Este capítulo profundiza en la estructura y construcción del sistema, detallando los componentes clave y las tecnologías empleadas en su desarrollo. Se abordará la implementación de la aplicación móvil, el servidor de Spring Boot, y el servidor de la red neuronal.

5.1. Aplicación Móvil

La aplicación móvil está diseñada para ofrecer una interfaz de usuario intuitiva y reactiva. Su desarrollo se realizó íntegramente utilizando el framework moderno de Android, Jetpack Compose, el cual permite construir interfaces declarativas de manera eficiente. La gestión de dependencias y la configuración del proyecto se realizan a través de Gradle, donde se configuran los módulos necesarios para el desarrollo de la interfaz de usuario y la navegación.

5.1.1. Tecnologías y herramientas utilizadas

- **Lenguaje de programación:** Kotlin (versión 2.1.0)
- **Entorno de desarrollo:** Android Studio (versión 2024.2.1 Patch 3)
- **SDK de Android:** API nivel 35 (versión 15.0)

5.1.2. Arquitectura MVVM (Model-View-ViewModel)

La aplicación sigue el patrón MVVM para separar claramente la lógica del negocio, la UI y los datos. A continuación, se detalla cada componente:

Model (Modelo)

Esta capa es la responsable de la gestión de los datos de la aplicación mediante el acceso y la comunicación con el backend (Java Spring Boot).

ViewModel

Esta capa conecta la lógica de datos con la interfaz de usuario. Se encarga de preparar los datos, aplicar reglas de negocio y actualizar la vista cuando la información cambia.

View (Vista)

Esta capa es la encargada de definir y manejar los elementos visuales de la aplicación. Su función principal es presentar la información al usuario y registrar sus interacciones.

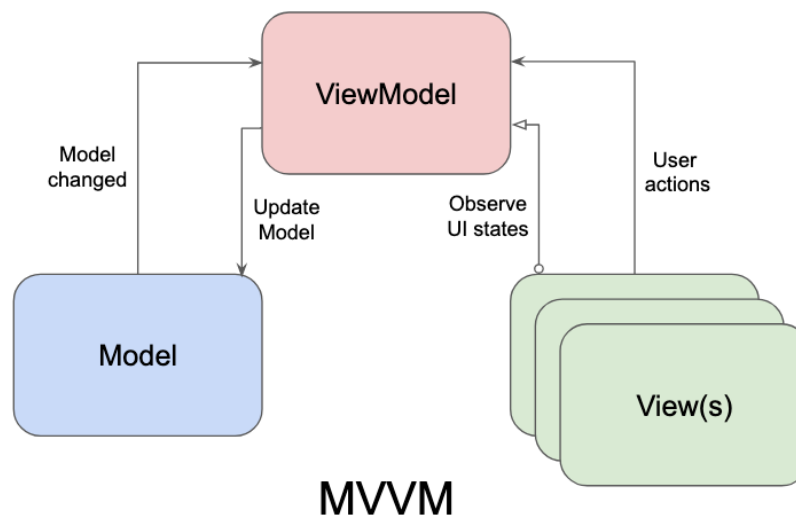


Figura 5.1: Arquitectura MVVM.

5.1.3. Componentes de las pantallas (Jetpack Compose y LaunchEffect)

Las pantallas de la aplicación móvil están compuestas por Composables de Jetpack Compose, que son funciones programables que describen cómo debe ser una parte de la interfaz de usuario. Estos Composables son los bloques fundamentales que construyen la interfaz de usuario, permitiendo un desarrollo modular y reutilizable. Para su funcionamiento, el proyecto utiliza dependencias clave de Jetpack Compose gestionadas por Gradle, tales como:

- `'androidx.compose.foundation'`: Para componentes de interfaz de usuario de bajo nivel y funcionalidades de diseño como fondos, bordes y gestos.
- `'androidx.compose.material3'`: Proporciona componentes de Material Design 3, facilitando la creación de interfaces modernas y consistentes.
- `'androidx.compose.runtime'`: Contiene las APIs principales del tiempo de ejecución de Compose, fundamentales para la composición y recomposición de la interfaz de usuario.
- `'androidx.compose.ui'`: Incluye las APIs básicas de UI de Compose, como elementos gráficos, eventos de entrada y diseños.
- `'androidx.navigation.NavController'`: Es crucial para la gestión de la navegación entre las diferentes pantallas de la aplicación, permitiendo una experiencia de usuario fluida al moverse entre los distintos destinos Composables.

Para la gestión del ciclo de vida de la interfaz de usuario y la ejecución de operaciones con efectos secundarios (como la carga de datos o la inicialización de estados), se utiliza `'LaunchEffect'`. `'LaunchEffect'` es un Composable de Jetpack Compose que se ejecuta cuando su clave cambia y se cancela cuando el Composable abandona la composición o la clave cambia nuevamente. Esto es fundamental para:

- **Carga inicial de datos:** Iniciar la obtención de información desde una fuente externa al entrar en una pantalla específica.
- **Manejo de eventos de ciclo de vida:** Realizar acciones cuando la pantalla se vuelve visible o deja de serlo.
- **Conexión con ViewModels:** `'LaunchEffect'` es el punto donde las pantallas suelen invocar funciones de sus ViewModels asociadas para iniciar procesos asíncronos, como la recuperación de datos de la API.

Además de los componentes visuales principales y la gestión del ciclo de vida, la aplicación hace uso extensivo de los `'Dialog'` (o `'AlertDialog'`) de Jetpack Compose. Estos elementos de interfaz son cruciales para:

- **Mostrar avisos al usuario:** Informar sobre operaciones completadas, errores, confirmaciones o cualquier mensaje importante que requiera la atención del usuario sin interrumpir drásticamente el flujo de la aplicación.
- **Solicitar confirmaciones:** Pedir al usuario que confirme una acción antes de proceder.
- **Presentar entradas cortas:** Recopilar información mínima en un contexto específico.

Estos diálogos mejoran la interacción del usuario al proporcionar retroalimentación contextual y opciones de control claras.

En resumen, la combinación de los Composables de Jetpack Compose para la construcción visual, el sistema de dependencias de Gradle para integrar las librerías esenciales (`'androidx.compose.foundation'`, `'androidx.compose.material3'`, `'androidx.compose.runtime'`, `'androidx.compose.ui'`,

'androidx.navigation.NavController'), 'LaunchEffect' para la orquestación de efectos secundarios en el ciclo de vida de la interfaz de usuario, y la utilización estratégica de 'Dialog' para los avisos al usuario, asegura que las pantallas de la aplicación móvil sean dinámicas, eficientes, navegables y respondan adecuadamente a las interacciones del usuario y a los cambios de estado del sistema.

5.2. Lógica de los ViewModels

La aplicación móvil se complementa y es orquestada por los ViewModels, que residen en el paquete 'com.example.prueba3.Views'. Estos componentes son fundamentales en una arquitectura moderna de Android, ya que actúan como un puente robusto entre la interfaz de usuario (las pantallas de Jetpack Compose) y la lógica de negocio o las fuentes de datos. Su principal ventaja es que están diseñados para sobrevivir a los cambios de configuración del dispositivo (como rotaciones de pantalla), manteniendo el estado de la interfaz de usuario y los datos sin interrupciones.

Los ViewModels extienden la clase 'androidx.lifecycle.ViewModel' y hacen un uso extensivo de las capacidades de concurrencia y gestión de estados de Kotlin Coroutines y Kotlin Flow para operaciones asíncronas y reactividad de datos.

5.2.1. Tecnologías y Componentes Clave en ViewModels

Para la gestión eficiente del estado de la interfaz de usuario y la ejecución de operaciones asíncronas, los ViewModels utilizan las siguientes tecnologías y componentes:

- **'androidx.lifecycle.ViewModel'**: Es la clase base principal de la que heredan todos los ViewModels de la aplicación. Proporciona un ámbito de ciclo de vida ('viewModelScope') que permite ejecutar operaciones asíncronas que se cancelan automáticamente cuando el ViewModel es destruido, evitando fugas de memoria.
- **'androidx.lifecycle.viewModelScope'**: Es un 'CoroutineScope' predefinido que pertenece al ViewModel. Esto significa que cualquier coroutine lanzada dentro de 'viewModelScope' se ejecutará en un hilo en segundo plano y se cancelará automáticamente si el ViewModel se destruye. Esto simplifica enormemente la gestión de operaciones de larga duración, como llamadas a la red o consultas a la base de datos.
- **'kotlinx.coroutines' (Coroutines)**: Los ViewModels emplean Coroutines para manejar la asincronía de manera estructurada y concisa. Permiten escribir código asíncrono que parece síncrono, mejorando la legibilidad y la depuración. Las coroutines son esenciales para:
 - 'kotlinx.coroutines.launch': Para iniciar una nueva coroutine en un 'CoroutineScope' específico, como 'viewModelScope'.
 - 'kotlinx.coroutines.Dispatchers': Para especificar el hilo en el que se ejecutará una coroutine (e.g., 'Dispatchers.IO' para operaciones de red o base de datos, 'Dispatchers.Main' para actualizaciones de interfaz de usuario).
 - 'kotlinx.coroutines.withContext': Para cambiar el contexto de ejecución de una coroutine a un 'Dispatcher' diferente de manera segura, útil para alternar entre hilos de secundarios y el hilo principal.

- **'kotlinx.coroutines.flow' (StateFlow y MutableStateFlow):** Los ViewModels gestionan el estado de la interfaz de usuario y exponen los datos de manera reactiva utilizando Kotlin Flow, específicamente:
 - 'kotlinx.coroutines.flow.MutableStateFlow<T>': Es un tipo de 'Flow' que puede emitir actualizaciones de estado mutables. Los ViewModels lo utilizan internamente para mantener el estado actual de la UI (ej., datos a mostrar, estado de carga, mensajes de error).
 - 'kotlinx.coroutines.flow.StateFlow<T>': Es una versión de solo lectura de 'MutableStateFlow'. Los ViewModels exponen sus 'MutableStateFlow' internos como 'StateFlow' a las Composables de la UI ('.asStateFlow()'). Esto garantiza que la interfaz de usuario solo pueda observar los cambios de estado, sin poder modificarlos directamente, promoviendo la inmutabilidad y una gestión de estado más segura.
- **Manejo de Respuestas de API y Errores:** Para la interacción con las APIs y el procesamiento de respuestas, los ViewModels utilizan:
 - 'retrofit2.HttpException': Esta excepción se utiliza para capturar y manejar errores específicos que ocurren durante las llamadas HTTP realizadas por Retrofit (por ejemplo, errores de servidor como 404, 500, etc.). Permite a los ViewModels reaccionar a problemas de red o de API de forma controlada.
 - 'org.json.JSONObject': Se emplea para el análisis y la manipulación de datos en formato JSON, especialmente útil para extraer información de cuerpos de respuesta complejos o para construir cuerpos de solicitud.

En resumen, los ViewModels son el corazón de la lógica de presentación, utilizando un conjunto robusto de herramientas de Jetpack y Kotlin Coroutines/Flow para gestionar el estado de la interfaz de usuario de forma eficiente, reactiva y segura, facilitando la comunicación con las capas de datos y la lógica de negocio del sistema.

5.3. Lógica de las Instancias de Retrofit

La aplicación móvil se comunica con los servicios backend a través de Retrofit, una biblioteca de cliente HTTP de tipo seguro para Android y Java. Retrofit simplifica enormemente el proceso de realizar solicitudes de red, mapeando APIs RESTful a interfaces Kotlin con anotaciones. Para gestionar la interacción con los distintos servidores, el sistema utiliza dos instancias de Retrofit, cada una configurada específicamente para su respectivo servicio.

5.3.1. Configuración de Instancias de Retrofit

Ambas instancias de Retrofit utilizan 'GsonConverterFactory' para la serialización y deserialización de objetos JSON, y 'OkHttpClient' para la gestión de las operaciones HTTP subyacentes, incluyendo la configuración de timeouts.

Instancia para el Servidor Java Spring-Boot

Esta instancia de Retrofit está dedicada a la comunicación con el servidor principal de lógica de negocio, el Servidor Java Spring-Boot, desplegado en Azure. Su configuración prioriza la fiabilidad y un balance adecuado en los tiempos de respuesta para las operaciones transaccionales.

- **'RetrofitInstance' (Objeto Singleton):** Implementado como un objeto singleton para asegurar que solo exista una única instancia de Retrofit para este servidor en toda la aplicación, optimizando recursos.
- **'OkHttpClient':** Se utiliza un cliente HTTP personalizado con los siguientes timeouts:
 - **'connectTimeout(10, TimeUnit.SECONDS)'**: Establece un tiempo máximo de 10 segundos para establecer una conexión con el servidor.
 - **'writeTimeout(120, TimeUnit.SECONDS)'**: Permite hasta 120 segundos para enviar un cuerpo de solicitud al servidor.
 - **'readTimeout(120, TimeUnit.SECONDS)'**: Define un límite de 120 segundos para recibir una respuesta completa del servidor.

Estos tiempos están ajustados para manejar operaciones que podrían requerir un procesamiento moderado en el servidor, sin ser excesivamente laxos.

- **'GsonConverterFactory.create()'**: Utilizado para convertir automáticamente objetos Kotlin en JSON para las solicitudes y viceversa para las respuestas.

Esta configuración asegura una comunicación eficiente y tolerante a cierto nivel de latencia con el servidor principal de la aplicación.

Instancia para el Servidor de la Red Neuronal

Esta instancia de Retrofit está optimizada para interactuar con el **Servidor Red Neuronal**, también alojado en Azure, que es responsable específicamente de las funcionalidades de reconocimiento facial. Dada la naturaleza de estas operaciones (envío de imágenes y procesamiento intensivo), se requieren timeouts más generosos.

- **'RetrofitCamaralInstance' (Objeto Singleton):** Similar al anterior, es un singleton para una gestión eficiente de la conexión con el servidor de la red neuronal.
- **'OkHttpClient' con 'HttpLoggingInterceptor':**
 - **'HttpLoggingInterceptor().apply level = HttpLoggingInterceptor.Level.BODY':** Se incluye un interceptor de logging para fines de depuración y monitoreo. Este interceptor permite registrar en detalle (cuerpo incluido) las solicitudes y respuestas HTTP, lo cual es invaluable durante el desarrollo y la fase de pruebas para entender el flujo de datos.
 - **'connectTimeout(300, TimeUnit.SECONDS)':** Configurado a 300 segundos (5 minutos) para permitir conexiones en entornos con latencia o carga.
 - **'writeTimeout(300, TimeUnit.SECONDS)':** Permite hasta 300 segundos para la escritura de datos, esencial para el envío de imágenes que pueden ser de tamaño considerable.
 - **'readTimeout(300, TimeUnit.SECONDS)':** Un timeout de lectura de 300 segundos para esperar la respuesta del servidor, anticipando que el procesamiento de la red neuronal puede ser intensivo y tomar tiempo.
- **'GsonConverterFactory.create()':** También utilizado para la serialización/deserialización JSON.

La configuración de esta instancia subraya la necesidad de manejar operaciones que implican el envío de datos más pesados (imágenes) y un procesamiento que puede ser más lento, asegurando que la conexión no se cierre prematuramente.

5.4. Lógica de las Data Classes

Para la correcta interpretación y manipulación de la información intercambiada entre la aplicación móvil y los servidores (tanto el de Spring Boot como el de la Red Neuronal), el sistema hace uso de Data Classes de Kotlin. Estas clases son un componente esencial en la arquitectura de datos del proyecto, ubicadas principalmente en el paquete `'com.example.prueba3.DataClasses'`.

5.4.1. Propósito y Características de las Data Classes

Las Data Classes en Kotlin están específicamente diseñadas para contener datos. El compilador de Kotlin genera automáticamente funciones útiles para ellas, como `'equals()'`, `'hashCode()'`, `'toString()'`, `'copy()'`, y `'componentN()'`, lo que las hace ideales para modelar la estructura de los datos que se reciben de las APIs o se envían a ellas.

Su rol principal en el sistema es:

- **Modelado de Estructuras de Datos:** Definen de forma clara y concisa la estructura de los objetos JSON que se serializan o deserializan a través de Retrofit y Gson. Cada propiedad de una Data Class se mapea a un campo correspondiente en el JSON.
- **Inmutabilidad y Seguridad:** Por convención, las propiedades de las Data Classes se declaran como `'val'` (valores inmutables), lo que promueve una gestión de datos más segura y predecible a lo largo de la aplicación.
- **Facilidad de Uso con Retrofit y Gson:** Son perfectamente compatibles con `'GsonConverterFactory'` de Retrofit, lo que permite que el proceso de conversión de JSON a objetos Kotlin y viceversa sea automático y eficiente.

5.4.2. Ejemplos de Data Classes Implementadas

A continuación, se presentan ejemplos de Data Classes utilizadas en el proyecto para ilustrar su función en el modelado de la información:

Estas Data Classes, junto con otras implementadas en el proyecto, aseguran que los datos sean manejados de manera estructurada y segura a lo largo de la aplicación móvil, facilitando la comunicación con los servicios backend y la manipulación de la lógica de negocio.

5.5. Sistema Web Django

El sistema web fue implementado para representar el proceso de un escenario real completo que va desde la inscripción de un alumno a una escuela, la creación de su credencial, hasta su inscripción a un ETS en caso de reprobación alguna asignatura.

Este sistema no se desplegó en ningún servicio en la nube puesto que para los fines del proyecto no fue necesario. Sin embargo, este sistema se comunica con el servidor Java, el cual contiene todo el backend, del cual hablaremos más adelante.

Este sistema web fue implementado con el framework Django, el cual tradicionalmente se basa en el patrón de arquitectura Modelo-Vista-Plantilla (MTV por sus siglas en inglés). Sin embargo, para este caso en particular, el modelo no se encuentra implementado dentro de Django, ya que, tanto la lógica como la persistencia están completamente administradas por el servidor Java, tal y como se mencionó anteriormente.

Apesar de no seguir fielmente este patrón de arquitectura, en este sistema se adaptó de tal manera en la que la vista (archivos Views) actúa como el controlador, es decir, recibe las peticiones del usuario, consume la API externa y procesa los datos recibidos. Por otro lado, las plantillas (archivos template) generan las páginas HTML que se le mostrarán al usuario, y a su vez, recibe los datos procesados desde la vista.

El tipo de comunicación que se usa en este sistema junto con el servidor Java, es como el de una arquitectura cliente servidor descoplada, en el que el sistema web Django actúa como el cliente, consumiendo servicios por una API REST desarrollada en el servidor mediante peticiones HTTP y con un intercambio de datos en formato JSON.

5.6. Servidor Java Spring Boot

El servidor principal está diseñado con Spring Boot y desplegado en Azure como un App Service. Su despliegue en la nube permite la disponibilidad en línea para poder ser consumido por otras aplicaciones, como el sistema web desarrollado en Django y la aplicación móvil desarrollada en Kotlin.

El sistema fue implementado con una arquitectura en capas, es decir, cada capa tiene una responsabilidad específica. Las capas en las que está dividido el servidor son:

- Capa de presentación: Se encarga de manejar las solicitudes HTTP, la interacción con el usuario y la presentación de los datos. Aquí entran en juego los archivos controlador, es decir, los que tienen la anotación `@Controller` y `@RestController`.
- Capa de negocio o servicio: Contiene la lógica de negocio central de la aplicación. Los archivos de servicio son los que se encargan de manejar todas las operaciones y las que interactúan con la capa de persistencia. Estos archivos presentan la anotación `@Service`.
- Capa de persistencia: Maneja la interacción con la base de datos. Las clases encargadas de esto son los archivos repositorio, es decir, los que tienen la anotación `@Repository`, o, en su defecto, las que extienden interfaces de Spring Data JPA.
- Capa de Base de Datos: Es decir, la capa en donde se almacenan todos los datos.

La comunicación entre cada una de estas capas se realiza mediante objetos DTO, mientras que se comunica con otros servicios mediante servicios REST. Por otra parte, es importante mencionar que para la implementar la base de datos, se hizo uso de JPA para el mapeo de las clases a las tablas dentro de la base de datos.

Se definieron entidades como 'Alumno', 'UnidadAcademica' y 'UnidadAprendizaje', las cuales tienen la anotación @Entity y son mapeadas a tablas PostgreSQL. La relación entre tablas se configuró con anotaciones @OneToMany, @ManyToOne, entre otras.

Por otro lado, para la capa de persistencia, como se mencionó anteriormente, se usaron archivos repositorio, los cuales son interfaces que extienden de JpaRepository, los cuales permiten el acceso a datos sin necesidad de usar SQL manual.

En los servicios se crearon archivos tales como AlumnoServicio, DocenteServicio, entre otros, en los cuales se encargan de guardar nuevos registros, realizar consultas o incluso actualizaciones.

Por último, los controladores, que como se mencionó, son los que se encargan de recibir peticiones HTTP y de devolver respuestas JSON al usuario.

Adicional a esto, se utilizaron servicios externos como 'Cloudinary' y 'Firebase'. El primero se usó para guardar las imágenes que se toman al momento del registro de los alumnos desde la simulación del SAES, mientras que el Firebase se usó para manejar los mensajes dentro de la aplicación móvil.

El aseguramiento de la calidad es un pilar fundamental en el desarrollo de cualquier sistema de software. En el presente capítulo, se detalla el proceso de validación del sistema propuesto a través de la ejecución de pruebas enfocadas en sus principales casos de uso. El objetivo es verificar que cada funcionalidad crucial del sistema se comporta de acuerdo con las especificaciones y requisitos definidos.

6.1. Pruebas de humo y detección temprana de errores

Durante la fase de codificación y desarrollo del sistema, se llevó a cabo varias pruebas de humo de manera continua. Estas pruebas informales, realizadas al finalizar cada sección o módulo de código, fueron cruciales para detectar defectos críticos en las etapas más tempranas del ciclo de desarrollo. Esta práctica permitió identificar y corregir errores conforme se presentaban, asegurando la funcionalidad básica de cada componente antes de la integración completa.

Entre los errores más destacables que se identificaron y subsanaron durante estas pruebas iniciales, se encuentran:

- **Errores de formato en la comunicación con el servidor:** Se observó que, en cualquier interacción con el servidor (tanto el de Spring Boot como el de la Red Neuronal), se producían errores de servidor ('Server Error') si el formato de la información enviada al servidor no coincidía con el esperado o, a la inversa, si la respuesta del servidor no se ajustaba a la estructura de datos que la aplicación móvil estaba preparada para recibir. Esto obligó a una revisión exhaustiva de la serialización y deserialización de datos para garantizar la coherencia.
- **Limitación de tamaño de imagen en el módulo de Reconocimiento Facial:** Se identificó un error crítico en el módulo de reconocimiento facial cuando un dispositivo móvil intentaba enviar una fotografía de muy alta resolución o de gran tamaño. El servidor de Azure, donde reside el modelo de red neuronal,

cancelaba la petición al exceder los límites de tamaño permitidos, lo que resultaba en fallos de comunicación. La solución implicó la implementación de un proceso de compresión de imágenes en el lado del cliente (la aplicación móvil) antes de su envío.

- **Manejo de errores en detección de rostros del módulo de reconocimiento facial:** Otro error significativo en el módulo de reconocimiento facial era que si la imagen enviada al servidor de la red neuronal no contenía un rostro detectable, el servidor fallaba inesperadamente en lugar de retornar un error manejable. Esto causaba un comportamiento no deseado en la aplicación y se requirió una modificación para que el servidor, al no detectar un rostro, respondiera con un error específico que pudiera ser capturado y gestionado adecuadamente por la aplicación móvil, informando al usuario de la situación de manera clara.

La detección temprana y corrección de estos y otros errores mediante las pruebas de humo fue vital para el correcto funcionamiento del sistema final, permitiendo avanzar a fases de prueba más formales con una base más sólida.

6.2. Pruebas de Integración

Además de las pruebas de humo realizadas durante el desarrollo, se llevaron a cabo pruebas de integración para asegurar que los distintos componentes del sistema interactuaran correctamente entre sí. Estas pruebas se centraron específicamente en la conexión y el flujo de datos entre la aplicación móvil, el servidor de Spring Boot, el servidor de la red neuronal y la simulación del SAES.

El objetivo principal de estas pruebas fue verificar que:

- La aplicación móvil pudiera establecer conexiones exitosas con ambos servidores.
- Los datos se enviaran y recibieran correctamente entre la aplicación móvil y el servidor de Spring Boot (ej. inicio de sesión, gestión de datos de alumnos/ETS).
- La comunicación para el reconocimiento facial funcionara fluidamente entre la aplicación móvil y el servidor de la red neuronal, incluyendo el envío de imágenes y la recepción de resultados de verificación.
- La **simulación del SAES** se conectara adecuadamente con el servidor de la red neuronal para llevar a cabo los procesos de registro de usuarios y la generación de embeddings faciales.

Estas pruebas de integración fueron fundamentales para validar la arquitectura de comunicación de los servicios desplegados.

6.3. Pruebas de Subsistema

Adicionalmente a las pruebas de humo e integración, se realizaron pruebas de subsistema exhaustivas para verificar el correcto funcionamiento y la interacción de conjuntos específicos de componentes que conforman funcionalidades clave del sistema. Estas pruebas se diseñaron para asegurar que cada módulo principal operara de manera coherente y eficiente, tanto de forma independiente como en conjunto con sus dependencias directas. Los subsistemas clave evaluados incluyen:

- **Gestión y visualización de reportes por docentes:** Se comprobó la funcionalidad completa de la creación y visualización de reportes por parte del personal docente. Esto incluyó la verificación de la generación de reportes de asistencia (con o sin el uso de reconocimiento facial), la correcta agregación de datos y la presentación adecuada de la información en la interfaz de usuario del docente.
- **Registro de asistencia escolar por personal de seguridad:** Se validó el flujo completo de registro de asistencia de alumnos a la escuela a través del personal de seguridad. Esta prueba abarcó desde la captura de la identificación del alumno hasta el registro exitoso de su entrada en la base de datos, asegurando la fiabilidad del proceso en los puntos de acceso.
- **Verificación de reconocimiento facial por alumnos:** Se realizaron pruebas detalladas sobre la funcionalidad de reconocimiento facial utilizada por los alumnos para verificar su identidad. Esto implicó la verificación de la captura de la imagen, el envío al servidor de la red neuronal y la correcta interpretación de la respuesta para determinar la identidad del alumno.
- **Funcionamiento del servidor Spring Boot:** Se sometió el servidor de Spring Boot a pruebas de subsistema intensivas para asegurar el correcto procesamiento de la lógica de negocio, la gestión de la base de datos (PostgreSQL) y la orquestación de las peticiones recibidas desde la aplicación móvil. Esto incluyó la verificación de la autenticación de usuarios, la gestión de datos de alumnos y ETS, y la integridad de las transacciones.
- **Funcionamiento de las funciones de creación de Embeddings y reconocimiento facial en el Servidor de la Red Neuronal:** El servidor de la red neuronal fue específicamente probado para validar la exactitud y eficiencia de sus funciones principales. Se verificó el correcto funcionamiento de:
 - **La creación de embeddings:** La capacidad de transformar una imagen facial en un vector numérico único que representa las características faciales.
 - **El reconocimiento facial:** La comparación de embeddings para determinar la similitud entre rostros y, por ende, la identidad o no identidad de un individuo.

Estas pruebas fueron cruciales para asegurar la precisión y el rendimiento del componente de inteligencia artificial.

Las pruebas de subsistema permitieron identificar y resolver interacciones complejas entre módulos, garantizando que cada pieza fundamental del sistema funcionara de manera óptima y contribuyera a la robustez general de la solución.

6.4. Pruebas de Aceptación: Encuestas de Usuario

La fase final de validación del sistema se llevó a cabo mediante pruebas de aceptación de usuario, específicamente a través de la aplicación de encuestas. Estas encuestas fueron diseñadas para recopilar la percepción directa de los usuarios finales (docentes, personal de seguridad y alumnos) sobre la usabilidad, la eficiencia, la satisfacción general y la efectividad del sistema en un entorno cercano al real.

El objetivo principal de estas encuestas fue:

- **Validar la satisfacción del usuario:** Medir el grado en que el sistema cumple con las expectativas y necesidades de los usuarios, más allá de la mera funcionalidad técnica.
- **Identificar oportunidades de mejora:** Recopilar retroalimentación cualitativa y cuantitativa que permita identificar áreas de mejora en la interfaz de usuario, la experiencia de usuario y las funcionalidades futuras del sistema.
- **Confirmar la aceptación del sistema:** Determinar si el sistema es adecuado para su implementación y uso en el entorno operativo previsto, asegurando que los usuarios se sientan cómodos y productivos con la herramienta.
- **Evaluar la efectividad percibida:** Cuantificar la percepción de los usuarios sobre cómo el sistema resuelve los problemas planteados, como la agilización del registro o la prevención de la suplantación de identidad.

Los resultados de estas encuestas, analizados detalladamente en el Capítulo de Resultados, fueron cruciales para obtener una perspectiva valiosa sobre la experiencia de usuario y confirmaron la viabilidad y aceptación del sistema en el contexto académico. La información recolectada no solo sirvió para validar el trabajo realizado, sino que también guió las recomendaciones para el trabajo futuro del proyecto.

6.5. Pruebas funcionales

Para cada caso de uso evaluado, se presentará un informe detallado que incluye la descripción del caso de prueba, los supuestos y condiciones previas necesarias para su ejecución, los datos de prueba utilizados, los pasos específicos a ejecutar, el resultado esperado y el resultado real obtenido. Asimismo, se documentarán los errores detectados, si los hubo, y el estado final de la prueba. Esta metodología permite una revisión exhaustiva del comportamiento del sistema frente a las interacciones clave de los usuarios.

6.5.1. Caso de prueba CP01 - Inicio de sesión de la app móvil

CASO DE PRUEBA - CP01 Inicio de sesión de la app móvil	
Caso de uso relacionado	CU-01 Inicio de sesión de la app móvil
Descripción	Validar la funcionalidad del inicio de sesión para todos los usuarios registrados dentro del sistema.
Cantidad de ejecuciones	100
Realizado por:	Daniel Martín Huertas Ramírez
Supuestos y Condiciones Previas	• El usuario debe de haber instalado la aplicación en su celular. • El usuario debe de estar registrado en la base de datos.

Datos de Prueba	Boleta en caso de ser alumno, RFC en caso de ser personal de seguridad o personal académico.
Pasos a Ejecutar	1. Ingresar la boleta o RFC dependiendo del tipo de usuario, en el campo de usuario. 2. Ingresar la boleta o RFC dependiendo del tipo de usuario, en el campo de contraseña. 3. Presionar el botón 'Entrar'.
Resultado Esperado	Redirige al usuario a su pantalla principal. Dependiendo del tipo de usuario que sea, lo redirigirá a una pantalla específica en el que podrá seleccionar entre diferentes opciones a realizar.
Errores Detectados	Ninguno.
Resultado Real y Condiciones Posteriores	En caso de tener conexión a internet, el usuario es redirigido a su pantalla de inicio. En caso contrario, despliega un 'Error de conexión'.
Estado	Aprobado - Si el logeo es exitoso, el usuario es redirigido a su pantalla de bienvenida específica.

6.5.2. Caso de prueba CP02 - Consultar calendario escolar

CASO DE PRUEBA - CP02 Consultar calendario escolar	
Caso de uso relacionado	CU-02 Consultar calendario escolar
Descripción	Validar la funcionalidad de la obtención del calendario escolar del IPN directamente de su página oficial, así como la visualización correcta del calendario en formato de imagen. Así también, validar la funcionalidad del cálculo de días faltantes para un periodo de ETS.
Cantidad de ejecuciones	100
Realizado por:	Daniel Martín Huertas Ramírez
Supuestos y Condiciones Previas	- El usuario debió de haber iniciado sesión en el sistema. - El usuario debió de ingresar a la pantalla  IU02 Pantalla Consultar calendario escolar - El usuario debe de tener una buena conexión a internet.

Datos de Prueba	• Ninguno.
Pasos a Ejecutar	1. El usuario presiona el ícono del calendario que se encuentra en el menú inferior de la aplicación. 2. El usuario presiona el botón 'Calcular cuántos días faltan para el periodo de ETS'.
Resultado Esperado	• Visualización de la imagen del calendario escolar actual del IPN. • Mensaje de los días faltantes para el periodo de ETS, si este ya pasó o si se encuentra en uno.
Errores Detectados	El mensaje no informaba al usuario si, de acuerdo a la fecha actual, se encontraba en un periodo de ETS.
Resultado Real y Condiciones Posteriores	• El sistema le informaba al usuario que el periodo de ETS ya había pasado apesar de que se encontraba en un periodo actual. • El sistema no reconocía si había dos periodos de ETS dentro de un mismo periodo escolar. • Se realizó un ajuste en la lógica del negocio para que devolviera mensajes más específicos y un cambio en la consulta para que reconociera los dos periodos de ETS.
Estado	Aprobado - El error fue corregido y la funcionalidad opera correctamente.

6.5.3. Caso de prueba CP05 - Consultar ETS asignados

CASO DE PRUEBA - CP05 Consultar ETS asignados	
Caso de uso relacionado	CU-05 Consultar ETS asignados
Descripción	Validar que el docente pueda consultar todos lo ETS y los ETS que tiene que aplicar .
Cantidad de ejecuciones	100
Realizado por:	Luis Antonio Flores Esquivel

Supuestos y Condiciones Previas	• El docente ha iniciado sesión en el sistema. • Existen ETS asignados al docente. • Existen ETS dados de alta.
Datos de Prueba	• nombre de usuario del docente. • Lista de ETS dados de alta.
Pasos a Ejecutar	1. Iniciar sesión como docente. 2. Seleccionar la opción ‘ETS’ desde el menú.
Resultado Esperado	Los ETS que están dados de alta se mostraran en pantalla.
Errores Detectados	• Para el filtro de ETS, mas específicamente el de ‘Mis ETS’ no se mostraba ningún ETS.
Resultado Real y Condiciones Posteriores	• Se modifico la lógica del envío de la solicitud de obtención los ETS para que devolvieran otra variable en la que se especificasen los ETS que el usuario tenía asignados. • Se verificó el correcto funcionamiento tanto en base de datos como en la interfaz.
Estado	Aprobado - El error fue corregido y la funcionalidad opera correctamente.

6.5.4. Caso de prueba CP06 - Mostrar información de los ETS asignados

CASO DE PRUEBA - CP06 Mostrar información de los ETS asignados	
Caso de uso relacionado	CU-06 Mostrar información de los ETS asignados
Descripción	Validar que el docente pueda revisar los datos específicos del ETS seleccionado.
Cantidad de ejecuciones	100
Realizado por:	Luis Antonio Flores Esquivel

Supuestos y Condiciones Previas	• El docente ha iniciado sesión en el sistema. • Existen ETS dados de alta. • El docente eligió un ETS. • Hay información del ETS seleccionado.
Datos de Prueba	• El docente se le muestra los detalles de ETS asignado: tipo de ETS, unidad de aprendizaje, periodo, fecha, turno, cupo, duración, salones (con tipo de salón por cada salón), Hora.
Pasos a Ejecutar	1. Iniciar sesión como docente. 2. Seleccionar la opción “ETS” desde el menú. 3. Elegir un ETS disponible en la lista. 4. Dar clic en el ETS. 5. Revisa los datos mostrados.
Resultado Esperado	Se le muestra al docente los datos del ETS seleccionado.
Errores Detectados	• En ocasiones al cambiar rápido entre ETS la aplicación se cerraba abruptamente.
Resultado Real y Condiciones Posteriores	• Se modificó el orden de obtención y asignación de los datos, para que nos aseguráramos que los datos fueran obtenidos antes de ser asignados a las variables (evitar asignación de null y/o datos vacíos). • Se verificó el correcto funcionamiento tanto en base de datos como en la interfaz.
Estado	Aprobado - El error fue corregido y la funcionalidad opera correctamente.

6.5.5. Caso de prueba CP07 - Solicitar reemplazo de ETS

CASO DE PRUEBA - CP07 Solicitar reemplazo de ETS	
Caso de uso relacionado	CU-07 Solicitar reemplazo de ETS

Descripción	Validar que el docente pueda solicitar un reemplazo para un ETS seleccionado, ingresando una razón.
Cantidad de ejecuciones	100
Realizado por:	Luis Antonio Flores Esquivel
Supuestos y Condiciones Previas	• El docente ha iniciado sesión en el sistema. • Existen ETS asignados al docente. • El botón "Solicitar reemplazo" se encuentra habilitado para los ETS disponibles.
Datos de Prueba	• ETS asignado al docente con fecha activa. • Texto de razón de la solicitud de reemplazo.
Pasos a Ejecutar	1. Iniciar sesión como docente. 2. Seleccionar la opción "ETS" desde el menú. 3. Elegir un ETS disponible en la lista. 4. Dar clic en el botón "Solicitar reemplazo". 5. Ingresar la razón. 6. Enviar la solicitud.
Resultado Esperado	La solicitud de reemplazo debe registrarse exitosamente y mostrarse con el estatus "Pendiente".
Errores Detectados	• Inicialmente, al enviar la solicitud, no se reflejaba el estatus de "Pendiente". • Se identificó un error en la lógica de actualización del estado posterior al envío.
Resultado Real y Condiciones Posteriores	• Tras modificar la lógica de persistencia y actualización de estado en el backend, el sistema ahora registra correctamente la solicitud con estatus "Pendiente". • Se verificó el correcto funcionamiento tanto en base de datos como en la interfaz.

Estado	Aprobado - El error fue corregido y la funcionalidad opera correctamente.
---------------	--

6.5.6. Caso de prueba CP08 - Consultar lista de alumnos inscritos a un ETS

CASO DE PRUEBA - CP08 Consultar lista de alumnos inscritos a un ETS	
Caso de uso relacionado	CU-08 Consultar lista de alumnos inscritos a un ETS
Descripción	Validar que el docente pueda revisar la lista de alumnos inscritos al ETS seleccionado.
Cantidad de ejecuciones	100
Realizado por:	Luis Antonio Flores Esquivel
Supuestos y Condiciones Previas	• El docente ha iniciado sesión en el sistema. • Existen ETS dados de alta. • El docente eligió un ETS. • Hay información del ETS seleccionado. • Hay alumnos inscritos al ETS.
Datos de Prueba	• La lista de alumnos inscritos al ETS.
Pasos a Ejecutar	1. Iniciar sesión como docente. 2. Seleccionar la opción “ETS” desde el menú. 3. Elegir un ETS disponible en la lista. 4. Dar clic en el ETS. 5. Revisa los datos mostrados. 6. El docente elige la opción “Ir a la lista de alumnos” 7. El docente revisa la lista de alumnos inscritos al ETS
Resultado Esperado	Se le muestra al docente la lista de alumnos inscritos al ETS.
Errores Detectados	• Ninguno.

Resultado Real y Condiciones Posteriores	Se verificó el correcto funcionamiento tanto en base de datos como en la interfaz.
Estado	Aprobado - la funcionalidad opera correctamente.

6.5.7. Caso de prueba CP09 - Tomar asistencias a los ETS

CASO DE PRUEBA - CP09 Tomar asistencias a los ETS	
Caso de uso relacionado	CU-09 Tomar asistencias a los ETS
Descripción	Validar que el docente pueda crear reportes de asistencias e incidencias.
Cantidad de ejecuciones	100
Realizado por:	Alejandra De la cruz De la cruz
Supuestos y Condiciones Previas	- El docente ha iniciado sesión en el sistema. - Existen ETS dados de alta. - El docente eligió un ETS. - Hay información del ETS seleccionado. - Hay alumnos inscritos al ETS.
Datos de Prueba	razón, motivo y si se realiza reconocimiento facial; foto y precisión.
Pasos a Ejecutar	1. Iniciar sesión como docente. 2. Seleccionar la opción "ETS" desde el menú. 3. Elegir un ETS disponible en la lista. 4. Dar clic en el ETS. 5. Revisa los datos mostrados. 6. El docente elige la opción "Ir a la lista de alumnos" 7. El docente revisa la lista de alumnos inscritos al ETS 8. El docente elige un alumno para crear un reporte de asistencia/incidencia.

Resultado Esperado	Se crea el reporte correctamente.
Errores Detectados	• Al tomar la foto para el reconocimiento facial en algunos dispositivos salía volteada y no dejaba que se realizara la correctamente el reconocimiento facial. • Al mandar la imagen al servidor de la red neuronal, si la imagen es muy grande el servidor corta la conexión. • La imagen se estira si no se re escala correctamente y altera la información para el reconocimiento facial.
Resultado Real y Condiciones Posteriores	• Para el error de la imagen volteada, e aplico un método que detecta la orientación de la foto y la rota al estado normal tomando en cuenta 0 grados como el estado de orientación correcto y dependiendo de la orientación se voltea hasta llegar a los 0 grados. • Para el error del tamaño al enviar al servidor se aplico un método de re escalado a un tamaño de 640 x 480 pixeles, esto después de la corrección de la orientación. • Para el error de que la foto se estire, se realizo una escala entre las dimensiones de la foto para mantener esa escala al re dimensionar la foto. • Se verificó el correcto funcionamiento tanto en base de datos como en la interfaz.
Estado	Aprobado - la funcionalidad opera correctamente.

6.5.8. Caso de prueba CP11 - Mostrar la foto e información del alumno

CASO DE PRUEBA - CP11 Mostrar la foto e información del alumno	
Caso de uso relacionado	CU-11 Mostrar la foto e información del alumno
Descripción	Validar que el docente revisar los reportes creados por el o por otros docentes.
Cantidad de ejecuciones	100
Realizado por:	Alejandra De la cruz De la cruz

Supuestos y Condiciones Previas	• El docente ha iniciado sesión en el sistema. • Existen ETS dados de alta. • El docente eligió un ETS. • Hay información del ETS seleccionado. • Hay alumnos inscritos al ETS.
Datos de Prueba	• Reporte.
Pasos a Ejecutar	1. Iniciar sesión como docente. 2. Seleccionar la opción “ETS” desde el menú. 3. Elegir un ETS disponible en la lista. 4. Dar clic en el ETS. 5. Revisa los datos mostrados. 6. El docente elige la opción “Ir a la lista de alumnos” 7. El docente revisa la lista de alumnos inscritos al ETS 8. El docente elige el reporte de un alumno.
Resultado Esperado	El docente revisa el reporte.
Errores Detectados	• La el dato de la hora y fecha de la creación del reporte salía mal por un error de tipos de datos.
Resultado Real y Condiciones Posteriores	• Antes de mostrar la hora y la fecha son convertidos a un tipo de dato String. • Se verificó el correcto funcionamiento tanto en base de datos como en la interfaz.
Estado	Aprobado - la funcionalidad opera correctamente.

6.5.9. Caso de prueba CP12 - Consultar alumno mediante código QR de la credencial

CASO DE PRUEBA - CP12 Consultar alumno mediante código QR de la credencial

Caso de uso relacionado	CU-12 Consultar alumno mediante código QR de la credencial
Descripción	Validar la funcionalidad de obtención de la credencial del alumno mediante técnicas de scraping con Selenium, asegurando su compatibilidad tanto en entornos locales como en entornos de despliegue en la nube (Azure).
Cantidad de ejecuciones	100
Realizado por:	Alejandra De la cruz De la cruz
Supuestos y Condiciones Previas	• El personal de seguridad debe haber iniciado sesión en el sistema. • El dispositivo debe contar con cámara funcional. • La URL de la credencial debe ser accesible y válida.
Datos de Prueba	• URL de la credencial del alumno.
Pasos a Ejecutar	1. Iniciar sesión en la aplicación como personal de seguridad. 2. Activar los permisos de la cámara del dispositivo para escanear el código QR. 3. Escanear el código QR correspondiente a la credencial del alumno. 4. Esperar la respuesta del sistema con la credencial renderizada. 5. Verificar que se muestran correctamente los datos personales del alumno junto con la imagen de la credencial.
Resultado Esperado	La credencial del alumno debe renderizarse correctamente y generar una imagen funcional, tanto en entorno local como en el entorno de producción.

Errores Detectados	• Error 1: Selenium y Playwright funcionaban correctamente en entorno local, pero fallaban en Azure. • Causa: Incompatibilidades con dependencias nativas y configuración del entorno en la nube. • Solución: Implementación de una solución alternativa basada en la generación de PDF de la credencial y su conversión a imagen.
Resultado Real y Condiciones Posteriores	• En entorno local: Selenium y Playwright funcionaron correctamente. • En Azure: Selenium y Playwright presentaron fallos debido a incompatibilidades con dependencias nativas. • La solución alternativa basada en PDF generó una imagen funcional y compatible.
Estado	Fallido con Selenium/Playwright en Azure. Aprobado con solución alternativa basada en PDF.

6.5.10. Caso de prueba CP13 - Consultar alumnos mediante búsqueda por boleta

CASO DE PRUEBA - CP13 Consultar alumnos mediante búsqueda por boleta	
Caso de uso relacionado	CU-13 Consultar alumnos mediante búsqueda por boleta
Descripción	Validar la funcionalidad de consultar alumnos inscritos a un ETS, mediante búsqueda por número de boleta.
Cantidad de ejecuciones	100
Realizado por:	José Alfredo Jiménez Rodríguez
Supuestos y Condiciones Previas	• El personal de seguridad ha iniciado sesión correctamente. • Existen alumnos inscritos a un ETS para la fecha consultada.
Datos de Prueba	• Las boletas de los alumnos deberán ser válidas y deben estar inscritos en al menos un ETS. • La fecha de consulta del ETS debe estar activa.

Pasos a Ejecutar	1. Iniciar sesión como personal de seguridad. 2. Seleccionar la opción "Consultar Alumnos" desde el menú. 3. Ingresar el número de boleta en el buscador. 4. Ejecutar la búsqueda y observar la lista de alumnos inscritos a ETS.
Resultado Esperado	Los alumnos inscritos a un ETS en la fecha consultada y con la boleta ingresada deben mostrarse correctamente en la pantalla.
Errores Detectados	• Inicialmente, no se mostraban alumnos al seleccionar "Consultar Alumnos" desde el menú. • La lista de alumnos inscritos a ETS ese día aparecía vacía. • Se verificó que la lógica de implementación no estaba retornando datos correctamente.
Resultado Real y Condiciones Posteriores	• Tras corrección, el sistema mostró correctamente la lista de alumnos inscritos al ETS filtrada por boleta. • Se validó que la consulta a la base de datos y el filtrado funcionaron correctamente.
Estado	Aprobado - El error fue corregido y la funcionalidad se encuentra funcionando.

6.5.11. Caso de prueba CP14 - Consultar alumnos mediante búsqueda por nombre

CASO DE PRUEBA - CP14 Consultar alumnos mediante búsqueda por nombre	
Caso de uso relacionado	CU-14 Consultar alumnos mediante búsqueda por nombre
Descripción	Validar la funcionalidad de consultar alumnos inscritos a un ETS, mediante búsqueda por nombre.
Cantidad de ejecuciones	100
Realizado por:	José Alfredo Jiménez Rodríguez

Supuestos y Condiciones Previas	• El personal de seguridad ha iniciado sesión correctamente. • Existen alumnos inscritos a un ETS para la fecha consultada.
Datos de Prueba	• El nombre de los alumnos deberán ser válidas y deben estar inscritos en al menos un ETS. • La fecha de consulta del ETS debe estar activa.
Pasos a Ejecutar	1. Iniciar sesión como personal de seguridad. 2. Seleccionar la opción "Consultar Alumnos" desde el menú. 3. Ingresar el nombre del alumno en el buscador. 4. Ejecutar la búsqueda y observar la lista de alumnos inscritos a ETS.
Resultado Esperado	Los alumnos inscritos a un ETS en la fecha consultada y con el nombre ingresado deben mostrarse correctamente en la pantalla.
Errores Detectados	• Inicialmente, no se mostraban alumnos al seleccionar "Consultar Alumnos" desde el menú. • La lista de alumnos inscritos a ETS ese día aparecía vacía. • Se verificó que la lógica de implementación no estaba retornando datos correctamente.
Resultado Real y Condiciones Posteriores	• Tras corrección, el sistema mostró correctamente la lista de alumnos inscritos al ETS filtrada por nombre. • Se validó que la consulta a la base de datos y el filtrado funcionaron correctamente.
Estado	Aprobado - El error fue corregido y la funcionalidad se encuentra funcionando.

6.5.12. Caso de prueba CP15 - Registrar ingreso a la instalación

CASO DE PRUEBA - CP15 Registrar ingreso a la instalación	
Caso de uso relacionado	CU-15 Registrar ingreso a la instalación
Descripción	Verificar que el sistema registre el ingreso de los alumnos únicamente si tienen un ETS programado para la fecha actual, evitando accesos no autorizados por parte de alumnos sin programación válida.
Cantidad de ejecuciones	100
Realizado por:	José Alfredo Jiménez Rodríguez
Supuestos y Condiciones Previas	• El personal de seguridad ha iniciado sesión correctamente. • El sistema cuenta con conexión activa a la base de datos de inscripciones a ETS. • El alumno presenta una credencial con código QR válido. • La fecha del sistema coincide con la fecha del ETS programado.
Datos de Prueba	• Alumno con ETS programado para la fecha actual. • Alumno con ETS programado para una fecha diferente. • Alumno sin ETS asignado.
Pasos a Ejecutar	1. Iniciar sesión como personal de seguridad. 2. Escanear el código QR de la credencial del alumno. 3. El sistema consulta si el alumno tiene un ETS asignado. 4. El sistema valida si la fecha del ETS coincide con la fecha actual. 5. Si ambas validaciones son exitosas, se registra el ingreso. 6. Si alguna validación falla, se muestra un mensaje de advertencia y se deniega el ingreso.

Resultado Esperado	El sistema debe permitir el registro de ingreso únicamente si el alumno tiene un ETS asignado para la fecha actual. En caso contrario, debe denegar el acceso y notificar al personal.
Errores Detectados	• Error 1: El sistema permitía registrar el ingreso de cualquier alumno, incluso sin ETS asignado. • Causa: No se realizaba validación de inscripción al ETS al escanear el código QR. • Solución: Se añadió validación de inscripción activa en el ETS correspondiente. • Error 2: El sistema registraba el ingreso de alumnos con ETS programado en una fecha diferente a la actual. • Causa: Se validaba la inscripción, pero no se comprobaba la coincidencia de la fecha. • Solución: Se implementó una validación adicional que compara la fecha del ETS con la fecha actual del sistema.
Resultado Real y Condiciones Posteriores	• Se realizaron pruebas con distintos escenarios: alumnos con y sin ETS asignado, y fechas coincidentes o no coincidentes. • El sistema bloqueó correctamente los accesos no válidos. • Solo se permitió el ingreso de alumnos con ETS programado para el día actual.
Estado	Aprobado , tras implementar las correcciones y realizar validaciones exitosas.

6.5.13. Caso de prueba CP042 - Asignar docente de reemplazo

CASO DE PRUEBA - CP42 Asignar docente de reemplazo	
Caso de uso relacionado	CU-42 Asignar docente de reemplazo
Descripción	Validar que el jefe de departamento o el presidente de academia pueda visualizar las solicitudes de reemplazo y asignar a un docente, cambiando el estatus de la solicitud de "Pendiente. ^a Aceptada. ^o Rechazada".

Cantidad de ejecuciones	100
Realizado por:	Daniel Martín Huertas Ramírez
Supuestos y Condiciones Previas	• El jefe de departamento o presidente de academia ha iniciado sesión correctamente. • Existen solicitudes de reemplazo previamente enviadas por docentes. • Se dispone de una lista de docentes disponibles para asignación.
Datos de Prueba	• Solicitud de reemplazo con estatus “Pendiente”. • Docente disponible para asignación.
Pasos a Ejecutar	1. Iniciar sesión como jefe de departamento o presidente de academia. 2. Seleccionar la opción “Solicitudes de reemplazo” desde el menú. 3. Visualizar las solicitudes pendientes. 4. Seleccionar una solicitud. 5. Asignar a un docente como reemplazo. 6. Enviar la solicitud con la decisión correspondiente (Aceptar o Rechazar).
Resultado Esperado	La solicitud debe actualizarse correctamente con el docente asignado y su estatus debe cambiar a “Aceptada” o “Rechazada”, según la decisión.
Errores Detectados	• Las solicitudes de reemplazo no se cargaban en la vista de jefe de departamento/presidente de academia. • El estatus “Pendiente” no se actualizaba tras aceptar o rechazar una solicitud.

Resultado Real y Condiciones Posteriores	• Se ajustó la lógica de obtención de solicitudes desde el backend, lo que permitió cargarlas correctamente en la interfaz. • Se corrigió la lógica de cambio de estado, permitiendo que el estatus pase de “Pendiente” a “Aceptada” o “Rechazada” correctamente. • Se verificó la actualización de los datos tanto en la base como en la interfaz.
Estado	Aprobado - Tras correcciones, la funcionalidad fue validada exitosamente.

6.6. Pruebas al modelo de reconocimiento facial

Debido a la naturaleza del proyecto, el uso de un modelo de detección de rostros y generación de embeddings pre-entrenado fue necesario, por lo tanto, se hizo uso de la biblioteca DeepFace la cual contiene una colección de modelos pre-entrenados tanto para la detección de rostros como para la generación de embeddings faciales.

Es por esta razón que, antes de integrar dicha funcionalidad a nuestro sistema fue necesario realizar pruebas sobre las alternativas de modelos de código abierto disponibles dentro de este paquete y, al mismo tiempo, experimentar el cómo diferentes configuraciones del proceso de reconocimiento facial, incluyendo el modelo de embeddings, el detector de rostros, la métrica de distancia usada para determinar la similitud y la alineación de las imágenes, influyen en la precisión del sistema.

6.6.1. Configuración experimental

Para llevar a cabo las pruebas, se definieron las siguientes reglas de configuración, abarcando un total de 8 combinaciones experimentales:

- **Alineación de Rostros (align):** Se evaluaron dos escenarios:
 - True: Las imágenes fueron alineadas antes de la extracción de características, lo que implica un preprocesamiento para normalizar la pose y el tamaño del rostro.
 - False: Las imágenes no fueron alineadas, procesando los rostros tal como se detectaron.
- **Modelos de Incrustación (models):** Se utilizaron dos modelos de redes neuronales pre-entrenadas para generar las incrustaciones (embeddings) de los rostros:
 - ArcFace: Un modelo conocido por su alta discriminación en tareas de reconocimiento facial.
 - Facenet512: Otra arquitectura popular para la generación de incrustaciones faciales.

- **Detectores de Rostros (detectors):** Se emplearon dos algoritmos para la detección de rostros en las imágenes:
 - `ssd` (Single Shot Detector): Un detector rápido y eficiente.
 - `fastmtcnn` (Fast Multi-task Cascaded Convolutional Networks): Una versión optimizada de MT-CNN, conocido por su precisión en la detección de puntos de referencia faciales.
- **Métricas de Distancia (distance_metrics):** Para comparar las incrustaciones de los rostros y determinar si pertenecen a la misma persona, se utilizaron dos métricas de distancia:
 - `euclidean_l2`: La distancia euclidiana L2 normalizada.
 - `cosine`: La similitud del coseno.

Cada combinación de estas configuraciones fue sometida a prueba, generando un archivo de resultados individual para su posterior análisis.

6.6.2. Preparación del Conjunto de Datos LFW

El conjunto de datos LFW se cargó específicamente su subconjunto de prueba (`subset='test'`), con imágenes en color y un factor de redimensionamiento de 2. Para cada par de imágenes en el dataset LFW (un total de 1000 instancias para las pruebas realizadas), se guardaron las imágenes individualmente en el sistema de archivos (`lfw/test/`) para su procesamiento. Esto aseguró que cada experimento se ejecutara sobre los mismos pares de imágenes, garantizando la reproducibilidad.

6.6.3. Metodología de Evaluación

Para cada combinación experimental, el proceso de reconocimiento facial fue ejecutado para calcular la distancia entre los pares de rostros en el conjunto de datos LFW. La función `DeepFace.verify` se utilizó para este propósito, registrando la distancia calculada para cada par. Es importante destacar que la detección de rostros se forzó a `False` (`enforce_detection=False`) para evitar errores en casos donde el rostro no pudiera ser detectado con certeza.

Una vez calculadas las distancias para todas las 1000 instancias de pares de imágenes por cada combinación, los resultados se almacenaron en archivos CSV (ej. `outputs/test/ArcFace_fastmtcnn_euclidean_l2_aligned.csv`). Cada archivo contenía las etiquetas reales (`actuals`) indicando si los pares correspondían a la misma persona (`verdadero/1`) o a personas diferentes (`falso/0`), junto con las distancias calculadas (`distances`).

Para determinar la precisión de cada configuración, se implementó un procedimiento para encontrar el umbral de distancia óptimo que maximiza la precisión en la clasificación. Este procedimiento implicó:

1. Calcular la media de las distancias para los pares positivos (misma persona) y negativos (diferentes personas).
2. Iterar a través de todas las distancias ordenadas.
3. Para cada distancia candidata a umbral, se clasificaron los pares como "misma persona" si su distancia era menor que el umbral y "diferente persona" en caso contrario.

4. Se calculó la precisión (`accuracy_score`) comparando estas predicciones con las etiquetas reales.
5. El umbral que resultó en la mayor precisión se seleccionó como el umbral óptimo para esa configuración.
6. La precisión máxima obtenida con este umbral se registró como el rendimiento de la configuración.

Los resultados de precisión para cada combinación de modelo, detector y métrica de distancia, tanto con alineación como sin ella, se tabularon en tablas pivotantes y se guardaron en archivos CSV (ej. `results/pivot_euclidean_l2_with_alignment_True.csv`).

6.6.4. Curvas ROC y Análisis Visual

Adicionalmente, para proporcionar una comprensión más profunda del rendimiento de cada configuración, se generaron Curvas de Característica Operativa del Receptor (ROC). Estas curvas visualizan la relación entre la Tasa de Verdaderos Positivos (TPR) y la Tasa de Falsos Positivos (FPR) en diferentes umbrales de clasificación. Para generar estas curvas:

1. Las distancias calculadas se normalizaron dividiéndolas por el valor máximo de distancia en el conjunto de datos.
2. Las etiquetas reales (`actuals`) se normalizaron invirtiendo los valores (0 para pares de la misma persona y 1 para pares diferentes). Esto se hizo para que las distancias más pequeñas (mayor similitud) correspondieran a una mayor probabilidad de ser del mismo sujeto, lo que es coherente con la interpretación de las curvas ROC donde un valor de predicción más alto indica una mayor confianza en la clase positiva (en este caso, personas diferentes, o una distancia mayor).
3. Se utilizó la función `metrics.roc_curve` de `sklearn` para calcular la FPR y la TPR.
4. El Área bajo la Curva (AUC) también se calculó utilizando `metrics.roc_auc_score`, proporcionando una métrica agregada del rendimiento del clasificador.

Las curvas ROC se graficaron para cada combinación, incluyendo la precisión (`acc`) y el AUC en la etiqueta de la curva, lo que facilita la comparación visual del rendimiento entre las diferentes configuraciones. Se generaron gráficos que muestran todas las configuraciones en un mismo plano para facilitar la comparación global, con un enfoque en un rango específico de FPR (0 a 0.3) para una mejor legibilidad en la región de baja tasa de falsos positivos.

Los resultados de esta configuración de pruebas se mostraron en el capítulo de Resultados.

6.7. Pruebas al modelo de reconocimiento facial sobre conjuntos de datos personales

Una vez que se seleccionó la configuración ideal, después de validarlo con un subconjunto del dataset LFW con 1000 entidades, procedimos a evaluarlo con algunas imágenes de rostros de nuestra comunidad

estudiantil.

La configuración del sistema de reconocimiento facial y la metodología de evaluación se establecieron con los siguientes parámetros clave:

6.7.1. Directorio de Imágenes

Todas las imágenes utilizadas para el registro y la evaluación del sistema se organizan en un directorio base:

- `BASE_IMAGES_DIR = 'imagenes_alumnos'`

Dentro de este directorio, se espera una subcarpeta por cada individuo (ej., 2022630082), conteniendo dos tipos de imágenes:

- `*_new.png`: Imagen utilizada para el registro del individuo (extracción del embedding de referencia).
- `*_old.png`: Imagen utilizada para la evaluación (simula una nueva captura a ser identificada).

6.7.2. Modelo de Reconocimiento Facial

Para la extracción de los **embeddings faciales**, se seleccionó uno de los modelos pre-entrenados más robustos disponibles en DeepFace:

- **Nombre del Modelo (MODEL_NAME):** Facenet

Facenet es conocido por su alta precisión en la tarea de verificación facial al generar representaciones vectoriales densas (embeddings) de los rostros.

6.7.3. Backend del Detector Facial

Para la detección y alineación de los rostros dentro de las imágenes antes de la extracción del embedding, se utilizó el siguiente detector:

- **Detector Backend (DETECTOR_BACKEND):** `fastmtcnn`

`fastmtcnn` es una opción eficiente que busca un equilibrio entre velocidad y precisión en la detección de rostros.

6.7.4. Métrica de Distancia y Umbral

La comparación entre los embeddings se realiza utilizando una métrica de distancia específica, y se aplica un umbral para determinar si dos rostros pertenecen a la misma persona:

- **Métrica de Distancia (DISTANCE_METRIC):** `cosine`

La distancia coseno (D_c) se calcula como $1 - \text{similitud_coseno}$, donde la similitud coseno es el coseno del ángulo entre dos vectores. Valores cercanos a 0 indican alta similitud (distancia pequeña).

- **Umbral de Distancia (DISTANCE_THRESHOLD): 0.4**

Un valor de distancia coseno **menor o igual** a este umbral indica que los dos embeddings corresponden al mismo individuo. Este umbral es crucial para la decisión de identificación.

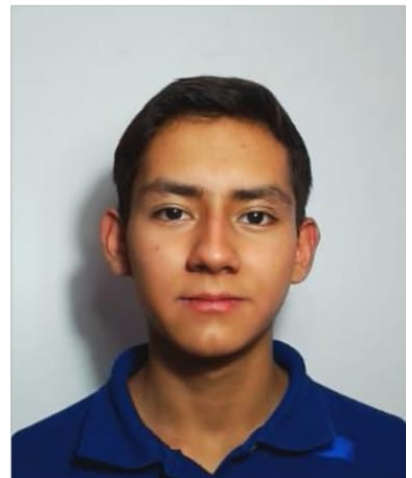
6.8. Propósito del Experimento

El propósito fundamental de este experimento es:

1. **Evaluar la Precisión de Identificación:** Determinar cuán exitosamente el sistema puede identificar a los alumnos registrados utilizando imágenes diferentes a las de registro (*_old.png). Esto implica medir la capacidad del sistema para clasificar correctamente cada imagen de prueba con su identidad verdadera.
2. **Medir el Rendimiento de Clasificación:** Utilizar métricas estándar como **precisión**, **exhaustividad (recall)** y **puntuación F1** para cada clase (cada alumno registrado), así como la **exactitud (accuracy)** general del sistema.
3. **Comprender el Comportamiento ante "Desconocidos":** Aunque en este conjunto de datos inicial no se incluyen explícitamente imágenes de "desconocidos" para la evaluación, el sistema está diseñado para clasificarlos como tal si no encuentran un match por debajo del umbral. La matriz de confusión y el reporte de clasificación permitirán observar cómo se manejarían estos casos, incluso si el soporte actual para esta clase es nulo. Este es un punto clave para futuras expansiones del conjunto de datos de prueba.
4. **Validar la Configuración de Umbral:** Observar el impacto del DISTANCE_THRESHOLD en las decisiones de identificación. Un umbral bien ajustado es vital para minimizar tanto los **falsos positivos** (identificar erróneamente a alguien) como los **falsos negativos** (no reconocer a alguien registrado).
5. **Generar una Matriz de Confusión:** Visualizar gráficamente el rendimiento del clasificador para cada clase, mostrando las identificaciones correctas e incorrectas, lo cual es fundamental para el análisis de errores.



(a) Imagen registrada en el sistema (ej., Imagen de Registro)



(b) Imagen de pruebaa (ej., Imagen de Prueba)

Figura 6.1: Conjunto de imágenes mostrando un ejemplo de imagen de registro y su correspondiente imagen de prueba.

En este capítulo se exponen los resultados obtenidos mediante las encuestas realizadas a los usuarios (Alumnos, docentes y personal de seguridad), además de los resultados de las pruebas de la red neuronal (el sistema de reconocimiento facial).

7.1. Alumnos

De la sección de los alumnos, se analizaron 41 resultados sobre el uso de la aplicación móvil, tanto para probar el reconocimiento, como para ver el listado de los ETS existentes. Los resultados, en su mayor parte, son comentarios positivos, sin embargo, a lo largo del desarrollo de las pruebas salieron a relucir varios puntos de mejora sobre la aplicación, en su mayor parte de visualización.

7.1.1. Satisfacción del usuario con la aplicación

Como primer punto, se le preguntó al usuario sobre su satisfacción de acuerdo al reconocimiento facial, sobre problemas acerca del mismo y qué tan preciso era este proceso, y se obtuvieron los siguientes resultados:

- Tiempo que tardó el proceso de reconocimiento facial para identificarlo (figura 7.1)

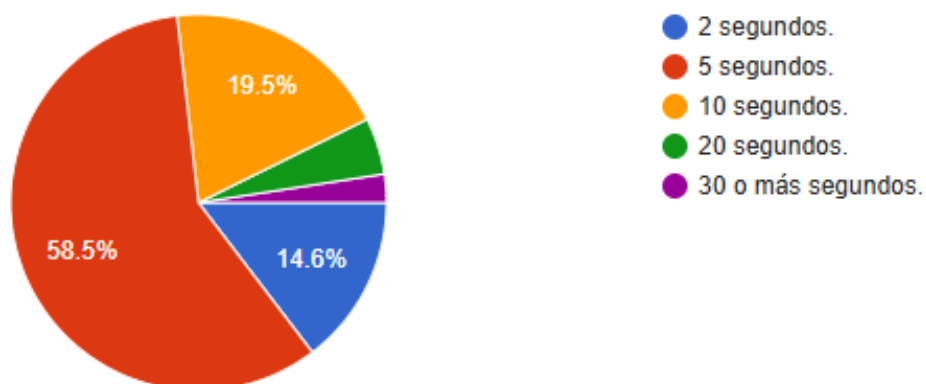


Figura 7.1: Tiempo del proceso del reconocimiento facial.

- Frecuencia con el que el reconocimiento facial lo identificó correctamente (figura 7.2)

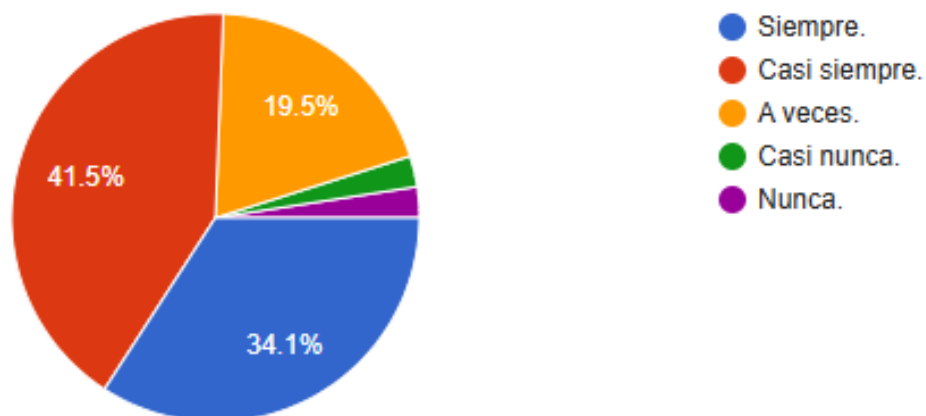


Figura 7.2: Frecuencia de resultados correctos del reconocimiento facial.

- Seguridad del usuario con la precisión del reconocimiento facial (figura 7.3)

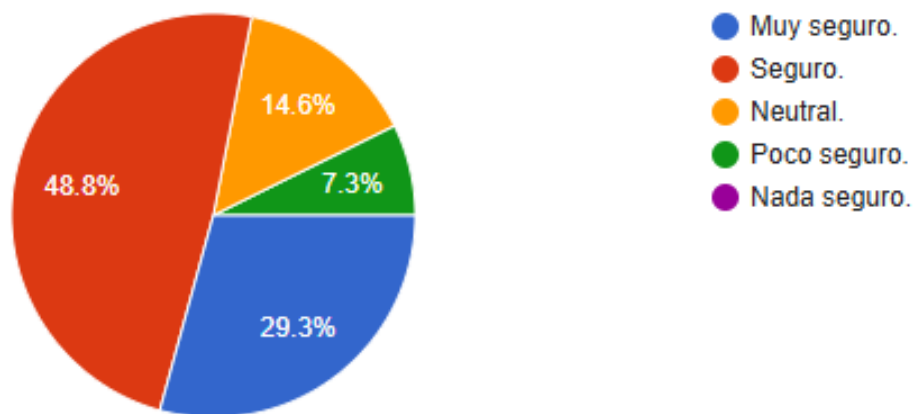


Figura 7.3: Seguridad del alumno con la precisión del reconocimiento facial.

- Intentos necesarios para para que el reconocimiento facial lo identificara correctamente (figura 7.4)

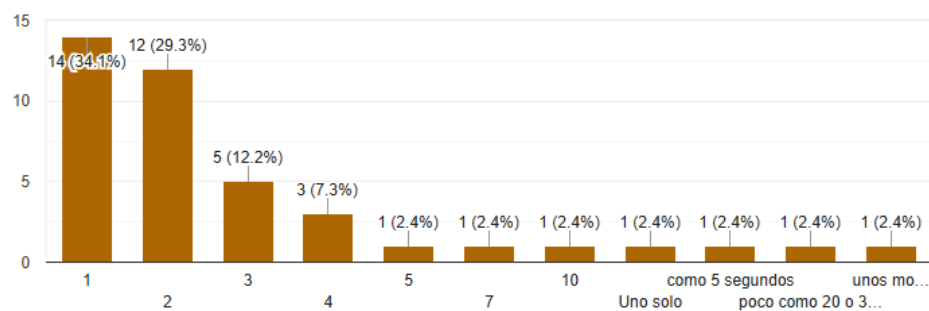


Figura 7.4: Intentos necesarios para que el reconocimiento facial lo identificara correctamente.

- Que tan rápido y fluido le pareció al alumno el proceso del reconocimiento facial (figura 7.5)

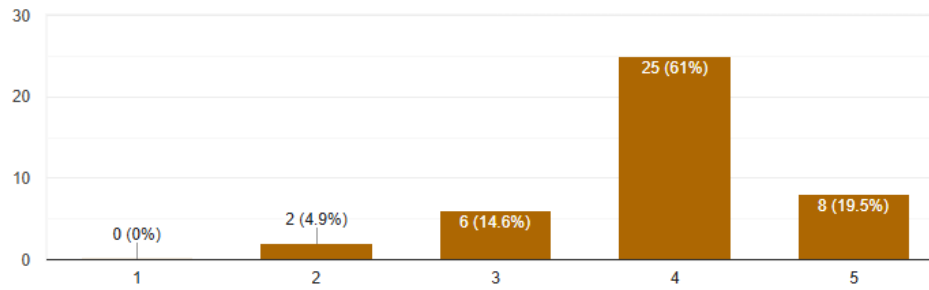


Figura 7.5: Rapidez y fluidez del proceso del reconocimiento facial.

- Que tan útil le resultó la información mostrada en los detalles de los ETS (figura 7.6)

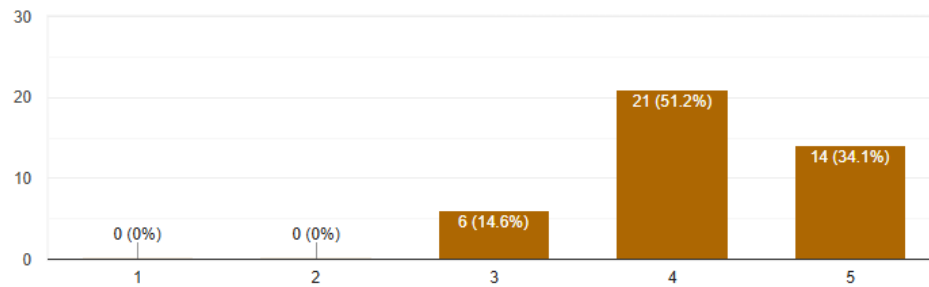


Figura 7.6: Utilidad de la información mostrada en los detalles del ETS.

- Qué tan útil le resultó la sección de instrucciones para ingresar a un ETS (figura 7.7)

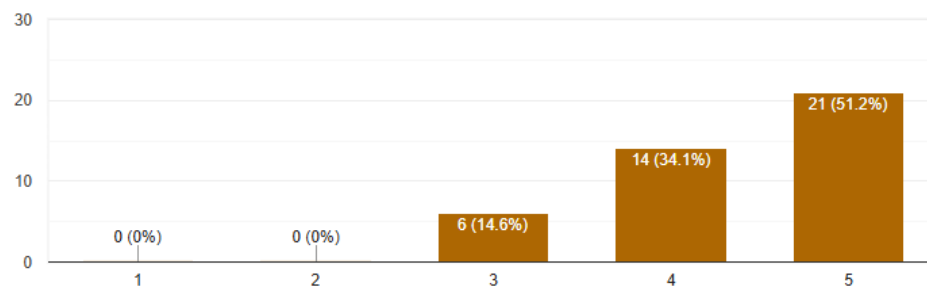


Figura 7.7: Utilidad de las instrucciones mostradas para el ingreso de los ETS.

- Comodidad del usuario al navegar por la app móvil (figura 7.8)

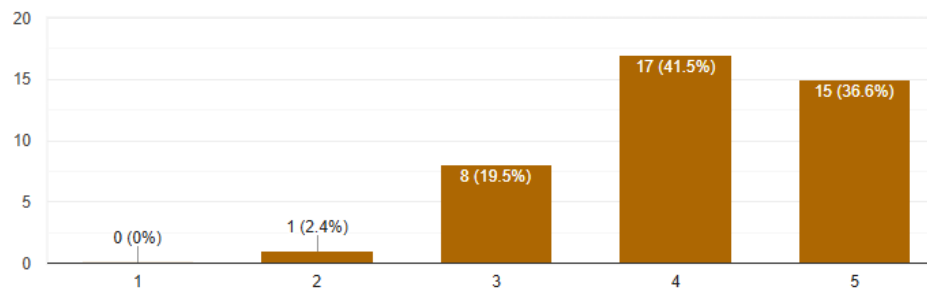


Figura 7.8: Comodidad del usuario al navegar por la app móvil.

- Que tan acertados le parecieron los colores de la app al alumno (figura 7.9)

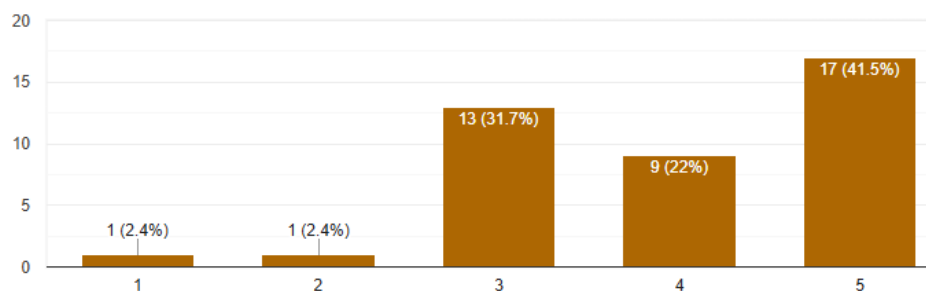


Figura 7.9: Correcta paleta de colores.

Por último punto, se le preguntó al alumno cuánto tiempo le tomó adaptarse a la app, es decir, en cuánto tiempo ya la podía usar sin ningún problema. De esta pregunta, la mayoría de los alumnos contestaron que les tomó muy poco tiempo el adaptarse a la aplicación.

7.1.2. Puntos de mejora

Por otra parte, respecto a los puntos de mejora que podría tener la aplicación. En este apartado, la mayoría de los alumnos contestaron que les hubiese gustado incluir un pequeño mapa en el que pudieran ver en qué parte de la escuela les tocaba realizar el ETS. Otro pequeño conjunto de alumnos mencionaron que les hubiese gustado que la aplicación tuviese una alarma un día antes del ETS para que les recordara sobre su ETS y otros puntos como lo es un filtro por fecha del listado de los ETS, una guía de uso dentro de la aplicación, guías para los ETS, asesorías, quejas y sugerencias, entre otras cosas.

Así también, dentro de las dificultades que se le presentó al alumno al momento de usar la aplicación se encontraron puntos como la velocidad del internet, que el reconocimiento facial no siempre los reconocía, que al irse a la parte de los ETS se les cerraba la aplicación, la mejora de la paleta de colores o que reconocía únicamente al alumno si tomaba la foto en horizontal. Muchos de estos comentarios y retroalimentaciones de los alumnos se tomaron en cuenta y ya fueron solucionados, es por esto que la satisfacción del usuario aumentó considerablemente en pruebas realizadas tiempo después.

7.2. Docentes

En esta sección, se presenta el análisis exhaustivo de los resultados obtenidos de las 15 encuestas realizadas a docentes. Estas encuestas tuvieron como propósito principal evaluar tanto el funcionamiento como la experiencia de uso de la aplicación móvil en funcionalidades clave. La recolección de esta retroalimentación es crucial para comprender a fondo la percepción de los usuarios sobre aspectos críticos del sistema y su desempeño en un entorno real.

Las preguntas de las encuestas se diseñaron para enfocarse en varios puntos específicos: la efectividad del reconocimiento facial, la facilidad y precisión para consultar el listado de ETS y la eficiencia en la comparación de credenciales mediante código QR.

Los resultados preliminares de estas encuestas, en sintonía con la retroalimentación de otros grupos de usuarios, arrojan una mayoría de comentarios positivos respecto a la operatividad general de las funcionalidades. Sin embargo, el proceso de encuestado también permitió identificar puntos específicos de mejora y áreas de oportunidad significativas. Estas incluyen tanto aspectos relacionados con la usabilidad y la experiencia del usuario, como la detección de errores de funcionalidad específicos que se detallan en la sección de pruebas.

A continuación, se presenta el análisis completo de las encuestas aplicadas a los docentes, desglosando los hallazgos y las implicaciones para futuras mejoras del sistema.

7.2.1. Resultados de la encuesta: reconocimiento facial en la sección del docente

El primer aspecto evaluado en la encuesta fue el reconocimiento facial. Se preguntó a los docentes sobre su satisfacción general con esta funcionalidad, los problemas experimentados y la precisión del proceso. Los resultados se muestran a continuación:

- **"¿Cuánto tiempo tardó aproximadamente el proceso de reconocimiento facial para identificar a un alumno?":** Esta pregunta buscó determinar la percepción de los docentes sobre la eficiencia de este proceso. Los resultados se visualizan en la Figura 7.10:

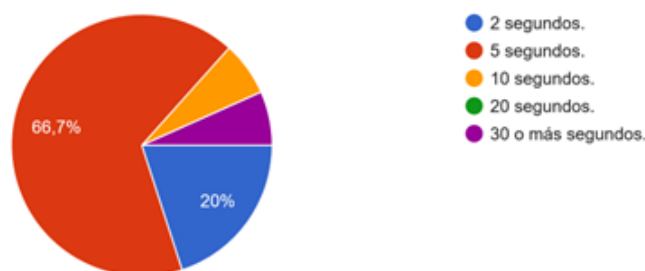


Figura 7.10: Tiempo aproximado del proceso de reconocimiento facial para identificar a un alumno.

- **"¿Con qué frecuencia el reconocimiento facial identificó correctamente al alumno?":** Esta pregunta buscó evaluar la precisión percibida del sistema de reconocimiento facial por parte de los docentes. Los resultados obtenidos se presentan en la Figura 7.11:

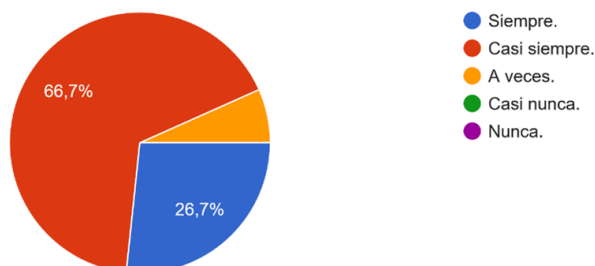


Figura 7.11: Frecuencia con la que el reconocimiento facial identificó correctamente al alumno, según la percepción de los docentes.

- **"¿Qué tan seguro se sintió con la precisión del reconocimiento facial para verificar la identidad del alumno?":** Esta pregunta evaluó el nivel de confianza de los docentes en la exactitud del reconocimiento facial como método de verificación de identidad. Los resultados se presentan en la Figura 7.12:

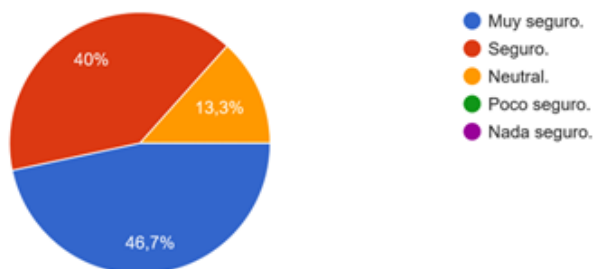


Figura 7.12: Nivel de seguridad de los docentes respecto a la precisión del reconocimiento facial.

- **"¿Cuántos intentos necesitó en promedio para que el reconocimiento facial identificara al alumno correctamente?":** Esta pregunta buscó cuantificar el esfuerzo requerido por parte de los docentes para lograr una identificación exitosa mediante el reconocimiento facial, evaluando así la fluidez del proceso. Los resultados obtenidos se presentan en la Figura 7.13:

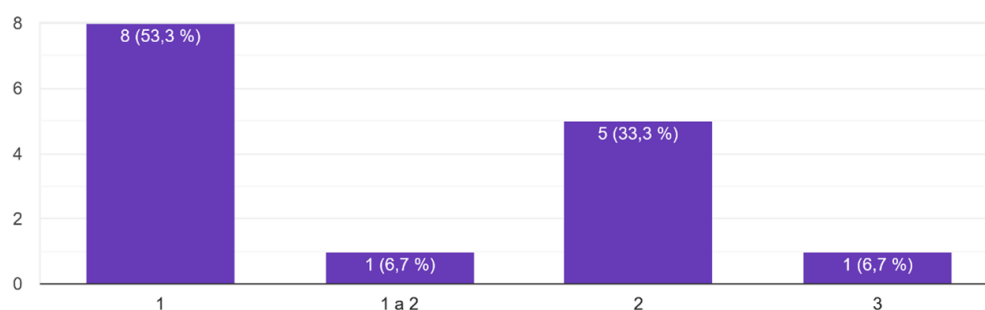


Figura 7.13: Número promedio de intentos requeridos para la identificación correcta por reconocimiento facial, según los docentes.

- **"¿Qué tan fluido y rápido le pareció el proceso general de verificación de identidad mediante reconocimiento facial?":** Esta pregunta tuvo como objetivo evaluar la percepción general de los docentes sobre la eficiencia y la experiencia de usuario del proceso completo de verificación de identidad con reconocimiento facial, combinando aspectos de rapidez y fluidez. La escala va de 1 a 5 donde, 5 es el máximo y 1 el mínimo, los resultados obtenidos se presentan en la Figura 7.14:

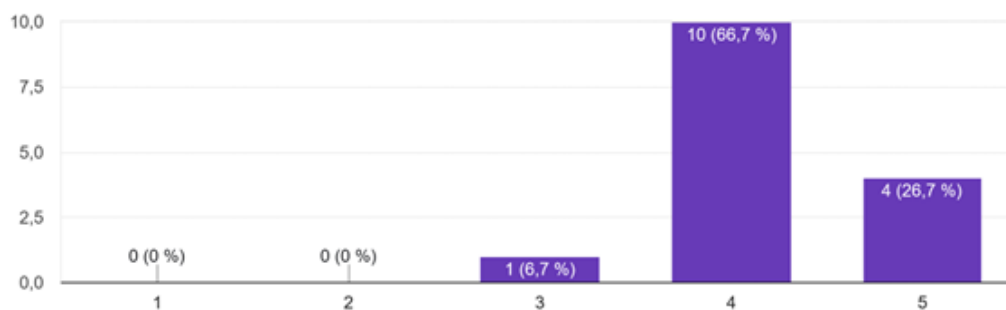


Figura 7.14: Percepción de los docentes sobre la fluidez y rapidez del proceso general de verificación de identidad por reconocimiento facial.

7.2.2. Comparación de Credenciales mediante Código QR

Esta subsección presenta los resultados de las encuestas relacionadas con la funcionalidad de escaneo y comparación de credenciales mediante códigos QR, una capacidad clave para la verificación de identidad. Se evaluó la facilidad de uso y la eficiencia percibida por los docentes al interactuar con esta característica de la aplicación móvil.

- **"¿Qué tan fácil fue para usted escanear el código QR de la credencial con la app móvil?":** Esta pregunta buscó determinar la percepción de los docentes sobre la facilidad de uso y la fluidez del proceso de escaneo de códigos QR para la verificación de credenciales. Los resultados obtenidos se presentan en la Figura 7.15:

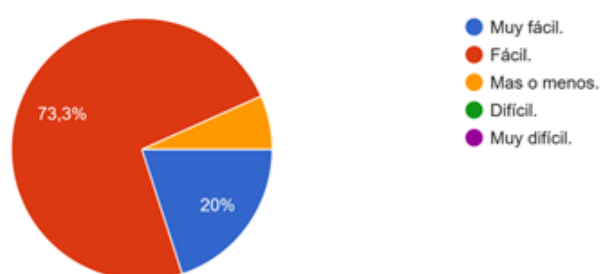


Figura 7.15: Percepción de los docentes sobre la facilidad para escanear el código QR de la credencial con la app móvil.

- **"¿Cuánto tiempo tardó aproximadamente la aplicación en leer el código QR de la credencial y mostrar la información?":** Esta pregunta se centró en la percepción de los docentes sobre la velocidad de respuesta del sistema desde el momento del escaneo del código QR hasta la visualización de la información. Los resultados se presentan en la Figura 7.16:

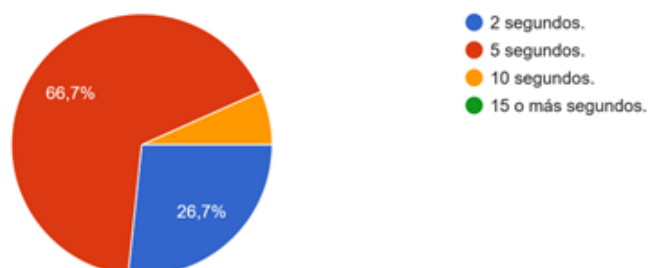


Figura 7.16: Tiempo percibido por los docentes para la lectura del código QR y visualización de la información de la credencial.

- **"¿Con qué frecuencia la app móvil leyó correctamente el código QR de la credencial?":** Esta pregunta buscó evaluar la fiabilidad del proceso de escaneo de códigos QR, donde la escala de respuesta va desde 1 (Nunca) hasta 5 (Siempre). Los resultados obtenidos se presentan en la Figura 7.17:

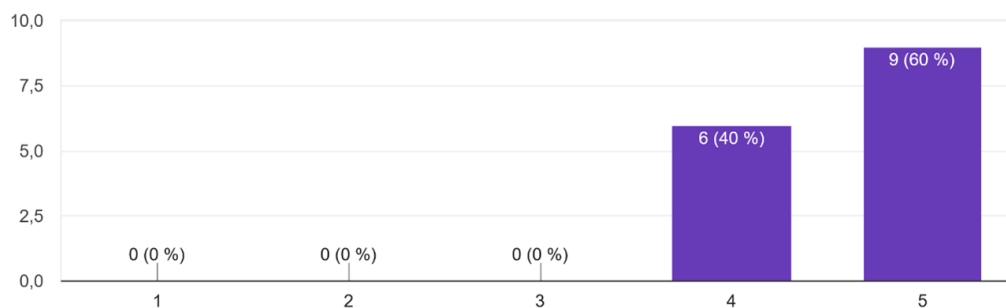


Figura 7.17: Frecuencia con la que la app móvil leyó correctamente el código QR de la credencial, según la percepción de los docentes.

- **"¿Qué tan clara y legible le parece la información mostrada después de escanear el código QR?":** Esta pregunta se enfocó en evaluar la percepción de los docentes sobre la calidad de la información presentada una vez que el código QR es escaneado, particularmente en términos de claridad y legibilidad. La escala va de 1 a 5 donde, 5 es el máximo y 1 el mínimo, los resultados obtenidos se presentan en la Figura 7.18:

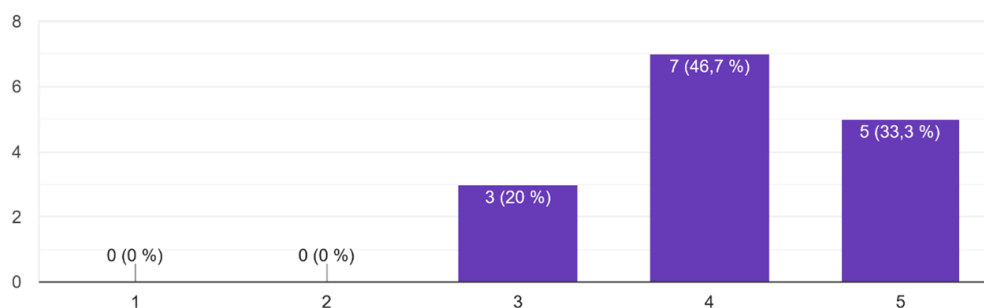


Figura 7.18: Percepción de los docentes sobre la claridad y legibilidad de la información mostrada tras escanear el código QR.

- **"¿Cuántos intentos necesitó en promedio para que la aplicación leyera el código QR correctamente?":** Esta pregunta buscó cuantificar la cantidad de intentos necesarios por parte de los docentes para que la aplicación móvil lograra leer el código QR de la credencial de forma exitosa, lo que indica la eficiencia del proceso de escaneo. Los resultados obtenidos se presentan en la Figura 7.19:

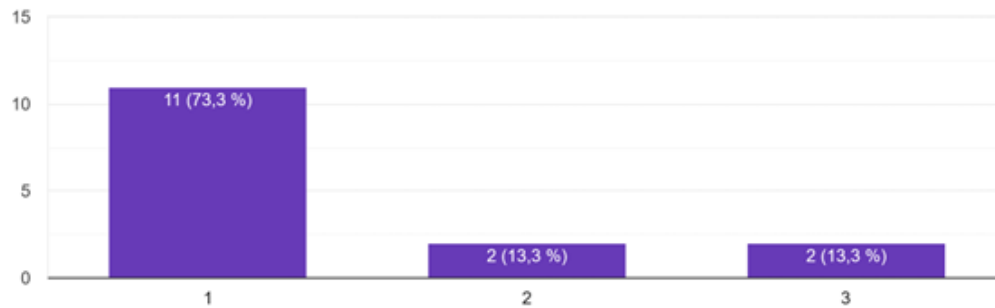


Figura 7.19: Número promedio de intentos para la lectura correcta del código QR por la aplicación, según los docentes.

7.2.3. Efectividad contra suplantación de identidad

Respecto a la percepción sobre la efectividad del sistema para prevenir la suplantación de identidad durante los Exámenes a Título de Suficiencia (ETS), los resultados revelan una confianza considerable por parte de los docentes. La Figura 7.20 ilustra que una vasta mayoría de los encuestados considera la aplicación altamente efectiva para este propósito.

Los resultados detallados son los siguientes:

- **(86.7%):** Percibieron una **máxima efectividad** del sistema. Este segmento de docentes destacó la gran utilidad del sistema, particularmente el componente de inteligencia artificial (IA) y la verificación mediante código QR, como un método robusto para evitar la suplantación de identidad en el contexto de los ETS.
- **(6.7%):** Consideraron que la efectividad era **moderada**.
- **(6.7%):** Expresaron una **baja percepción de efectividad**.

15. ¿Del 1 al 10 que tanto cree que la aplicación ayudaría para evitar la suplantación de la identidad de los alumnos durante la realización de los ETS?

15 respuestas

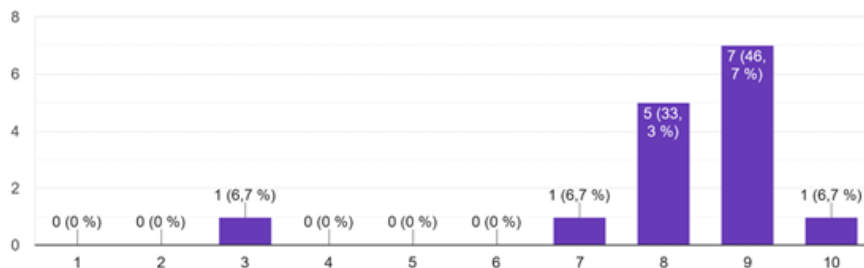


Figura 7.20: Evaluación de docente sobre la efectividad del sistema de inteligencia artificial contra suplantación de identidad.

En síntesis, la alta proporción de docentes que perciben una máxima efectividad subraya el potencial del sistema de inteligencia artificial en conjunto con el código QR para reforzar significativamente la seguridad y la integridad del proceso de evaluación, generando confianza en su capacidad para mitigar riesgos de suplantación de identidad.

7.3. Personal de seguridad

Se analizaron 15 respuestas del personal de seguridad sobre el uso de la aplicación móvil para la verificación de credenciales estudiantiles mediante códigos QR. Los resultados reflejan una percepción mayormente positiva, aunque se identificaron áreas clave de mejora, particularmente en la visualización de datos y la consistencia del escaneo.

A continuación se detalla el análisis completo:

7.3.1. Facilidad de uso del escaneo del código QR

- **Facilidad de escaneo:** El 60 % de los usuarios calificó la experiencia como **Muy fácil** y el 33.3 % como **Fácil** (ver gráfica 7.21). Solo un 6.7 % **Muy difícil** reportó problemas técnicos, donde la aplicación tardó 10 segundos en detectar el QR (de acuerdo con el comentario del usuario, por baja iluminación en su entorno).

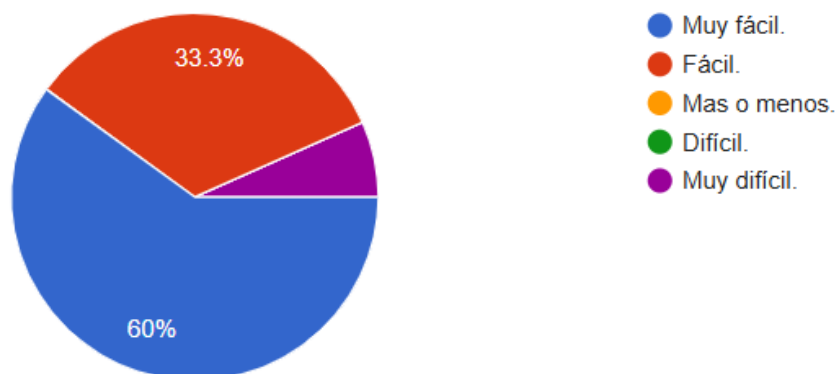


Figura 7.21: Distribución de respuestas sobre la facilidad de escaneo del código QR.

- **Tiempo de lectura del QR:** El 46.7 % de los usuarios (7/15) reportó un tiempo de escaneo de 5 segundos, mientras que el 33.3 % (5/15) observó 2 segundos y el 20 % (3/15) tardó 10 segundos (ver gráfica 7.22). Aunque el 80 % de los casos se resolvió en 5 segundos, los tiempos mayores (10s) sugieren la necesidad de optimizar el procesamiento en dispositivos antiguos o con baja iluminación, así como la conexión a internet.

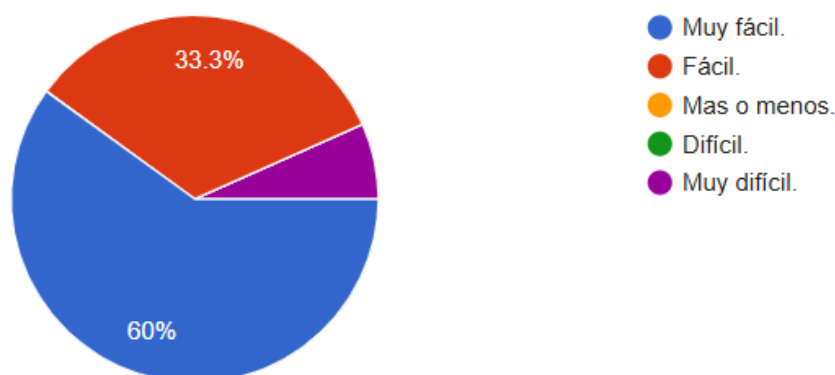


Figura 7.22: Distribución de respuestas sobre el tiempo de escaneo del código QR.

- **Precisión del escaneo QR:** La aplicación funciona correctamente en la mayoría de los casos, con 13 de 15 usuarios (87 %) calificando su precisión como buena o excelente (4-5 puntos de 5). Solo 2 usuarios reportaron dificultades ocasionales, como que **la cámara no reconocía el código inmediatamente**. Esto indica que, aunque el sistema es confiable, existen oportunidades para hacer el proceso aún más consistente para todos los usuarios.

7.3.2. Visualización de la información

Legibilidad de la información: La mayoría de los usuarios calificaron positivamente la claridad de la información (4-5 puntos en una escala de 5), lo que demuestra que el diseño general es efectivo. Sin embargo, varios usuarios reportaron problemas específicos con la visualización completa de los datos, comentando que: **los datos de la credencial del alumno no se muestran completos y la información aparece cortada**. Estos comentarios señalan la necesidad de ajustar el formato de visualización de la credencial del alumno para garantizar que toda la información sea visible.

7.3.3. Experiencia de usuario

- **Navegación:** El 100% de los usuarios (15 de 15) calificaron con 4 o 5 puntos (en una escala de 5) la facilidad de navegación en la aplicación. De ellos, 8 usuarios (53.3%) asignaron una calificación de 4 y 7 usuarios (46.7%) otorgaron una calificación de 5, lo que refleja una percepción positiva generalizada sobre la usabilidad del sistema (ver gráfica 7.23).

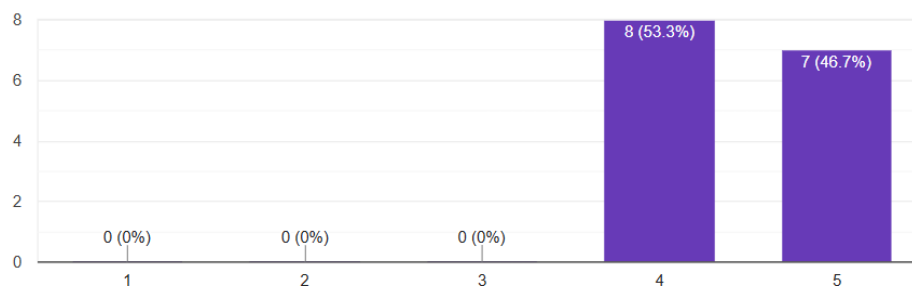


Figura 7.23: Evaluación de la experiencia de navegación en la aplicación.

- **Colores:** La evaluación del esquema de colores fue mayormente positiva, con puntuaciones entre 4 y 5. El 53.3% de los usuarios lo calificó con la máxima puntuación (5), mientras que el 46.7% lo valoró con un 4 (ver gráfica 7.24).

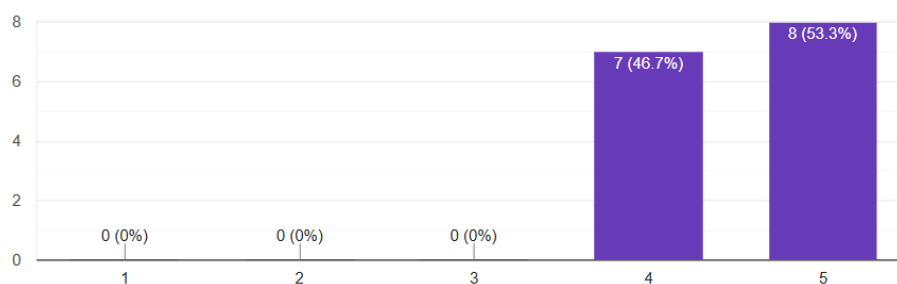


Figura 7.24: Evaluación del esquema de colores por los usuarios.

- **Comodidad para tomar asistencia:** La mayoría de los usuarios calificó la experiencia con una puntuación de 4 a 5 en una escala de 5, lo que indica una alta aceptación y satisfacción con el proceso de toma de asistencia mediante la aplicación. Esto sugiere que el sistema es intuitivo y facilita la labor del personal de seguridad al momento de registrar la asistencia de los alumnos.

7.3.4. Efectividad contra suplantación de identidad

La mayoría de los usuarios (93.4%) consideró que la aplicación sería muy efectiva para evitar la suplantación de identidad durante los ETS, otorgando calificaciones altas (8-10 puntos en una escala de 10). Los resultados detallados fueron (ver gráfica 7.25):

- **(68.7%):** Máxima efectividad, destacando la utilidad del sistema QR como un método para evitar la suplantación de identidad.
- **(26.7%):** Efectividad moderada, con sugerencias para implementar verificaciones adicionales.
- **(6.7%):** Baja percepción de efectividad.

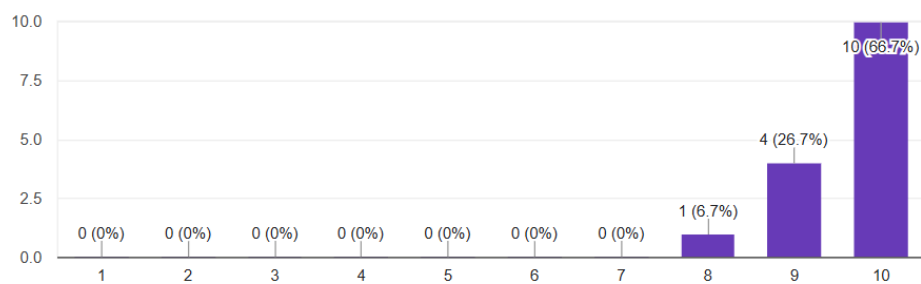


Figura 7.25: Evaluación de usuarios sobre la efectividad del sistema QR contra suplantación de identidad.

7.4. Modelos de reconocimiento facial

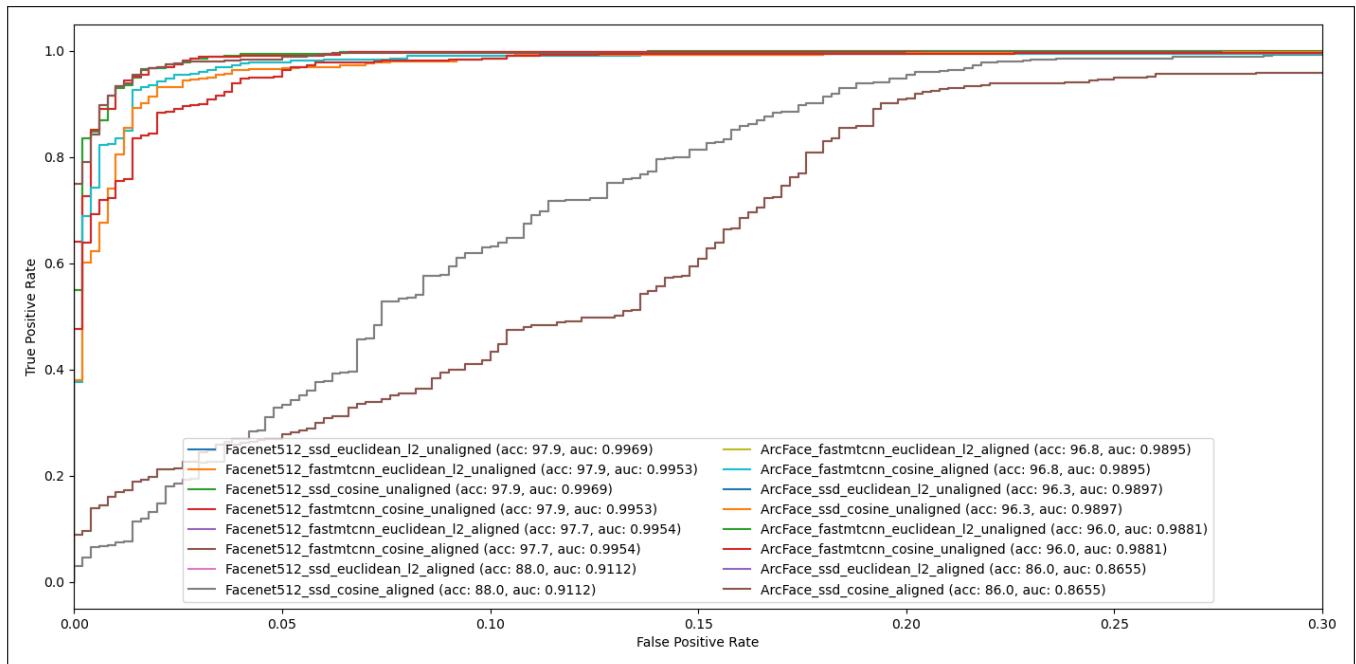


Figura 7.26: Curvas ROC para las distintas configuraciones experimentales en el dataset LFW (subconjunto 1000 muestras).

```
Iniciando evaluación del sistema de reconocimiento facial.  
Modelo: Facenet, Detector: fastmtcnn, Métrica: cosine, Umbral: 0.40  
Cargando registros desde 'imagenes alumnos'...  
2025-05-29 01:09:50.114416: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.  
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.  
- Registrado alumno 2022630082  
- Registrado alumno 2022630358  
- Registrado alumno 2022630370  
- Registrado alumno 2022630374  
- Registrado alumno 2022630384  
- Registrado alumno 2022630386  
- Registrado alumno 2022630393  
- Registrado alumno 2022630467  
- Registrado alumno 2022630738  
Embeddings cargados para 9 alumnos.
```

Figura 7.27: Captura de pantalla de la ejecución de la matriz de confusión.

```
Iniciando evaluación con imágenes antiguas desde 'imagenes_alumnos'...
--- Comparando 2022630082_old.png con registros:
- Distancia a 2022630082: 0.1567 (Umbral: 0.40)
- Distancia a 2022630358: 0.7353 (Umbral: 0.40)
- Distancia a 2022630370: 0.5668 (Umbral: 0.40)
- Distancia a 2022630374: 0.6190 (Umbral: 0.40)
- Distancia a 2022630384: 0.6862 (Umbral: 0.40)
- Distancia a 2022630386: 0.6173 (Umbral: 0.40)
- Distancia a 2022630393: 0.7114 (Umbral: 0.40)
- Distancia a 2022630467: 0.5691 (Umbral: 0.40)
- Distancia a 2022630738: 0.8336 (Umbral: 0.40)
- 2022630082_old.png: Verdadera: 2022630082, Predicción FINAL: 2022630082 (Match único. Dist: 0.1567)

--- Comparando 2022630358_old.png con registros:
- Distancia a 2022630082: 0.8445 (Umbral: 0.40)
- Distancia a 2022630358: 0.1277 (Umbral: 0.40)
- Distancia a 2022630370: 0.7147 (Umbral: 0.40)
- Distancia a 2022630374: 0.6768 (Umbral: 0.40)
- Distancia a 2022630384: 0.7291 (Umbral: 0.40)
- Distancia a 2022630386: 0.6023 (Umbral: 0.40)
- Distancia a 2022630393: 0.7533 (Umbral: 0.40)
- Distancia a 2022630467: 0.6102 (Umbral: 0.40)
- Distancia a 2022630738: 0.9211 (Umbral: 0.40)
- 2022630358_old.png: Verdadera: 2022630358, Predicción FINAL: 2022630358 (Match único. Dist: 0.1277)

--- Comparando 2022630370_old.png con registros:
- Distancia a 2022630082: 0.4136 (Umbral: 0.40)
- Distancia a 2022630358: 0.6104 (Umbral: 0.40)
- Distancia a 2022630370: 0.1363 (Umbral: 0.40)
- Distancia a 2022630374: 0.7352 (Umbral: 0.40)
- Distancia a 2022630384: 0.4871 (Umbral: 0.40)
- Distancia a 2022630386: 0.6258 (Umbral: 0.40)
- Distancia a 2022630393: 0.7106 (Umbral: 0.40)
- Distancia a 2022630467: 0.6417 (Umbral: 0.40)
- Distancia a 2022630738: 0.5271 (Umbral: 0.40)
- 2022630370_old.png: Verdadera: 2022630370, Predicción FINAL: 2022630370 (Match único. Dist: 0.1363)
```

Figura 7.28: Captura de pantalla de la ejecución de la matriz de confusión.

```

--- Comparando 2022630374_old.png con registros:
- Distancia a 2022630082: 0.5581 (Umbral: 0.40)
- Distancia a 2022630358: 0.6085 (Umbral: 0.40)
- Distancia a 2022630370: 0.6221 (Umbral: 0.40)
- Distancia a 2022630374: 0.1743 (Umbral: 0.40)
- Distancia a 2022630384: 0.4767 (Umbral: 0.40)
- Distancia a 2022630386: 0.3861 (Umbral: 0.40)
- Distancia a 2022630393: 0.6227 (Umbral: 0.40)
- Distancia a 2022630467: 0.5849 (Umbral: 0.40)
- Distancia a 2022630738: 0.8091 (Umbral: 0.40)
- 2022630374_old.png: Verdadera: 2022630374, Predicción FINAL: 2022630374 (MÚLTIPLES MATCHES <= Umbral. Candidatos: 2022630374 (dist: 0.1743), 2022630386 (dist: 0.3861))

--- Comparando 2022630384_old.png con registros:
- Distancia a 2022630082: 0.6705 (Umbral: 0.40)
- Distancia a 2022630358: 0.6359 (Umbral: 0.40)
- Distancia a 2022630370: 0.5162 (Umbral: 0.40)
- Distancia a 2022630374: 0.5832 (Umbral: 0.40)
- Distancia a 2022630384: 0.2502 (Umbral: 0.40)
- Distancia a 2022630386: 0.4772 (Umbral: 0.40)
- Distancia a 2022630393: 0.4746 (Umbral: 0.40)
- Distancia a 2022630467: 0.5427 (Umbral: 0.40)
- Distancia a 2022630738: 0.5250 (Umbral: 0.40)
- 2022630384_old.png: Verdadera: 2022630384, Predicción FINAL: 2022630384 (Match único. Dist: 0.2502)

--- Comparando 2022630386_old.png con registros:
- Distancia a 2022630082: 0.6303 (Umbral: 0.40)
- Distancia a 2022630358: 0.6020 (Umbral: 0.40)
- Distancia a 2022630370: 0.6866 (Umbral: 0.40)
- Distancia a 2022630374: 0.7001 (Umbral: 0.40)
- Distancia a 2022630384: 0.7678 (Umbral: 0.40)
- Distancia a 2022630386: 0.1504 (Umbral: 0.40)
- Distancia a 2022630393: 0.6568 (Umbral: 0.40)
- Distancia a 2022630467: 0.4827 (Umbral: 0.40)
- Distancia a 2022630738: 0.6954 (Umbral: 0.40)
- 2022630386_old.png: Verdadera: 2022630386, Predicción FINAL: 2022630386 (Match único. Dist: 0.1504)

```

Figura 7.29: Captura de pantalla de la ejecución de la matriz de confusión.

```

--- Comparando 2022630393_old.png con registros:
- Distancia a 2022630082: 0.8990 (Umbral: 0.40)
- Distancia a 2022630358: 0.8255 (Umbral: 0.40)
- Distancia a 2022630370: 0.6467 (Umbral: 0.40)
- Distancia a 2022630374: 0.6631 (Umbral: 0.40)
- Distancia a 2022630384: 0.5405 (Umbral: 0.40)
- Distancia a 2022630386: 0.6784 (Umbral: 0.40)
- Distancia a 2022630393: 0.1500 (Umbral: 0.40)
- Distancia a 2022630467: 0.6631 (Umbral: 0.40)
- Distancia a 2022630738: 0.7151 (Umbral: 0.40)
- 2022630393_old.png: Verdadera: 2022630393, Predicción FINAL: 2022630393 (Match único. Dist: 0.1500)

--- Comparando 2022630467_old.png con registros:
- Distancia a 2022630082: 0.5893 (Umbral: 0.40)
- Distancia a 2022630358: 0.6530 (Umbral: 0.40)
- Distancia a 2022630370: 0.6478 (Umbral: 0.40)
- Distancia a 2022630374: 0.6874 (Umbral: 0.40)
- Distancia a 2022630384: 0.6456 (Umbral: 0.40)
- Distancia a 2022630386: 0.5397 (Umbral: 0.40)
- Distancia a 2022630393: 0.6688 (Umbral: 0.40)
- Distancia a 2022630467: 0.1914 (Umbral: 0.40)
- Distancia a 2022630738: 0.6217 (Umbral: 0.40)
- 2022630467_old.png: Verdadera: 2022630467, Predicción FINAL: 2022630467 (Match único. Dist: 0.1914)

--- Comparando 2022630738_old.png con registros:
- Distancia a 2022630082: 0.8023 (Umbral: 0.40)
- Distancia a 2022630358: 0.7853 (Umbral: 0.40)
- Distancia a 2022630370: 0.6037 (Umbral: 0.40)
- Distancia a 2022630374: 0.9407 (Umbral: 0.40)
- Distancia a 2022630384: 0.6508 (Umbral: 0.40)
- Distancia a 2022630386: 0.7270 (Umbral: 0.40)
- Distancia a 2022630393: 0.7980 (Umbral: 0.40)
- Distancia a 2022630467: 0.5855 (Umbral: 0.40)
- Distancia a 2022630738: 0.2888 (Umbral: 0.40)
- 2022630738_old.png: Verdadera: 2022630738, Predicción FINAL: 2022630738 (Match único. Dist: 0.2888)

```

Figura 7.30: Captura de pantalla de la ejecución de la matriz de confusión.

Evaluación completada.

--- Reporte de Clasificación ---

	precision	recall	f1-score	support
2022630082	1.00	1.00	1.00	1
2022630358	1.00	1.00	1.00	1
2022630370	1.00	1.00	1.00	1
2022630374	1.00	1.00	1.00	1
2022630384	1.00	1.00	1.00	1
2022630386	1.00	1.00	1.00	1
2022630393	1.00	1.00	1.00	1
2022630467	1.00	1.00	1.00	1
2022630738	1.00	1.00	1.00	1
desconocido	0.00	0.00	0.00	0
accuracy			1.00	9
macro avg	0.90	0.90	0.90	9
weighted avg	1.00	1.00	1.00	9

Proceso de evaluación finalizado.

Figura 7.31: Reporte final para la matriz de confusión.

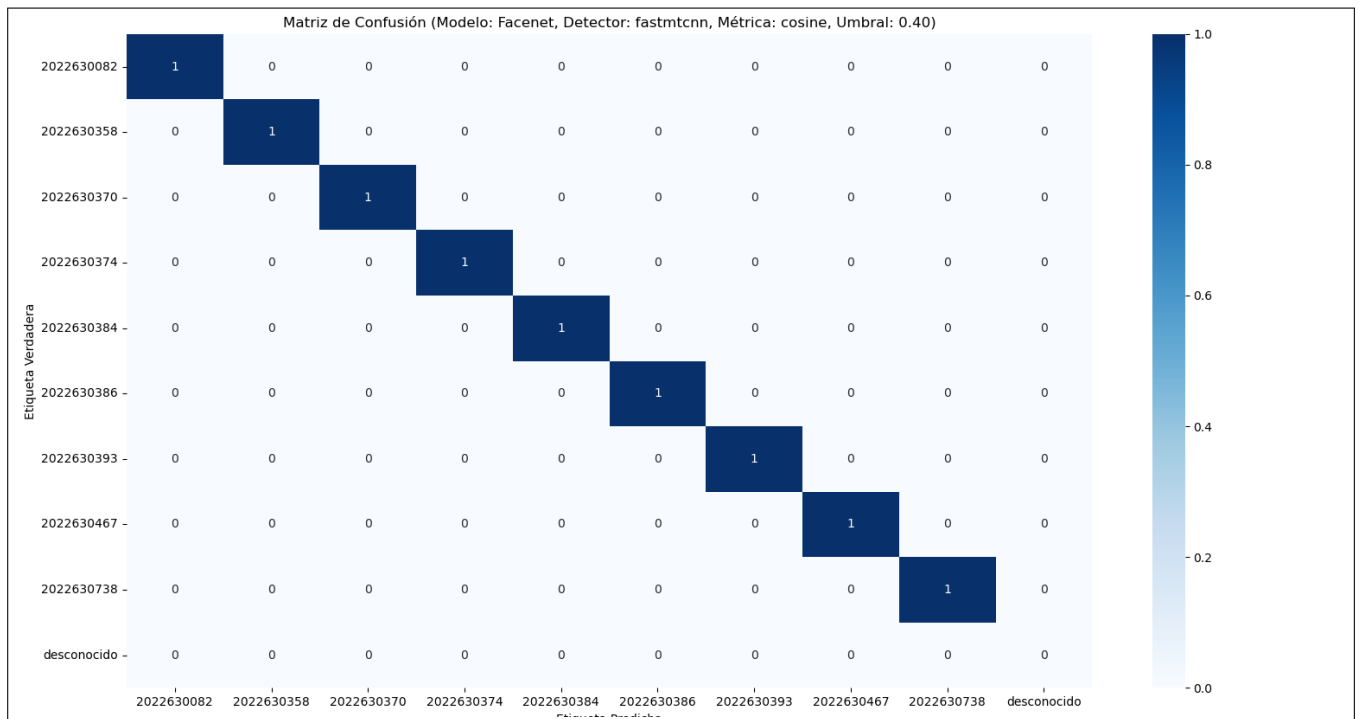


Figura 7.32: Matriz de confusión final.

El presente Trabajo Terminal ha culminado con la implementación exitosa de un sistema funcional, validado mediante pruebas rigurosas y la retroalimentación directa de los usuarios. Las funcionalidades desarrolladas han alcanzado los objetivos planteados inicialmente, estableciendo una base sólida para la solución propuesta. Sin embargo, el potencial de crecimiento y mejora del sistema es considerable.

Aunque el alcance de este Trabajo Terminal se ha completado, y la fase de desarrollo académico llega a su fin, se han identificado diversas áreas que representan direcciones futuras significativas en caso de que se presente la oportunidad de continuar el desarrollo del proyecto fuera del entorno escolar. Estas futuras implementaciones tienen como objetivo expandir las capacidades del sistema, optimizar la experiencia del usuario y escalar su impacto.

Las líneas de trabajo consideradas para una eventual continuación del proyecto incluyen:

- **Implementación de canales seguros (HTTPS) para Intercambio de Datos Sensibles:** Se considera fundamental fortalecer la seguridad de la comunicación mediante la implementación de canales seguros utilizando el protocolo HTTPS en todas las interacciones que involucren el intercambio de datos sensibles. Esto garantizará la confidencialidad e integridad de la información transmitida entre la aplicación móvil y los servidores, protegiéndola contra interceptaciones y manipulaciones no autorizadas.
- **Expansión del alcance a otras escuelas del IPN:** Una de las proyecciones más ambiciosas es la posibilidad de extender la implementación del sistema a otras escuelas del Instituto Politécnico Nacional. Esto implicaría un estudio detallado de la viabilidad, la adaptación a los requisitos específicos de cada institución y la escalabilidad de la arquitectura actual para soportar una base de usuarios y datos mucho más amplia, manteniendo los estándares de eficiencia y seguridad.
- **Implementación de mejoras gráficas y optimización de la Interfaz de Usuario:** Basándonos en la valiosa retroalimentación obtenida a través de las encuestas y las sesiones de prueba con los usuarios, se contempla la aplicación de mejoras sustanciales en la interfaz gráfica de usuario (GUI) y la experiencia de usuario (UX). Esto abarcaría específicamente la mejora de la visibilidad de los iconos, el

reacomodo de las palabras para una lectura más clara y la optimización de la organización general de los elementos en pantalla, buscando una aplicación más intuitiva, moderna y atractiva para todos los perfiles de usuario.

- **Integración de módulo de ubicación (mapa de salones de ETS):** Atendiendo a las recomendaciones y sugerencias de los usuarios en las encuestas, se plantea la integración de un módulo de mapa interactivo. Este módulo tendría como objetivo principal indicar la ubicación exacta de los salones donde se llevarán a cabo los Exámenes a Título de Suficiencia (ETS), facilitando a los alumnos y docentes la localización de las aulas dentro de la institución y mejorando la logística de los eventos.
- **Reimplementación del módulo de notificaciones:** Aunque una funcionalidad de notificación ha sido integrada, se visualiza una reimplementación y optimización profunda del módulo de notificaciones. El objetivo sería desarrollar un sistema de alertas más robusto, personalizable y eficiente, que permita mantener a los usuarios (tanto alumnos como docentes) informados de manera proactiva sobre cambios, actualizaciones o eventos importantes relacionados con los Exámenes a Título de Suficiencia (ETS) o cualquier otra funcionalidad relevante, explorando la integración con servicios de notificación push avanzados y diversificación de canales.

Estas posibles líneas de desarrollo futuras representarían una evolución natural del proyecto, consolidando la funcionalidad existente y permitiendo que el sistema alcance un mayor grado de madurez y un impacto más amplio en la comunidad académica del IPN, más allá de los objetivos iniciales de este Trabajo Terminal.

- [1] IPN, "Título profesional ipn." <https://www.ipn.mx/dae/tramites/título-profesional.html>, 2023. Último acceso: 20 de noviembre de 2024.
- [2] IPN, "Evaluación a título de suficiencia." <https://www.cecyl18.ipn.mx/gestionescolar/evaluaciontitulo.html>, 2023. Último acceso: 19 de noviembre de 2024.
- [3] UNAM, "Seguridad escolar." https://www.dgire.unam.mx/webdgire/contenido_wp/documentos/seguridadescolar/, 2022. Último acceso: 20 de Noviembre del 2024.
- [4] M. A. N. Selwyn, "Facial recognition technology in schools: critical questions and concerns." [10.1080/17439884.2020.1686014](https://doi.org/10.1080/17439884.2020.1686014), 2019. Último acceso: 20 de Noviembre del 2024.
- [5] J. M. H. Vázquez, "Inseguridad escolar y problemas académicos en una universidad pública mexicana." <https://www.redalyc.org/journal/5216/521665144011/html/>, 2021. Último acceso: 20 de Noviembre del 2024.
- [6] CSIS, "How accurate are facial recognition systems and why does it matter?." <https://www.csis.org/blogs/strategic-technologies-blog/how-accurate-are-facial-recognition-systems-and-why-does-it>. Último acceso: 20 de Noviembre del 2024.
- [7] M. Mahboob and N. Mohsin, "Attendance management system using face recognition," 10 2024.
- [8] J. R. A. T. Dr. S. Balaji, Mrs. S. Sugashini and A. Vaishnavi³, "Automated student monitoring and attendance system with faster r cnn," 05 2025.
- [9] F. I. P. TORRES, "Sistema web con reconocimiento facial para la escuela "unidad educativa ulpiano navarro," Agosto 2021.
- [10] M. A. . N. Selwyn, "Facial recognition technology in schools: critical questions and concerns," *Learning, Media and Technology*, vol. 45, no. 2, pp. 115–128, 2019.

- [11] I. Ahmad, F. Alqurashi, E. Abozinadah, and R. Mehmood, "A novel deep learning-based online proctoring system using face recognition, eye blinking, and object detection techniques," *International Journal of Advanced Computer Science and Applications*, vol. 12, 01 2021.
- [12] A. S. A. S. S. K. Akshat Shrivastava, Ashray Parmar, "Face recognition classification in proctored exam," *INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT)*, vol. 09, 12 2020.
- [13] H. S. y. S. G. Galindo, D., "Reconocimiento facial para la identificación de los alumnos en exámenes finales en la modalidad presencial de la universidad continental - huancayo," 2021.
- [14] M. Ejaz, S. Debnath, M. K. Hasan, and M. M. Alam, "Facial recognition-based entry system for student residence halls: Enhancing security and accessibility," *Asian Journal of Research in Computer Science*, vol. 16, pp. 344–353, 12 2023.
- [15] E. Vásquez, M. Huerta, R. Clotet, M. Astronomy, and J.-I. Castillo-Velazquez, "Facial recognition system for access control through the application of convolutional neural networks," 04 2022.
- [16]
- [17] W. Mei and W. Deng, "Deep face recognition: A survey," *Neurocomputing*, vol. 429, 04 2018.
- [18] H. Imaoka, H. Hashimoto, K. Takahashi, A. F. Ebihara, J. Liu, A. Hayasaka, Y. Morishita, and K. Sakurai, "The future of biometrics technology: from face recognition to related applications," *APSIPA Transactions on Signal and Information Processing*, vol. 10, p. e9, 2021.
- [19] J. L. Bustos, "Qué es jetpack compose." <https://keepcoding.io/blog/que-es-jetpack-compose/>, 25 de abril del 2024. Último acceso: 10 de noviembre de 2024.
- [20] EMMA, "Tipos de aplicaciones, características, ejemplos y comparativa | emma".|. <https://emma.io/blog/tipos-aplicaciones-caracteristicas-ejemplos/>, (s.f). Último acceso: 28 de noviembre de 2024.
- [21] L. Nunez, "Tipos de aplicaciones, características, ejemplos y comparativas." <https://emma.io/blog/>, 31 de enero del 2023. Último acceso: 10 de noviembre de 2024.
- [22] P. Concepts, "¿qué es kotlin y para qué sirve?." <https://www.plainconcepts.com/es/kotlin-android/>, 5 de septiembre del 2023. Último acceso: 18 de noviembre de 2024.
- [23] EduTools, "Android studio | edutools." <https://edutools.tec.mx/es/colecciones/tecnologias/android-studio>, (s. f.). Último acceso: 18 de noviembre de 2024.
- [24] Y. Muradas, "Qué es gradle: La herramienta para ser más productivo desarrollando." <https://openwebinars.net/blog/que-es-gradle/>, 25 de febrero del 2020. Último acceso: 18 de noviembre de 2024.
- [25] J. A. Saavedra, "Qué es github y para qué sirve: una guía para principiantes." <https://ebac.mx/blog/que-es-github>, 4 de junio del 2023. Último acceso: 18 de noviembre de 2024.

- [26] Microsoft, "Conceptos básicos sobre bases de datos - soporte técnico de microsoft." <https://support.microsoft.com/es-es/topic/conceptos-b%C3%A1sicos-sobre-bases-de-datos-a849ac16-07c7-4a31-9948-3c8c94a7c204>. Último acceso: 15 de noviembre de 2024.
- [27] B. V. y B. V, "Qué es un sgbd: Guía completa sobre los sistemas de gestión de bases de datos." <https://www.hostinger.mx/tutoriales/sgbd>, 2 de mayo de 2023. Último acceso: 15 de noviembre de 2024.
- [28] Oracle, "What is a database." <https://www.oracle.com/mx/database/what-is-database/>, 24 de noviembre de 2020. Último acceso: 15 de noviembre de 2024.
- [29] Maldeadora, "Bases de datos: qué tipos existen y cómo funcionan." <https://platzi.com/blog/bases-de-datos-que-son-que-tipos-existen/>, 24 de noviembre de 2020. Último acceso: 15 de noviembre de 2024.
- [30] J. Larque, "Tipos de bases de datos: cuáles hay y por qué es importante elegirlos bien." <https://www.hiberus.com/crecemos-contigo/tipos-de-bases-de-datos-cuales-hay-y-por-que-es-importante-elegirlos-bien>, 4 de junio de 2024. Último acceso: 15 de noviembre de 2024.
- [31] Arsys, "Tipos de bases de datos que existen." <https://www.arsys.es/blog/tipos-de-bases-de-datos-que-existen>. Último acceso: 15 de noviembre de 2024.
- [32] IBM, "¿qué es postgresql? | ibm." <https://www.ibm.com/mx-es/topics/postgresql>, 2 de octubre de 2023. Último acceso: 15 de noviembre de 2024.
- [33] Instaclustr, "Vector databases explained: Use cases, algorithms and key features," 4 2025.
- [34] IBM, "¿qué es java spring boot? | ibm." <https://www.ibm.com/mx-es/topics/java-spring-boot>, 17 de julio de 2023. Último acceso: 20 de noviembre de 2024.
- [35] R. Maldonado, "Spring boot: Herramienta para desarrollar aplicaciones web." <https://keepcoding.io/blog/que-es-spring-boot/>, 13 de septiembre de 2024. Último acceso: 20 de noviembre de 2024.
- [36] J. M. Alarcón, "La api de persistencia de java: ¿qué es jpa? - jpa vs hibernate vs eclipselink vs spring jpa - campusmvp.es." <https://www.campusmvp.es/recursos/post/la-api-de-persistencia-de-java-que-es-jpa-jpa-vs-hibernate-vs-eclipselink-vs-spring-jpa.aspx>. Último acceso: 20 de noviembre de 2024.
- [37] IBM, "Java persistence api (jpa)." <https://www.ibm.com/docs/es/was-liberty/nd?topic=liberty-java-persistence-api-jpa>, 30 de enero de 2024. Último acceso: 20 de noviembre de 2024.
- [38] H. Dhaduk, "10 software architecture patterns you must know about." <https://www.simform.com/blog/software-architecture-patterns/>, 2023. Último acceso: 20 de Octubre del 2024.
- [39] W. Vicky, "14 software architecture design patterns to know." <https://www.redhat.com/en/blog/14-software-architecture-patterns>, 2022. Último acceso: 20 de Octubre del 2024.

- [40] GeeksforGeeks, “Software architectural patterns in system design.” <https://www.geeksforgeeks.org/design-patterns-architecture/>, 2024. Último acceso: 20 de Octubre del 2024.
- [41] D. S. Carter, “Tensorflow — neural network playground.” <https://playground.tensorflow.org/>, 2022. Último acceso: 21 de Octubre del 2024.
- [42] IBM, “What is a neural network?.” <https://www.ibm.com/topics/neural-networks>, 2023. Último acceso: 22 de Octubre del 2024.
- [43] GeeksforGeeks, “What is a neural network?.” <https://www.geeksforgeeks.org/neural-networks-a-beginners-guide/>, 2024. Último acceso: 22 de Octubre del 2024.
- [44] S. Rawat, “Advantages and disadvantages of neural networks | analytics steps.” <https://www.analyticssteps.com/blogs/advantages-and-disadvantages-neural-networks>, 2022. Último acceso: 22 de Octubre del 2024.
- [45] V. Gupta, “Face recognition for beginners - nearly everything you need to know.” <https://learnopencv.com/face-recognition-an-introduction-for-beginners/>, 2022. Último acceso: 24 de Octubre del 2024.
- [46] S. Chandhok, “Face recognition with siamese networks, keras, and tensorflow - pyimagesearch.” <https://pyimagesearch.com/2023/01/09/face-recognition-with-siamese-networks-keras-and-tensorflow/>, 2023. Último acceso: 24 de Octubre del 2024.
- [47] Varun, “What is face detection? ultimate guide 2023 + model comparison.” <https://learnopencv.com/what-is-face-detection-the-ultimate-guide/>, 2023. Último acceso: 24 de Octubre del 2024.
- [48] P. Durai and P. Durai, “Face recognition models: Advancements, toolkit, and datasets,” 12 2023.
- [49] B. Kromydas, “Convolutional neural network: A complete guide.” <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>, 2024. Último acceso: 25 de Octubre del 2024.
- [50] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, “Joint face detection and alignment using multitask cascaded convolutional networks,” *IEEE Signal Processing Letters*, vol. 23, pp. 1499–1503, Oct 2016.
- [51] F. Schroff, D. Kalenichenko, and J. Philbin, “Facenet: A unified embedding for face recognition and clustering,” June 2015.
- [52] D. Yi, Z. Lei, S. Liao, and S. Z. Li, “Learning face representation from scratch,” 2014.
- [53] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, “Labeled faces in the wild: A database for studying face recognition in unconstrained environments,” October 2007.